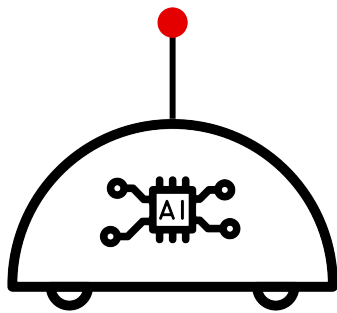




universidad
de león

Escuela de Ingenierías
Industrial, Informática y Aeroespacial

MÁSTER EN ROBÓTICA E INTELIGENCIA ARTIFICIAL

Trabajo de Fin de Máster

Aprendizaje por imitación en la apertura de
válvulas con un humanoide Unitree G1

Imitation Learning for Valve Opening with a
Unitree G1 Humanoid

Autor: Izan Viñes Castaño

Tutor Académico: Francisco Javier Rodríguez Lera

(Septiembre, 2026)

UNIVERSIDAD DE LEÓN Departamento de Ingenierías Mecánica, Informática y Aeroespacial MÁSTER EN ROBÓTICA E INTELIGENCIA ARTIFICIAL Trabajo de Fin de Máster	
ALUMNO: Izan Viñes Castaño	
TUTOR: Francisco Javier Rodríguez Lera	
TÍTULO: Aprendizaje por imitación en la apertura de válvulas con un humanoide Unitree G1: modelos Visión-Lenguaje-Acción frente a políticas clásicas	
TITLE: Imitation Learning for Valve Opening with a Unitree G1 Humanoid: Vision-Language-Action Models versus Classical Policies	
CONVOCATORIA: Septiembre, 2026	
RESUMEN: Este trabajo compara dos familias de aprendizaje por imitación sobre una misma tarea de manipulación industrial: la apertura de válvulas de volante mediante el humanoide Unitree G1, íntegramente en simulación. La tarea se ha construido sobre Isaac Lab Arena, con una válvula procedente de un modelo CAD real y con el robot equilibrado por una política de control de cuerpo completo. Se han grabado cien demostraciones teleoperadas con unas gafas PICO 4 Ultra, con la válvula en dos disposiciones con posiciones aleatorizadas en cada reinicio, y renderizando un fondo fotorrealista a posteriori de las grabaciones. El mismo conjunto de datos alimenta los dos métodos comparados: el ajuste fino del modelo visión-lenguaje-acción NVIDIA GR00T N1.7 y una política ACT de inferior capacidad. Ambas se evalúan en el mismo simulador y sobre condiciones iniciales idénticas, lo que habilita un contraste pareado. Sobre doscientos rollouts, GR00T alcanza un 92 % de éxito frente al 86 % de ACT (McNemar exacto, $p = 0,047$), y treinta y seis de los treinta y siete casos discordantes corresponden a la disposición más difícil. La curva de eficiencia en datos resulta plana: la limitación dominante no es la cantidad de demostraciones, sino su composición.	
Palabras clave: aprendizaje por imitación, modelos visión-lenguaje-acción, robot humanoide, manipulación robótica, simulación, teleoperación, reconstrucción 3D.	
Firma del alumno:	VºBº Tutor:

Declaración de uso de Inteligencia Artificial

En cumplimiento de las directrices aplicables al uso de herramientas de inteligencia artificial generativa en el Trabajo Fin de Máster del **Máster Universitario en Robótica e Inteligencia Artificial**, se declara de forma explícita el alcance con el que estas herramientas han sido empleadas durante la realización del presente trabajo.

Herramienta utilizada

Se ha empleado el asistente de inteligencia artificial generativa **Claude** (Anthropic) como herramienta de apoyo durante distintas fases del desarrollo y documentación del proyecto.

Alcance del uso

El uso de la herramienta se ha limitado a tareas de apoyo realizadas bajo la dirección, supervisión y posterior revisión del autor, entre las que se incluyen:

- **Revisión y corrección de código $\text{\LaTeX}$:** identificación y corrección de errores de sintaxis, así como apoyo en la estructuración y formato del documento, incluyendo tablas, figuras, referencias y comandos $\text{\LaTeX}$.
- **Generación de material gráfico de apoyo:** asistencia en la elaboración de diagramas, esquemas y otras representaciones visuales a partir de los requisitos, información técnica e indicaciones proporcionadas por el autor.
- **Asistencia en la implementación de código:** apoyo en la implementación, revisión y depuración de funciones concretas de los módulos desarrollados, siguiendo los requisitos, la arquitectura y el comportamiento especificados por el

autor. El código generado o modificado con asistencia de IA ha sido posteriormente revisado y validado por el autor antes de su incorporación al sistema.

Responsabilidad y autoría

El planteamiento del problema, la definición de requisitos, el diseño de la solución y de la arquitectura software, las decisiones técnicas adoptadas, el diseño y ejecución de los experimentos, el análisis de los resultados y la elaboración de las conclusiones son responsabilidad del autor del Trabajo Fin de Máster.

Las herramientas de inteligencia artificial se han utilizado como instrumentos de apoyo y no como sustituto del proceso de diseño, experimentación o evaluación desarrollado por el autor. Todas las aportaciones obtenidas mediante estas herramientas han sido revisadas, adaptadas y, cuando ha sido necesario, verificadas antes de su incorporación al trabajo.

El autor asume la responsabilidad sobre el contenido de la memoria, el código desarrollado, los resultados presentados y las conclusiones obtenidas, con independencia de las herramientas empleadas como apoyo durante su elaboración.

Índice general

Declaración de uso de Inteligencia Artificial	II
Índice de figuras	VII
Índice de tablas	VIII
Glosario de términos	X
Introducción	1
1. Estudio del problema	6
1.1. El contexto del problema	6
1.1.1. La operación de válvulas en instalaciones de proceso	6
1.1.2. Robots humanoides como plataforma de intervención	7
1.1.3. Por qué el trabajo se realiza en simulación	8
1.2. El estado de la cuestión	9
1.2.1. Aprendizaje por imitación	9
1.2.2. Políticas de manipulación entrenadas desde cero	10
1.2.3. Modelos visión-lenguaje-acción	11
1.2.4. GR00T y el control de humanoides	12
1.2.5. Entornos de simulación y formatos de conjuntos de datos	12
1.2.6. Teleoperación en realidad mixta	13
1.2.7. Realismo visual mediante reconstrucción de escenas	14
1.3. La definición del problema	15
1.4. Normativa asociada	16
2. Gestión de proyecto software	19
2.1. Alcance del proyecto	19
2.1.1. Definición del proyecto	19
2.1.2. Estimación de tareas y recursos	20

2.1.3. Presupuesto	21
2.2. Plan de trabajo	23
2.2.1. Identificación de tareas	23
2.2.2. Estimación de tareas	24
2.2.3. Planificación de tareas	25
2.3. Gestión de recursos	25
2.3.1. Especificación de recursos	25
2.3.2. Asignación de recursos	26
2.4. Gestión de riesgos	26
2.4.1. Identificación de riesgos	26
2.4.2. Análisis de riesgos	27
3. Solución	30
3.1. Descripción de la solución	30
3.2. El proceso de desarrollo	31
3.2.1. Análisis	32
3.2.2. Diseño	34
3.2.3. Implementación	42
3.2.4. Pruebas	45
3.3. El producto del desarrollo	47
4. Evaluación	49
4.1. Proceso de evaluación	49
4.1.1. Forma de evaluación	49
4.1.2. Casos de prueba	51
4.2. Análisis de resultados	52
4.2.1. Convergencia del entrenamiento	52
4.2.2. Comparación principal	54
4.2.3. Distribución de los fallos	57
4.2.4. Eficiencia en datos	59
4.2.5. Latencia de inferencia	61
4.2.6. Amenazas a la validez	62
Conclusión	65
Lista de referencias	72
A. Control de versiones	76
A.1. Organización del repositorio	76

A.2. Los cambios sobre el marco de simulación	77
A.3. Historial de desarrollo	79
B. Seguimiento de proyecto fin de máster	80
B.1. Forma de seguimiento	80
B.2. Planificación inicial	81
B.3. Planificación final	82
C. Protocolo de evaluación	84
C.1. Condiciones de la evaluación	84
C.2. Procedimiento	84
C.3. Registro de resultados	86
C.4. Análisis estadístico	87
D. Reconstrucción fotorrealista mediante Gaussian Splatting	88
D.1. El activo y su integración en la escena	88
D.2. El procedimiento de reproducción sobre demostraciones grabadas . .	90
D.3. La divergencia de estado y su corrección	91
D.4. Parámetros y configuración	92

Índice de figuras

2.1. Planificación temporal del proyecto	25
2.2. Matriz de riesgos	28
3.1. <i>pipeline</i> completo del sistema	31
3.2. Arquitectura del sistema	35
3.3. Las dos disposiciones de la válvula	38
3.4. Variabilidad de las dimensiones del espacio de acción	40
3.5. Reparto de disposiciones por sesión	42
3.6. Duración de las demostraciones	43
3.7. Escena de captura y escena del conjunto de datos	45
3.8. Vista de la cámara del robot	48
4.1. Convergencia del entrenamiento	53
4.2. Tasa de éxito de ambas políticas	54
4.3. Secuencia de una tirada de evaluación	55
4.4. Tasa de éxito por disposición de la válvula	56
4.5. Distribución del ángulo final alcanzado	58
4.6. Curva de eficiencia en datos	60
4.7. Coste de inferencia frente al presupuesto de tiempo real	61

Índice de tablas

1.1. Normativa aplicable a un despliegue del sistema	18
2.1. Presupuesto de personal	21
2.2. Equipamiento empleado y coste imputado	22
2.3. Presupuesto total	23
2.4. Tareas del proyecto y dependencias	24
2.5. Catálogo de riesgos	27
2.6. Análisis de riesgos: valoración, mitigación y contingencia	29
3.1. Requisitos funcionales	33
3.2. Requisitos no funcionales	34
3.3. Tecnologías empleadas y versiones	36
3.4. Composición del espacio de acción de veintitrés dimensiones	39
3.5. Pruebas de verificación del <i>pipeline</i>	46
4.1. Parámetros fijos de toda evaluación	50
4.2. Resultado sobre doscientas tiradas por política	54
4.3. Tabla de contingencia emparejada sobre las doscientas tiradas	55
4.4. Pasos hasta cruzar el umbral de éxito, en los episodios con éxito	57
4.5. Caracterización de los episodios fallidos	58
4.6. Tasa de éxito frente al número de demostraciones, cien tiradas por punto	59
4.7. Latencia de inferencia por bloque de cuarenta acciones, medida en lazo cerrado	61
A.1. Organización del repositorio entregado	77
A.2. Parches aplicados sobre el marco de simulación	78
B.1. Desviaciones entre la planificación inicial y la final	83
C.1. Condiciones operativas de toda tirada de evaluación	85

C.2. Campos registrados por episodio	86
D.1. Parámetros del fondo fotorrealista y versiones de las herramientas . .	92

Glosario de términos

Catálogo de términos específicos del contexto del trabajo. Se recogen tanto los conceptos de aprendizaje automático como la terminología de simulación y de control de robots humanoides que aparece a lo largo de la memoria, así como los nombres propios de las herramientas empleadas y los contrastes estadísticos con los que se analizan los resultados.

ACT (*Action Chunking with Transformers*) : Política de imitación basada en un autocodificador variacional condicional que predice bloques de acciones futuras en lugar de una acción aislada.

AGILE : Política de control de cuerpo completo (aunque también con posibilidad de desacoplar torso inferior y torso superior), desarrollada por NVIDIA e implementada como red neuronal en formato ONNX, encargada de mantener el equilibrio del humanoide durante la manipulación.

Ajuste fino (*fine-tuning*) : Reentrenamiento de un modelo ya preentrenado sobre los datos de una tarea concreta, partiendo de sus pesos en lugar de una inicialización aleatoria. Es la vía por la que GR00T se adapta a la tarea de este trabajo.

Apertura (*openness*) : Magnitud normalizada entre 0 y 1 que expresa cuánto se ha abierto un elemento articulado, calculada sobre los límites de su articulación. En este trabajo, una apertura de 0,5 equivale a media vuelta del volante y es el umbral de éxito.

Aprendizaje por imitación (*Imitation learning, IL*) : Familia de técnicas en las que una política de control se obtiene a partir de demostraciones de un experto, en lugar de mediante una función de recompensa definida de forma explícita.

Behavior cloning : Enfoque más directo del aprendizaje por imitación: se ajusta un modelo supervisado que asocia observaciones a acciones sobre el conjunto de demostraciones, sin interacción adicional con el entorno.

Chunk de acciones : Bloque de acciones consecutivas que la política emite de una sola inferencia. En este trabajo el bloque es de 40 pasos, que a 50 Hz equivalen a 0,8 segundos de tiempo de robot.

Cinemática inversa (IK) : Cálculo de las posiciones articulares necesarias para que un extremo del robot alcance una pose dada. Es el problema que resuelve Pink IK para llevar las muñecas a donde las coloca el operador.

CVAE (*Conditional Variational Autoencoder*) : Autocodificador variacional condicionado, arquitectura sobre la que se construye ACT.

Deriva de distribución (*distribution shift*) : Es el problema característico del aprendizaje por imitación: al ejecutarse, la política visita estados que no aparecían en las demostraciones, donde sus errores se acumulan y la alejan aún más de aquello que sabe hacer.

Diffusion Policy : Política de imitación que genera las acciones mediante un proceso de difusión. Es la alternativa a ACT dentro de la familia de métodos entrenados desde cero.

Disposición cenital : Colocación de la válvula con el volante mirando hacia arriba y el eje de giro vertical, correspondiente a una válvula montada sobre una línea horizontal. El robot la alcanza por encima.

Disposición frontal : Colocación de la válvula con el volante encarado al robot y el eje de giro horizontal, correspondiente a una válvula sobre una línea vertical. Se gira como el timón de un barco.

DoF (*Degrees of Freedom*) : Grados de libertad. Número de articulaciones independientes de un mecanismo.

Embodiment : Descripción del cuerpo del robot que una política asume: qué articulaciones tiene, en qué orden y cómo se le envían las órdenes. Un mismo modelo puede servir a varios *embodiments*.

Episodio : Cada ejecución de la tarea desde el reinicio del entorno hasta que se alcanza el éxito o se agota el presupuesto de tiempo asignado. Un episodio que se agota por tiempo se considera fallido.

End-to-end : Enfoque en el que un único modelo aprende directamente de la observación cruda a la acción de control, sin etapas intermedias diseñadas a mano ni módulos independientes de percepción y planificación. Es la propiedad que comparten GR00T y ACT, y la razón por la que ambos se sirven a través del mismo protocolo de evaluación.

Espacio de acción : Conjunto de magnitudes que la política emite en cada paso para gobernar al robot. En este trabajo tiene veintitrés dimensiones: el estado de las dos manos, la pose de ambas muñecas, las órdenes de locomoción, la altura del pelvis y la orientación del torso.

Flow matching : Técnica generativa que aprende un campo de velocidades capaz de transportar ruido hasta la distribución objetivo. Es el mecanismo con el que GR00T genera sus acciones.

Gaussian Splatting : Técnica de reconstrucción y representación de escenas tridimensionales mediante nubes de primitivas gaussianas, que permite renderizar en tiempo real y de manera fotorrealista un entorno capturado a partir de vídeo real, y contribuye así a reducir la brecha entre simulación y realidad.

GR00T : Familia de modelos fundacionales de NVIDIA para control de robots humanoides. La variante empleada aquí, GR00T N1.7, es un modelo visión-lenguaje-acción de 3140 millones de parámetros.

HDF5 : Formato de fichero jerárquico para conjuntos de datos científicos. Es el que produce el grabador de demostraciones del simulador.

Intervalo de Wilson : Método de cálculo del intervalo de confianza de una proporción que, a diferencia del intervalo normal, se mantiene dentro de $[0, 1]$ cuando la proporción se acerca a los extremos. Es el caso de las tasas de éxito de este trabajo, próximas al 90 %.

Isaac Lab Arena : Marco de trabajo de NVIDIA, construido sobre Isaac Lab e Isaac Sim, que compone escenas de manipulación a partir de un registro de recursos y ofrece la infraestructura de grabación y evaluación de políticas en su conjunto.

Latencia de inferencia : Tiempo que tarda la política en producir un bloque de acciones a partir de una observación. Se contrasta contra el presupuesto que impone el lazo de control.

Lazo de control : Ciclo de percepción y acción del robot. Aquí opera a 50 Hz y, como la política emite bloques de 40 pasos, dispone de 0,8 segundos para cada inferencia.

LeRobot : Biblioteca y formato de conjuntos de datos de *Hugging Face* para aprendizaje de políticas robóticas. Almacena los episodios como ficheros *parquet* más vídeo comprimido.

Modelo fundacional (*foundation model*) : Modelo de gran tamaño preentrenado sobre volúmenes masivos de datos heterogéneos y concebido para adaptarse después a tareas concretas mediante ajuste fino. GR00T es el representante de esta familia en el trabajo.

NuRec : Componente de Omniverse que permite incorporar reconstrucciones neuronales de escenas reales, en este trabajo la representación por *Gaussian Splatting* de una oficina.

ONNX (*Open Neural Network Exchange*) : Formato abierto de intercambio de redes neuronales, que permite ejecutar un modelo entrenado con una biblioteca desde otra distinta. Es el formato en el que se distribuye la política AGILE.

Pipeline : Secuencia de etapas encadenadas por las que pasan los datos desde la teleoperación hasta la política evaluada: grabación, conversión al formato de LeRobot, entrenamiento y evaluación. Cada etapa consume la salida de la anterior, de modo que un error silencioso en una etapa temprana invalida todo el trabajo posterior sin producir ningún síntoma visible.

Pink IK : Resolvedor de cinemática inversa por tareas ponderadas, empleado para convertir las poses de las muñecas que marca el operador, mediante un dispositivo de VR, en órdenes articulares para los brazos del robot en la simulación.

Política (*policy*) : Función que asocia a cada observación del robot la acción que debe ejecutar. Es el objeto que producen ambos métodos comparados y sobre el que se mide la tasa de éxito.

Preentrenamiento : Fase previa en la que un modelo aprende sobre un corpus amplio y ajeno a la tarea final. Es lo que distingue a un modelo fundacional de una arquitectura entrenada desde cero.

Prueba exacta de Fisher : Contraste de independencia entre dos variables categóricas sobre muestras pequeñas. Aquí se emplea para medir el efecto de la disposición de la válvula sobre la tasa de éxito.

Prueba exacta de McNemar : Contraste para datos emparejados dicotómicos. Emplea únicamente los casos discordantes —aquellos en los que una política acierta y la otra falla— y es el contraste apropiado para comparar dos políticas evaluadas sobre las mismas condiciones iniciales.

Retargeting : Traslación del movimiento capturado de un operador humano al cuerpo del robot, que tiene proporciones y articulaciones distintas.

Rollout : Ejecución completa de un episodio con una política ya entrenada, empleada para medir su tasa de éxito, es decir, cada una de las pruebas.

Semilla (*seed*) : Valor que inicializa el generador de números pseudoaleatorios. Fijarla hace reproducible el sorteo de las condiciones iniciales, y es lo que permite enfrentar a ambas políticas exactamente a las mismas escenas.

Sim2Real : Traslación al robot físico de una política entrenada en simulación, y pérdida de rendimiento que suele acompañarla por las diferencias entre el simulador y el mundo real.

Tasa de éxito : Proporción de episodios en los que la válvula supera media vuelta de apertura. Es la métrica principal del trabajo.

Teleoperación : Control directo del robot por parte de una persona, aquí mediante unas gafas de realidad mixta y sus mandos, con el propósito de generar las demostraciones para elaborar un *dataset*.

USD (*Universal Scene Description*) : Formato de descripción de escenas tridimensionales sobre el que se apoyan Omniverse e Isaac Sim.

VLA (*Vision-Language-Action*) : Modelo que recibe imágenes y una instrucción en lenguaje natural y produce directamente acciones de control.

WBC (*Whole-Body Control*) : Control de cuerpo completo. Conjunto de técnicas que coordinan todas las articulaciones del robot para mantener el equilibrio mientras se ejecuta una tarea con los brazos.

Introducción

Los robots humanoides han pasado en pocos años de ser plataformas de investigación costosas y frágiles a productos comerciales con un precio comparable al de un vehículo industrial. En paralelo, el aprendizaje de políticas de manipulación a partir de demostraciones humanas —el aprendizaje por imitación (*Imitation Learning*, IL)— se ha consolidado como una de las vías más prometedoras para dotar a estos sistemas de destreza motora, desplazando en gran medida a la programación explícita de trayectorias. No obstante, trasladar estas metodologías a una plataforma humanoide de alta dimensionalidad no se reduce a entrenar un modelo aislado; exige concebir e implantar un *pipeline* integral *end-to-end* que abarque desde la simulación de la tarea y la captura de datos mediante teleoperación, hasta el procesamiento del conjunto de datos, el entrenamiento de las redes y su despliegue en bucle cerrado para control en tiempo real.

El objetivo central de este Trabajo de Fin de Máster es, precisamente, el diseño, implementación y validación de dicho *pipeline* completo de aprendizaje por imitación sobre un robot humanoide (el Unitree G1) para la ejecución autónoma de una tarea concreta y representativa del entorno industrial: la apertura de una válvula de volante, desarrollada íntegramente en simulación. Para contrastar la viabilidad de este flujo y responder a las tendencias actuales de la disciplina, se han implementado y evaluado dos aproximaciones metodológicas y arquitectónicas radicalmente distintas dentro del mismo sistema: por un lado, una arquitectura clásica y dedicada de imitación entrenada desde cero (ACT), y por otro, la adaptación y ajuste de un modelo fundacional de gran tamaño visión-lenguaje-acción (VLA) preentrenado específicamente para robótica humanoide (NVIDIA GR00T).

La articulación de ambas aproximaciones sobre un mismo flujo de trabajo permite, además de validar el *pipeline* completo de ingeniería, responder de forma acotada y empírica a una pregunta clave en el estado actual de la técnica: cuando se dispone de un conjunto de demostraciones de una tarea industrial concreta, ¿compensa

asumir la complejidad y el coste computacional de un modelo fundacional de gran escala, o resulta suficiente una política clásica de imitación entrenada exclusivamente sobre los datos de la tarea?

Planteamiento del problema

En las instalaciones del sector del petróleo y el gas, la operación manual de válvulas de volante es una tarea rutinaria, físicamente exigente y a menudo situada en zonas clasificadas como atmósferas potencialmente explosivas. Es, por su naturaleza repetitiva y por el riesgo asociado al entorno, una de las candidatas naturales a la automatización mediante robots móviles de propósito general. Sin embargo, se trata de una tarea que resiste mal el enfoque clásico de programar la trayectoria: la válvula puede estar montada en cualquier orientación, el volante exige un agarre firme y un giro sostenido, y la reacción del mecanismo se propaga por los brazos hasta comprometer el equilibrio del robot.

El aprendizaje por imitación ofrece una alternativa. En lugar de describir cómo debe moverse el robot, se le muestra la tarea un cierto número de veces y se ajusta una política que reproduzca ese comportamiento. El coste se traslada entonces de la ingeniería de la trayectoria a la recogida de demostraciones, que en el caso de un humanoide de veintinueve grados de libertad no es trivial.

Dentro de ese marco conviven hoy dos aproximaciones de naturaleza muy distinta. Por un lado, los modelos visión-lenguaje-acción (VLA), que parten de un modelo fundacional preentrenado sobre grandes volúmenes de datos heterogéneos y se ajustan a la tarea concreta; NVIDIA GR00T [1] es un exponente reciente orientado específicamente a humanoides. Por otro, las arquitecturas de imitación específicas, entrenadas desde cero sobre los datos de la tarea, de las que ACT [2] es probablemente la más extendida. La diferencia de escala entre ambas es de casi dos órdenes de magnitud en número de parámetros.

La literatura ofrece abundantes resultados de cada familia por separado, pero comparaciones rigurosas entre ambas sobre una misma tarea, un mismo conjunto de datos y un mismo criterio de éxito son escasas. Y sin esa comparación, la decisión de qué método emplear en un despliegue concreto se toma sobre intuiciones acerca del tamaño del modelo, no sobre evidencia.

Objetivos

El objetivo general del trabajo es doble. Por un lado, **diseñar, implementar y validar un *pipeline* íntegro *end-to-end* de aprendizaje por imitación** —desde la simulación de la tarea y la captura de demostraciones por teleoperación hasta el entrenamiento y la evaluación en bucle cerrado— sobre un robot humanoide y una tarea de manipulación industrial representativa. Por otro, **emplear ese *pipeline* para cuantificar**, en igualdad de condiciones, la diferencia de rendimiento entre un modelo visión-lenguaje-acción y una política de imitación clásica. El segundo objetivo depende del primero: sin un *pipeline* validado de punta a punta no hay datos fiables sobre los que comparar nada.

De ambos se derivan los siguientes objetivos específicos:

1. Diseñar, implementar y verificar el *pipeline* completo de aprendizaje por imitación —simulación, teleoperación, conversión de datos, entrenamiento y evaluación en bucle cerrado—, comprobando cada etapa antes de invertir esfuerzo en la siguiente, de modo que sea capaz de servir indistintamente a arquitecturas de naturaleza distinta sobre el mismo conjunto de datos.
2. Construir un entorno de simulación reproducible de la tarea, con el humanoide Unitree G1 y una válvula procedente de un modelo CAD real, que sirva a la vez para grabar demostraciones y para evaluar políticas con un criterio de éxito objetivo.
3. Recoger un conjunto de demostraciones teleoperadas con variabilidad suficiente para que la tarea no sea memorizable, incluyendo dos disposiciones distintas de la válvula y aleatorización de su posición.
4. Establecer un único *pipeline* de conversión que alimente a ambos métodos desde el mismo fichero de demostraciones, de modo que ninguna diferencia observada pueda atribuirse al tratamiento de los datos.
5. Entrenar y evaluar las dos políticas, y contrastar sus tasas de éxito con la prueba estadística adecuada a un diseño de medidas emparejadas.
6. Caracterizar el coste de cada método en los dos ejes que condicionan un despliegue real: la cantidad de demostraciones necesaria y la latencia de inferencia frente al presupuesto de tiempo del lazo de control.

Metodología

El trabajo sigue una metodología experimental comparativa, articulada en cuatro etapas encadenadas: teleoperación y grabación, conversión del conjunto de datos, entrenamiento y evaluación. El diseño descansa sobre tres decisiones que condicionan la validez de las conclusiones y que conviene adelantar aquí.

Un único conjunto de datos para ambos métodos. El formato de conjunto de datos de GR00T es un superconjunto del formato LeRobot que consume ACT, de modo que una sola conversión sirve a los dos entrenamientos. Cualquier diferencia de resultado procede, por tanto, del método y no de una preparación distinta de los datos.

Un único banco de evaluación. Ambas políticas se sirven como procesos independientes que implementan el mismo protocolo de comunicación con el simulador. El comando que lanza la evaluación es idéntico hasta el último carácter para las dos; lo único que cambia es qué proceso atiende la petición. Esto elimina toda una clase de diferencias espurias derivadas del montaje experimental.

Comparación emparejada. Las dos políticas se evalúan sobre las mismas condiciones iniciales, generadas con la misma semilla, de manera que cada episodio constituye un par. El contraste estadístico apropiado es entonces la prueba exacta de McNemar sobre los casos discordantes, no una comparación de proporciones independientes, que desperdiciaría la información del emparejamiento y sobreestimaría la incertidumbre.

Sobre esa base se realizan tres experimentos: la comparación principal sobre doscientas tiradas por política, una curva de eficiencia en datos con subconjuntos estratificados de veinticinco, cincuenta y cien demostraciones, y una medida de latencia de inferencia en lazo cerrado.

Todo el desarrollo se ha realizado en simulación. Aunque se disponía de acceso a la plataforma física, no se contó con un espacio adecuadamente acondicionado para la experimentación segura ni con el conjunto de herramientas e infraestructura de software necesarias —en particular, actualizaciones pendientes de *firmware*, controladores de bajo nivel y el entorno de despliegue requerido para la inferencia en tiempo real de estas políticas concretas—. Esto constituye una limitación del trabajo que se discute con detalle en el capítulo de evaluación.

Estructura del trabajo

El resto de la memoria se organiza como sigue.

El capítulo 1, *Estudio del problema*, sitúa el trabajo en su contexto industrial y tecnológico, revisa el estado de la cuestión en aprendizaje por imitación, modelos visión-lenguaje-acción, simulación robótica y teleoperación, delimita el problema concreto que se aborda y recoge la normativa aplicable a un sistema de estas características.

El capítulo 2, *Gestión de proyecto software*, presenta la simulación de la gestión del proyecto en condiciones de entorno empresarial: alcance, estimación, presupuesto, planificación, recursos y riesgos.

El capítulo 3, *Solución*, describe el sistema construido: la arquitectura, el diseño de la tarea de simulación, el *pipeline* de datos, los dos entrenamientos y las pruebas realizadas sobre cada etapa. Incluye el papel del módulo de reconstrucción fotorrealista de la escena, una contribución instrumental cuyo detalle técnico se desarrolla en el anexo D.

El capítulo 4, *Evaluación*, expone el protocolo experimental y analiza los resultados obtenidos, con especial atención a qué afirmaciones sostienen los datos y cuáles no.

El capítulo *Conclusión* recoge las aportaciones del trabajo, los problemas encontrados y las líneas de continuación.

Cierran el documento cuatro anexos: el control de versiones del proyecto, el seguimiento real del Trabajo de Fin de Máster, el protocolo de evaluación empleado y el desarrollo técnico completo del módulo de reconstrucción fotorrealista mediante Gaussian Splatting.

Capítulo 1

Estudio del problema

Este capítulo sitúa el trabajo. Comienza por el contexto industrial que motiva la tarea y por la plataforma robótica elegida, y justifica por qué todo el desarrollo se hace en simulación. Revisa después el estado de la cuestión en las áreas que el trabajo cruza —aprendizaje por imitación, políticas entrenadas desde cero, modelos visión-lenguaje-acción, control de humanoides, simulación, teleoperación y reconstrucción de escenas—, delimita a continuación el problema concreto que se aborda, con sus limitaciones, y termina recogiendo la normativa que afectaría a un despliegue real del sistema.

1.1. El contexto del problema

1.1.1. La operación de válvulas en instalaciones de proceso

Una instalación de proceso —una refinería, una planta de tratamiento de gas, una plataforma de extracción— es, vista de cerca, una red de tuberías interrumpida cada pocos metros por una válvula. Las de compuerta y las de globo accionadas por volante son las más numerosas: son baratas, robustas y no necesitan más energía que la del brazo de quien las gira. Una válvula de compuerta abre y cierra desplazando un disco perpendicular al flujo mediante un husillo roscado, de modo que pasar de cerrada a completamente abierta exige varias vueltas completas del volante; el modelo CAD empleado en este trabajo, tomado de una válvula real, autoriza seis vueltas y media de recorrido. El par necesario crece con el diámetro nominal y con la presión de la línea, y en una válvula que lleva meses sin moverse hay que sumar el de arranque.

Pese a la extensión de la instrumentación, una fracción muy grande de esas válvulas sigue siendo manual y va a seguir siéndolo: motorizarlas todas es prohibitivo, y muchas solo se accionan en paradas, arranques o emergencias. Su operación recae en personal de campo que recorre la planta siguiendo hojas de ruta, y tiene tres características que la hacen candidata natural a la automatización. Es **repetitiva**: el gesto es el mismo válvula tras válvula. Es **físicamente exigente**: volantes a ras de suelo o por encima de la cabeza, pares de arranque altos, posturas forzadas. Y se realiza en **entornos hostiles**: temperaturas extremas, ruido y, sobre todo, zonas clasificadas como atmósferas potencialmente explosivas, en las que la mera presencia de una persona es un riesgo que hay que gestionar y en las que, durante un incidente, la operación manual es lo último que se quiere tener que hacer.

No es casual que girar una válvula fuera una de las pruebas del *DARPA Robotics Challenge* [3], el concurso que en 2015 puso a los humanoides a realizar tareas de intervención en un escenario de accidente industrial, ni que las iniciativas de robótica para el sector —el reto ARGOS promovido por Total, o los despliegues de cuadrúpedos de inspección en plataformas marinas— la cuenten entre sus capacidades objetivo. Es una tarea con un criterio de éxito objetivo —la válvula abre o no abre—, bien delimitada y con valor directo. Y es, al mismo tiempo, una tarea que resiste mal el enfoque clásico de programar la trayectoria: la válvula puede estar montada en cualquier orientación, el volante exige un agarre firme y un giro sostenido, y el par de reacción se transmite por los brazos hasta la base del robot.

1.1.2. Robots humanoides como plataforma de intervención

La pregunta obvia es por qué un humanoide y no un manipulador sobre base móvil, que es una tecnología madura. La respuesta está en la instalación, no en el robot. Una planta de proceso está diseñada para el cuerpo humano: los pasillos tienen la anchura de una persona, los accesos son escaleras y pasos elevados de peldaños, los volantes están a la altura y a la distancia a la que un operario los alcanza de pie, y su diámetro es el que una mano agarra. Un robot con la morfología del operario hereda ese diseño sin reformar nada; cualquier otra morfología obliga a adaptar la planta o a renunciar a parte de ella.

Lo que ha cambiado en los últimos tres años no es ese argumento, que es antiguo, sino el precio de la plataforma. Los humanoides de investigación de la década pasada eran prototipos únicos con presupuestos de siete cifras; hoy varios fabricantes venden humanoides completos por el precio de un vehículo industrial. El Unitree G1 [4], empleado en este trabajo, es representativo de esa generación: un humanoide de

tamaño reducido —del orden de 1,3 m y 35 kg—, con veintinueve grados de libertad en el cuerpo y, en la variante utilizada, manos de tres dedos con siete articulaciones actuadas cada una, hasta un total de cuarenta y tres. Es además la plataforma para la que NVIDIA distribuye su pila de aprendizaje para humanoides, lo que a efectos de este trabajo significa que existen un modelo de simulación mantenido, una política de equilibrio de cuerpo completo ya entrenada y una descripción del *embodiment* para el modelo fundacional. Elegir otro robot habría supuesto reconstruir esas tres cosas.

Su tamaño es también una restricción. Un G1 de pie alcanza con comodidad un volante frontal a unos 43 cm del pelvis y uno cenital a unos 30 cm, distancias que se han medido con el operador dentro del sistema de teleoperación y que fijan dónde puede estar la válvula. Un humanoide de talla adulta tendría más alcance y más par; también más masa que controlar cuando el volante devuelve la reacción.

1.1.3. Por qué el trabajo se realiza en simulación

El trabajo se realiza íntegramente en simulación por una razón metodológica y por dos prácticas.

La metodológica es la que dimensiona el experimento. Comparar dos políticas con rigor exige evaluarlas muchas veces —en este trabajo, doscientas tiradas por política en la comparación principal y seiscientas más en la curva de eficiencia— y, sobre todo, exige evaluarlas sobre las **mismas** condiciones iniciales, de modo que cada episodio sea un par y el contraste estadístico pueda aprovecharlo. Sobre un robot físico, ni lo uno ni lo otro es viable en el plazo de un trabajo de fin de máster: reproducir la pose de una válvula al milímetro cientos de veces no es posible, y cada tirada cuesta minutos de preparación y expone al robot a una caída. En simulación, una semilla fija garantiza que la tirada i de una política ve exactamente la escena que vio la tirada i de la otra, y mil episodios son una noche de máquina.

A pesar de haber dispuesto de un G1 físico durante el desarrollo, recoger demostraciones por teleoperación sobre el robot real habría requerido una infraestructura de seguridad —arnés, zona acotada, un segundo operador— que no estaba al alcance del proyecto.

Conviene ser explícito sobre lo que se pierde. Primero, la dinámica de contacto: el agarre del volante y el par que devuelve son, en el simulador, lo que el motor de física y los parámetros de la articulación dicen que son, no lo que hace un volante de fundición. Segundo, el ruido de los sensores y las latencias de la plataforma real.

Tercero, y más importante, que la política de equilibrio que sostiene al robot está aquí liberada de parte de su trabajo: la base del robot se fija durante la manipulación, una decisión que se justifica en el capítulo 3 y que en un despliegue real no existiría.

Se ha intentado mitigar esa distancia por tres vías. La válvula procede de un modelo CAD de una válvula real, no de una primitiva geométrica. Las imágenes que ven las políticas no muestran un fondo sintético, sino la reconstrucción fotorrealista de una oficina real obtenida a partir de vídeo (sección 1.2.7 y anexo D). Y la evaluación no se limita a la tasa de éxito: mide también la latencia de inferencia en lazo cerrado, que es la magnitud que decidiría si la política cabe en el robot.

1.2. El estado de la cuestión

1.2.1. Aprendizaje por imitación

El aprendizaje por imitación es la familia de técnicas que obtienen una política de control a partir de demostraciones, en lugar de a partir de una función de recompensa. Su forma más simple, la clonación de comportamiento (*behavior cloning*), reduce el problema a uno de aprendizaje supervisado: se recogen pares observación–acción de un experto y se ajusta un modelo que prediga la segunda a partir de la primera. Es una idea antigua. ALVINN [5] condujo un vehículo en 1988 con una red de tres capas que asociaba la imagen de una cámara a un ángulo de dirección, entrenada con las maniobras de un conductor humano.

Su limitación fundamental también se conoce desde entonces, y tiene nombre: la deriva de distribución. La política se entrena sobre los estados que el experto visitó, pero al ejecutarse comete pequeños errores que la llevan a estados ligeramente distintos, sobre los que nunca vio una demostración y en los que sus errores son mayores; el proceso se realimenta. Ross *et al.* [6] formalizaron el problema —en el peor caso, el coste acumulado de la clonación ingenua crece con el cuadrado de la longitud del episodio— y propusieron DAgger como respuesta: alternar la ejecución de la política con el etiquetado por el experto de los estados que esta visita, de modo que el conjunto de entrenamiento acabe cubriendo la distribución que la propia política induce. DAgger exige un experto disponible en línea, lo que en la teleoperación de un humanoide es costoso, y por eso las aproximaciones actuales atacan la deriva por otro lado: prediciendo bloques de acciones en lugar de acciones aisladas, lo que reduce el número de decisiones en las que la política puede equivocarse y da a cada una un contexto temporal más largo; y, en los modelos fundacionales, apoyándose

en un preentrenamiento que ha visto muchos más estados que las demostraciones de la tarea.

Se fija aquí el vocabulario que emplea el resto de la memoria. Una **demonstración** es un episodio completo ejecutado por el operador y grabado. Un **episodio** de evaluación es un intento de la política, que termina en éxito o por agotamiento del tiempo; cada uno de esos intentos es una **tirada** (*rollout*). Una **política** es la función que asocia observaciones a acciones. Todos ellos, y los demás términos técnicos, están recogidos en el glosario.

1.2.2. Políticas de manipulación entrenadas desde cero

Dentro de las arquitecturas específicas —entrenadas exclusivamente sobre las demostraciones de la tarea, sin preentrenamiento— dos han dominado los últimos tres años.

ACT (*Action Chunking with Transformers*) [2] nació junto con ALOHA, un sistema bimanual de bajo coste, con un objetivo declarado: aprender manipulación fina con pocas demostraciones, del orden de cincuenta, y hardware imperfecto. Su aportación es doble. La primera es el *chunking*: la política no predice la siguiente acción sino un bloque de acciones futuras —en el trabajo original, un centenar de pasos a 50 Hz—, lo que reduce el horizonte efectivo de la tarea y con él la acumulación de errores. La segunda es modelar las demostraciones con un autocodificador variacional condicional (CVAE): durante el entrenamiento, un codificador comprime la secuencia de acciones en una variable latente, y un decodificador basado en *transformer* genera el bloque a partir de las imágenes, del estado articular y de esa variable; en inferencia, el latente se fija a cero y la política es determinista. El resultado es una arquitectura compacta, de decenas de millones de parámetros, rápida y muy extendida, lo que la convierte en el representante natural del brazo clásico de esta comparación.

Diffusion Policy [7] es la alternativa generativa. En lugar de regresar el bloque de acciones, lo obtiene por un proceso de difusión que parte de ruido y lo refina en varios pasos condicionados por la observación. Su ventaja está en las demostraciones multimodales: cuando hay varias formas válidas de hacer lo mismo, una regresión aprende la media, que no es ninguna de ellas, y un modelo generativo puede elegir una. A cambio, cada inferencia requiere varias evaluaciones de la red.

Se ha elegido ACT y no Diffusion Policy para el brazo clásico por tres razones. Es la arquitectura más extendida y mejor documentada, con una implementación de

referencia mantenida en LeRobot. Su inferencia es de una sola pasada, lo que hace más limpia la medida de latencia. Y la tarea, tal como se ha grabado —un único operador, un gesto de agarre fijado de antemano—, no es especialmente multimodal, de modo que la ventaja principal de la difusión no tendría ocasión de manifestarse. Nada impide añadir Diffusion Policy como tercer brazo: el *pipeline* de datos y el banco de evaluación le servirían sin cambios.

1.2.3. Modelos visión-lenguaje-acción

La otra familia parte de una observación: los grandes modelos de visión y lenguaje ya saben mucho del mundo —qué es una válvula, dónde tiene el volante, qué significa «abrir»— y ese conocimiento puede transferirse al control si se enseña al modelo a emitir acciones.

La trayectoria es reciente y rápida. RT-1 [8] demostró en 2022 que un *transformer* entrenado sobre unos 130 000 episodios reales, recogidos durante diecisiete meses con una flota de trece robots, generalizaba a cientos de tareas descritas en lenguaje natural. RT-2 [9] dio el paso decisivo: en lugar de entrenar un modelo de control desde cero, tomó modelos visión-lenguaje preentrenados sobre la web y los ajustó para que produjeran las acciones como si fueran texto —una secuencia de *tokens* que codifican el desplazamiento del efector—. Acuñó el término *vision-language-action* y mostró generalización a objetos y conceptos que no aparecían en los datos robóticos. OpenVLA [10] llevó esa receta a un modelo abierto de siete mil millones de parámetros, entrenado sobre casi un millón de episodios de Open X-Embodiment [11], y mostró que el ajuste fino con adaptadores de bajo rango basta para adaptarlo a un robot nuevo. π_0 [12] cambió la forma de generar la acción: en lugar de *tokens* discretos, un «experto de acción» separado produce bloques continuos de acciones mediante *flow matching*, lo que permite emitirlas a 50 Hz con la destreza que exige, por ejemplo, doblar ropa.

Lo que esta familia promete es generalización —a instancias del objeto no vistas, a variaciones de la escena, a instrucciones nuevas— con menos demostraciones de la tarea concreta que una arquitectura entrenada desde cero. Lo que cuesta es cómputo. Los VLA tienen entre uno y dos órdenes de magnitud más parámetros que ACT, su ajuste fino requiere aceleradoras de gama profesional y su inferencia tarda decenas de milisegundos donde ACT tarda unidades. Si esa diferencia importa depende del presupuesto de tiempo del lazo de control, y por eso se mide en el capítulo 4.

1.2.4. GR00T y el control de humanoides

GR00T N1 [1] es el modelo fundacional de NVIDIA para humanoides, y su variante N1.7 es la empleada en este trabajo. Su arquitectura es de dos sistemas, por analogía con la cognición humana. El «sistema 2» es un modelo visión-lenguaje que procesa las imágenes y la instrucción y produce una representación de la escena a baja frecuencia. El «sistema 1» es un *transformer* de difusión que, condicionado por esa representación y por el estado del robot, genera bloques de acciones continuas mediante *flow matching*, en la línea de π_0 . Ambos se entrenan de extremo a extremo.

Dos rasgos lo hacen especialmente adecuado a este trabajo. El primero es que está diseñado para servir a varios *embodiments*: el estado y la acción de cada robot pasan por codificadores y decodificadores propios —pequeñas redes que se entrenan durante el ajuste fino— mientras el núcleo del modelo es común. Incorporar el G1 con la configuración de este trabajo consiste en describir su espacio de estado y de acción, no en modificar el modelo. El segundo es la «pirámide de datos» con la que se preentrenó: en la base, vídeo humano y de la web; en el medio, datos sintéticos generados en simulación; en la cúspide, teleoperación real de humanoides. Es un preentrenamiento que ha visto humanoides manipulando, lo que no puede decirse de los VLA anteriores, entrenados sobre brazos de sobremesa.

El modelo produce acciones para el tren superior; mantener al humanoide de pie mientras las ejecuta es un problema distinto, y en este trabajo lo resuelve una capa separada. Es la arquitectura habitual en manipulación con humanoides: una política de control de cuerpo completo (*whole-body control*, WBC), entrenada por refuerzo en simulación, gobierna las piernas y mantiene el equilibrio, y recibe de arriba órdenes de alto nivel —velocidad de avance, altura del pelvis, orientación del torso—, mientras las poses de las muñecas que marca la política de manipulación se convierten en ángulos articulares mediante cinemática inversa. La política de imitación aprende, por tanto, sobre un robot que ya sabe estar de pie. Esa división es lo que hace tratable el problema: el espacio de acción que aprende la política tiene veintitrés dimensiones, no cuarenta y tres.

1.2.5. Entornos de simulación y formatos de conjuntos de datos

La pila de simulación empleada es la de NVIDIA. Isaac Sim [13] es el simulador: motor de física PhysX, render por trazado de rayos y escenas descritas en USD. Isaac Lab [14], nacido como Orbit, es el marco de aprendizaje construido sobre él: aporta

la ejecución paralela en GPU de miles de entornos, la definición modular de una tarea mediante gestores de observaciones, acciones, recompensas y terminaciones, y los adaptadores para las bibliotecas de aprendizaje. Isaac Lab Arena [15] es la capa más reciente: compone escenas de manipulación a partir de un registro de activos —robots, objetos, fondos— y añade la infraestructura de teleoperación, grabación de demostraciones, conversión de datos y evaluación de políticas, incluidas las que se ejecutan en otro proceso.

Se consideraron y descartaron dos alternativas. MuJoCo, por su precisión de contacto y su bajo coste de cómputo, habría sido una opción razonable para la física, pero no dispone del sistema de teleoperación en realidad mixta ni del *embodiment* del G1 con su política de equilibrio ya integrada, y reconstruir ambas cosas no cabía en el plazo. Dentro de la propia pila de NVIDIA se consideró generar las demostraciones automáticamente con Isaac Lab Mimic, que multiplica un puñado de demostraciones humanas recomblando sus segmentos, y se rechazó como riesgo de calendario: exige anotar la estructura de la tarea y no estaba probado sobre este *embodiment*.

En cuanto a los datos, el denominador común es el formato LeRobot [16], de Hugging Face: cada episodio como filas de un fichero *parquet* con el estado y la acción, las imágenes como vídeo comprimido, y metadatos que describen las modalidades. Es el formato que consumen las implementaciones de referencia de ACT y de Diffusion Policy, y el que GR00T adopta con una extensión —un fichero de modalidades que asigna rangos del vector de estado a partes del cuerpo— que ACT simplemente ignora. Esa compatibilidad es la que permite que una sola conversión alimente a los dos métodos. Detrás del formato hay un movimiento más amplio de agregación de datos robóticos, del que Open X-Embodiment [11] —más de un millón de trayectorias de veintidós *embodiments*, aportadas por veintiuna instituciones— es el exponente principal, y que es lo que ha hecho posibles los VLA abiertos.

1.2.6. Teleoperación en realidad mixta

Las demostraciones hay que grabarlas, y la forma de hacerlo condiciona su calidad. Para brazos de sobremesa la solución dominante son los dispositivos maestro-esclavo —ALOHA es el ejemplo—, en los que el operador mueve una réplica pasiva del robot. Para un humanoide no hay réplica que mover: la captura debe hacerse sobre el cuerpo del operador.

La vía que se ha impuesto es la realidad mixta con dispositivos de consumo. Unas gafas con seguimiento de cabeza y de manos —o mandos— proporcionan la pose

de la cabeza y de las muñecas del operador varias decenas de veces por segundo; un módulo de *retargeting* traslada esas poses al cuerpo del robot, que tiene otras proporciones y otras articulaciones; y el vídeo estereoscópico de las cámaras del robot se devuelve a las gafas, de modo que el operador ve lo que el robot ve y cierra el lazo con su propia percepción. Open-TeleVision [17] es una referencia reciente de este esquema.

En este trabajo las gafas son unas PICO 4 Ultra [18], un visor autónomo: lleva su propio procesador y no está unido a ningún ordenador por cable. Eso plantea un problema, porque el simulador necesita una aceleradora de gama profesional y no puede ejecutarse en el visor. Lo resuelve NVIDIA CloudXR [19], la capa de transmisión que separa dónde se renderiza de dónde se muestra: el simulador genera las dos vistas estereoscópicas en la estación de trabajo, CloudXR las codifica como vídeo y las envía por la red al visor, y en sentido contrario recibe la pose de la cabeza y de los mandos varias decenas de veces por segundo y se la entrega al simulador a través de la interfaz OpenXR. Para el sistema, el visor es un cliente de vídeo con sensores; para el operador, está dentro de la escena. Los mandos son la fuente de la pose de las muñecas y el gatillo la orden de cierre de la mano; el capítulo 3 documenta el montaje y los problemas que dio.

Dos propiedades de esta vía importan para lo que viene después. La primera es que el operador forma parte del sistema: su fatiga, sus hábitos y sus errores quedan grabados, y la política los copia. Por eso las sesiones de grabación se limitan a veinticinco demostraciones, y por eso el gesto de agarre se fijó antes de grabar el conjunto definitivo. La segunda es que la latencia entre el movimiento del operador y la imagen que recibe determina cuánto puede afinar; un fondo costoso de renderizar la empeora, y ese es el motivo por el que el fondo fotorrealista no se aplica durante la grabación.

1.2.7. Realismo visual mediante reconstrucción de escenas

Una política que aprende de imágenes aprende también del fondo. Si las imágenes de entrenamiento muestran un plano gris bajo una luz uniforme, la política no tiene motivo para ignorarlo y, puesta ante una escena real, verá algo que nunca vio. Los fondos sintéticos tradicionales, modelados a mano, mitigan el problema a un coste de modelado alto y con un realismo limitado.

La reconstrucción neuronal de escenas ofrece otra vía: capturar un entorno real con una cámara y obtener una representación que pueda renderizarse desde cual-

quier punto de vista. Los campos de radiancia neuronales abrieron la línea, pero su render era demasiado lento para un simulador interactivo. El *Gaussian Splatting* tridimensional [20] resolvió esa limitación representando la escena como millones de gaussianas anisótropas explícitas —cada una con posición, covarianza, opacidad y color dependiente de la dirección de vista— que se optimizan a partir de las imágenes y se rasterizan en tiempo real con una calidad comparable a la de los mejores campos de radiancia. NVIDIA lo ha integrado en Omniverse bajo el nombre de NuRec, lo que permite cargar una de esas reconstrucciones como un activo más de una escena USD y renderizarla desde la cámara del robot.

En este trabajo el fondo es la reconstrucción de una oficina real. Su papel es acercar las imágenes de entrenamiento a las de un despliegue, y su coste —unos cuarenta milisegundos por paso de simulación— es lo que obliga a aplicarlo a posteriori, reproduciendo las demostraciones con el fondo cargado. Cómo se hace, y el problema que planteó, se desarrollan en el anexo D.

1.3. La definición del problema

El problema que aborda este trabajo cabe en una frase: **dado un conjunto de demostraciones teleoperadas de la apertura de una válvula de volante con un humanoide, determinar, en condiciones idénticas y con el contraste estadístico apropiado, si un modelo fundacional visión-lenguaje-acción ajustado a la tarea supera a una política de imitación clásica entrenada desde cero, y a qué coste en datos y en latencia.** Resolverlo exige, antes, construir el sistema que lo hace medible: la tarea, el *pipeline* de datos y el banco de evaluación.

El argumento que lo motiva es el vacío que deja la literatura. Cada familia acumula resultados por separado: los VLA publican tasas de éxito sobre sus propios conjuntos de tareas, y las arquitecturas específicas sobre los suyos. Cuando se comparan entre sí, suele ser dentro del artículo que presenta uno de los métodos, sobre tareas elegidas por sus autores, con conjuntos de datos que no siempre coinciden y con una comparación de proporciones independientes como único contraste, cuando hay alguno. Faltan comparaciones en las que la tarea, los datos, el criterio de éxito y las condiciones iniciales sean los mismos para ambos métodos, y en las que el contraste aproveche ese emparejamiento. Sin ellas, la elección del método para un despliegue concreto se toma sobre el tamaño del modelo y la reputación de la familia, no sobre evidencia.

Para que la comparación sea limpia, el trabajo asume las restricciones siguientes, que son a la vez sus limitaciones:

- **Una sola tarea.** La apertura de la válvula, en dos disposiciones y con la posición aleatorizada. Los resultados describen esa tarea; lo que se transfiere a otras es el banco de comparación, no las cifras.
- **Sin validación sobre el robot físico.** Todo ocurre en simulación, con las pérdidas descritas en la sección 1.1.3. La distancia entre el simulador y el robot no se mide.
- **Una única semilla de entrenamiento por configuración.** Cada punto de la curva de eficiencia es un solo entrenamiento, de modo que la variación entre puntos mezcla el efecto de los datos con la varianza del propio entrenamiento.
- **Un único operador.** Las cien demostraciones las grabó la misma persona. Eso da un gesto consistente, que facilita el aprendizaje, y a la vez sesga el conjunto hacia un estilo.
- **Una base fija.** La pelvis del robot se inmoviliza durante la manipulación; la política de equilibrio sigue activa, pero no tiene que absorber el par de reacción del volante.
- **Un plazo acotado.** El trabajo debía cerrarse antes de finales de agosto de 2026, lo que fijó el tamaño del conjunto de datos en cien demostraciones y la comparación en dos brazos.

Ninguna de ellas se oculta en el análisis: el capítulo 4 dedica una sección a las amenazas a la validez que derivan de ellas.

1.4. Normativa asociada

Todo el trabajo se desarrolla en simulación, de modo que ninguna norma le es directamente exigible. Pero el sistema que estudia —un robot autónomo que manipula válvulas en una planta de proceso— está regulado en todos sus aspectos, y un dictamen sobre su viabilidad que ignorase ese marco estaría incompleto. Se recoge aquí la normativa que afectaría a un despliegue, organizada en cuatro bloques, señalando en cada uno qué implicaría para un sistema basado en aprendizaje. La tabla 1.1 la resume.

Seguridad de máquinas y robótica. Un robot es una máquina, y el marco legal europeo para las máquinas es la Directiva 2006/42/CE, que el Reglamento (UE)

2023/1230 sustituye a partir de enero de 2027. El nuevo reglamento es relevante para este trabajo por una razón concreta: considera de forma explícita los componentes de seguridad con comportamiento total o parcialmente autoevolutivo basado en aprendizaje automático, y les exige evaluación de conformidad por un organismo notificado. Por debajo del marco legal, ISO 12100 fija la metodología de evaluación del riesgo; ISO 10218-1 y -2 recogen los requisitos del robot industrial y de su integración; ISO/TS 15066 los límites de contacto para robots que comparten espacio con personas, que un humanoide en una planta comparte por definición; e ISO 13849-1 el diseño de las partes del sistema de mando relacionadas con la seguridad.

Atmósferas potencialmente explosivas. Es el bloque determinante en el contexto del petróleo y el gas. La Directiva 2014/34/UE, transpuesta por el Real Decreto 144/2016, regula los equipos destinados a zonas clasificadas, y la Directiva 1999/92/CE, transpuesta por el Real Decreto 681/2003, la clasificación de esas zonas y la protección de los trabajadores; la serie IEC 60079 contiene los requisitos técnicos del material eléctrico. Un humanoide —baterías de litio, motores, electrónica de potencia— es, desde este punto de vista, un equipo eléctrico que habría que certificar para la zona en la que opere, y esa certificación es, hoy, el obstáculo más serio para llevar un sistema como este a una zona clasificada. Un despliegue realista empezaría en áreas no clasificadas o en zona 2.

Seguridad funcional. IEC 61508 establece los requisitos de los sistemas eléctricos, electrónicos y programables relacionados con la seguridad, e IEC 61511 los particulariza para los sistemas instrumentados de seguridad de la industria de proceso, que son los que protegen la planta en la que el robot actuaría. El informe técnico ISO/IEC TR 5469 aborda la relación entre seguridad funcional y sistemas de inteligencia artificial, y su conclusión práctica es la que cualquier arquitectura de despliegue debería adoptar: la política aprendida no puede estar en el camino de seguridad. Los límites de par, la parada de emergencia y la vigilancia de la posición del robot deben garantizarlos capas deterministas, y la política solo debe decidir cómo se ejecuta la tarea dentro de ellos.

Inteligencia artificial. El Reglamento (UE) 2024/1689 clasifica como de alto riesgo los sistemas de inteligencia artificial que son componentes de seguridad de productos cubiertos por la legislación de armonización, entre ellos las máquinas. Un sistema como el estudiado quedaría en esa categoría, con las obligaciones que conlleva: gestión de riesgos, gobernanza de los datos de entrenamiento, documentación técnica, registro de la actividad, supervisión humana y requisitos de exactitud y robustez. La documentación del conjunto de datos y del protocolo de evaluación que

esta memoria recoge, y que en un trabajo académico es una cuestión de rigor, sería en un despliegue una obligación legal.

Tabla 1.1: Normativa aplicable a un despliegue del sistema

Bloque	Referencia	Ámbito
Máquinas y robótica	Reglamento (UE) 2023/1230 ISO 12100 ISO 10218-1/-2 ISO/TS 15066 ISO 13849-1	Máquinas; sustituye a la Directiva 2006/42/CE e incluye las funciones de seguridad basadas en aprendizaje automático Principios generales de diseño y evaluación del riesgo Robots industriales: requisitos del robot y de su integración Robots colaborativos: límites de contacto entre robot y persona Partes de los sistemas de mando relativas a la seguridad
Atmósferas explosivas	Directiva 2014/34/UE Directiva 1999/92/CE IEC 60079 (serie)	Equipos para atmósferas potencialmente explosivas (RD 144/2016) Clasificación de zonas y protección de los trabajadores (RD 681/2003) Requisitos técnicos del material eléctrico para atmósferas explosivas
Seguridad funcional	IEC 61508 IEC 61511 ISO/IEC TR 5469	Sistemas eléctricos, electrónicos y programables relacionados con la seguridad Sistemas instrumentados de seguridad en la industria de proceso Seguridad funcional y sistemas de inteligencia artificial
Inteligencia artificial	Reglamento (UE) 2024/1689	Reglamento europeo de inteligencia artificial: sistemas de alto riesgo

Por último, y en otro plano, las referencias bibliográficas de esta memoria siguen la norma ISO 690, tal como establece la plantilla de la titulación.

Capítulo 2

Gestión de proyecto software

Este capítulo simula la gestión del proyecto en las condiciones habituales de un entorno empresarial, tal como exige la titulación. No describe, por tanto, la forma en que se ha gestionado el Trabajo de Fin de Máster —eso se recoge en el anexo B—, sino cómo se habría planificado, presupuestado y controlado el mismo desarrollo si lo hubiera acometido una empresa de ingeniería para un cliente del sector del petróleo y el gas.

El supuesto de partida es el siguiente: una empresa de servicios de automatización recibe el encargo de evaluar la viabilidad de automatizar el accionamiento de válvulas de volante mediante un robot humanoide, y de entregar un dictamen fundamentado sobre qué familia de algoritmos de aprendizaje conviene adoptar antes de invertir en la fase de despliegue. Para que la simulación tenga fundamento, las duraciones, los recursos y los riesgos se han tomado del desarrollo real; las tarifas y los precios son estimaciones razonables a fecha de 2026, no ofertas.

2.1. Alcance del proyecto

2.1.1. Definición del proyecto

El objetivo del encargo es responder a una pregunta de decisión —¿modelo funcional o política específica?— con evidencia experimental obtenida en simulación, antes de comprometer inversión en hardware y en integración. El cliente no compra un robot que abre válvulas; compra el criterio para elegir cómo construirlo.

Los entregables comprometidos son cuatro:

1. Un **entorno de simulación** reproducible de la tarea, con el robot y una válvula procedente de un modelo CAD del cliente, instrumentado con un criterio de éxito automático.
2. Un **conjunto de demostraciones** teleoperadas, documentado y en un formato que sirva a ambas familias de métodos.
3. Un **banco de evaluación** que enfrente a las políticas a condiciones idénticas y produzca las métricas acordadas.
4. El **dictamen**: la comparación cuantitativa, su análisis estadístico y la recomendación, en forma de informe técnico.

Los criterios de aceptación son verificables: el entorno se acepta cuando una demostración teleoperada abre la válvula y queda grabada con imagen; el conjunto de datos, cuando supera la verificación de formato y alimenta ambos entrenamientos sin transformación adicional; el banco, cuando dos políticas distintas se evalúan con el mismo comando y las mismas semillas; y el dictamen, cuando cada afirmación va acompañada de su intervalo de confianza y su contraste.

Queda **fuera de alcance**, y así se hace constar en la oferta: el despliegue sobre robot físico, la certificación del sistema para zona ATEX, la integración con el sistema de control de la planta, y cualquier tarea distinta de la apertura de válvulas de volante. Cada una de esas cuatro cosas es un proyecto por sí misma, y el dictamen debe precisamente decir si merece la pena acometer la primera.

2.1.2. Estimación de tareas y recursos

El trabajo se descompone en cinco paquetes, más uno transversal de gestión, y cada paquete se asigna al perfil que lo lidera:

PT1 — Entorno de simulación. Construcción de la tarea sobre el marco de simulación: robot, válvula, criterio de éxito, aleatorización, teleoperación en realidad mixta y grabación. Ingeniero de robótica sénior. Es el paquete de mayor incertidumbre, porque integra una pila de realidad mixta cuyo comportamiento no se conoce hasta que se monta.

PT2 — Captura de demostraciones. Sesiones de teleoperación, verificación de cada sesión y aplicación del fondo fotorrealista. Técnico de teleoperación, con apoyo del sénior.

PT3 — Pipeline de datos y entrenamiento. Conversión al formato común, un entorno de ejecución por método y los dos entrenamientos. Ingeniero de aprendizaje automático.

PT4 — Evaluación y análisis. Banco de evaluación, tiradas, análisis estadístico y figuras. Ingeniero de pruebas, con el de aprendizaje automático.

PT5 — Documentación y dictamen. Informe técnico, documentación de lanzamiento y entrega del repositorio. Ingeniero de robótica sénior.

Gestión y seguimiento. Planificación, reuniones con el cliente, control de riesgos. Jefe de proyecto, a tiempo parcial durante todo el proyecto.

La estimación de esfuerzo se ha hecho por analogía con el desarrollo real, redondeada al alza para absorber la incertidumbre de PT1, y suma **800 horas** repartidas en dieciséis semanas. La tabla 2.1 la desglosa por perfil.

2.1.3. Presupuesto

A continuación se detalla un presupuesto estimado para el coste total del proyecto. Las tarifas por hora son coste para la empresa —salario, cargas sociales y estructura— y no tarifas de venta.

Coste de personal

Tabla 2.1: Presupuesto de personal

Tarea	Perfil	Horas	Euros/Hora	Total
Entorno de simulación	Ingeniero de Robótica Senior	200	100	20000 €
Captura de demostraciones	Técnico de Teleoperación	120	60	7200 €
<i>Pipeline</i> de datos y entrenamiento	Ingeniero de Aprendizaje Automático	240	100	24000 €
Evaluación y análisis	Ingeniero de Pruebas	100	80	8000 €
Documentación y dictamen	Ingeniero de Robótica Senior	80	100	8000 €
Supervisión del proyecto	Jefe de Proyecto	60	120	7200 €
Total				74400 €

El personal es, con diferencia, la partida dominante: más del 95 % del coste antes de impuestos. Es lo esperable en un proyecto de viabilidad, donde no se compra nada que no exista ya y todo el valor está en las horas de ingeniería.

Coste del hardware

El equipamiento es el que efectivamente se ha empleado en el desarrollo. La tabla 2.2 recoge su precio de adquisición estimado. Puesto que el equipo sigue siendo útil después del proyecto, al presupuesto se imputa únicamente su **amortización** durante los cuatro meses de duración, con un periodo de amortización lineal de tres años, que es el habitual para equipos informáticos: $4/36$ del precio de compra.

Tabla 2.2: Equipamiento empleado y coste imputado

Concepto	Unidades	Precio (Euros)	Imputado (Euros)
Estación de trabajo PCSpecialist: AMD Ryzen Threadripper 9980X (64 núcleos, 128 hilos), placa Gigabyte TRX50 AERO D, 256 GB de memoria DDR5, NVMe Corsair MP600 PRO de 4 TB, chasis, fuente y refrigeración	1	8600	956
GPU NVIDIA RTX PRO 6000 Blackwell Max-Q Workstation Edition, 96 GB	2	9000	2000
Visor de realidad mixta PICO 4 Ultra, con mandos	1	600	67
Total		27200	3023

Cada pieza tiene una función concreta en el proyecto, y ninguna está sobredimensionada. Las dos aceleradoras no son un lujo: la teleoperación en realidad mixta necesita una GPU dedicada —la transmisión al visor no tolera compartirla— y el entrenamiento, la evaluación y la aplicación del fondo fotorrealista corren en la otra sin interrumpirla; la variante Max-Q, de 300 W, es lo que permite alojar dos en una sola estación. Los sesenta y cuatro núcleos del procesador tampoco son un exceso: la grabación de demostraciones se hace con la física en CPU, porque la combinación de física en GPU, realidad mixta y cámaras hace fallar al simulador, y con la física en CPU el número de núcleos es lo que sostiene la tasa de simulación. Los 256 GB de memoria absorben la reconstrucción fotorrealista de la escena y el simulador a la vez, y los 4 TB de almacenamiento alojan el conjunto de datos —que con el fondo aplicado pesa del orden de 110 MB por demostración—, los puntos de control de los modelos y los resultados de cientos de tiradas.

Coste del software y licencias

Toda la pila de simulación y de aprendizaje es de uso gratuito o de código abierto: Isaac Sim, Isaac Lab e Isaac Lab Arena se distribuyen sin coste para desarrollo, CloudXR se obtiene con el registro de desarrollador de NVIDIA, y LeRobot, los pesos de GR00T, Docker y el sistema operativo son libres. La única partida de software es la suscripción a herramientas de desarrollo asistido por inteligencia artificial, cuyo uso se declara al inicio de esta memoria, estimada en 100 € mensuales durante los cuatro meses: **400 €**.

Coste total

El presupuesto incluye una **contingencia del 10 %** sobre el coste directo, que es la forma habitual de cubrir la materialización de los riesgos de la sección 2.4 sin renegociar la oferta; las horas de PT1 ya están redondeadas al alza, pero un proyecto que integra una pila de realidad mixta por primera vez no debe ofertarse sin margen.

Tabla 2.3: Presupuesto total

Concepto	Coste (Euros)
Costes de personal	74400
Costes de hardware (amortización)	3023
Costes de software y licencias	400
Coste directo	77823
Contingencia (10 %)	7782
Subtotal	85605
IVA (21 %)	17977
Total Proyecto	103582 €

2.2. Plan de trabajo

2.2.1. Identificación de tareas

Los paquetes de trabajo se desglosan en las tareas de la tabla 2.4, que son las filas del diagrama de Gantt de la figura 2.1. Las dependencias son estrictas: cada tarea consume la salida de la anterior, de modo que no se puede empezar a capturar sin entorno, ni a entrenar sin datos convertidos, ni a evaluar sin políticas entrenadas.

Tabla 2.4: Tareas del proyecto y dependencias

Tarea	Contenido	Semanas	Depende de
T0 Análisis y arranque	Requisitos, activos del cliente, plan	0–2	—
T1 Entorno de simulación	Tarea, robot, válvula, teleoperación, grabación	1–6	T0
T2 <i>Pipeline</i> de datos	Conversión, verificación, entornos de ejecución	4–8	T1 (parcial)
T3 Captura de demostraciones	Sesiones, verificación, fondo fotorealista	7–11	T1, T2
T4 Entrenamiento	Ajuste fino y entrenamiento desde cero	10–13	T3 (parcial)
T5 Evaluación y análisis	Tiradas, estadística, figuras	12–15	T4
T6 Documentación y dictamen	Informe, documentación, entrega	13–16	T5 (parcial)
T7 Gestión y seguimiento	Transversal	0–16	—

2.2.2. Estimación de tareas

Las duraciones proceden del desarrollo real, con dos ajustes. La primera es que T1 se estima en seis semanas cuando el núcleo del entorno se levantó en menos: la diferencia es la integración de la pila de realidad mixta, cuyos fallos no se pueden prever y que en el desarrollo real consumió sesiones enteras en problemas de configuración sin síntoma visible. Es la tarea de mayor incertidumbre y la que justifica la contingencia. La segunda es que T3 se estima para un conjunto de datos de hasta cuatrocientas demostraciones en sesiones de veinticinco, aunque el análisis pueda concluir antes que el tamaño no es el factor limitante —como de hecho ocurrió— y liberar parte de ese tiempo.

T4 y T5 son las de menor incertidumbre: un ajuste fino de GR00T dura del orden de una hora en una aceleradora, un entrenamiento de ACT menos, y una noche de máquina cubre cientos de tiradas de evaluación. Su duración en el plan la marca la revisión humana de los resultados, no el cómputo.

Las tareas se solapan deliberadamente donde la dependencia es parcial: el *pipeline* de datos se construye sobre la primera demostración grabada, sin esperar al entorno definitivo; el entrenamiento arranca con la primera sesión completa; y la documentación empieza con los primeros resultados. Ese solapamiento es lo que comprime dieciséis semanas de trabajo dependiente en un calendario que, en serie, superaría las veinte.

2.2.3. Planificación de tareas

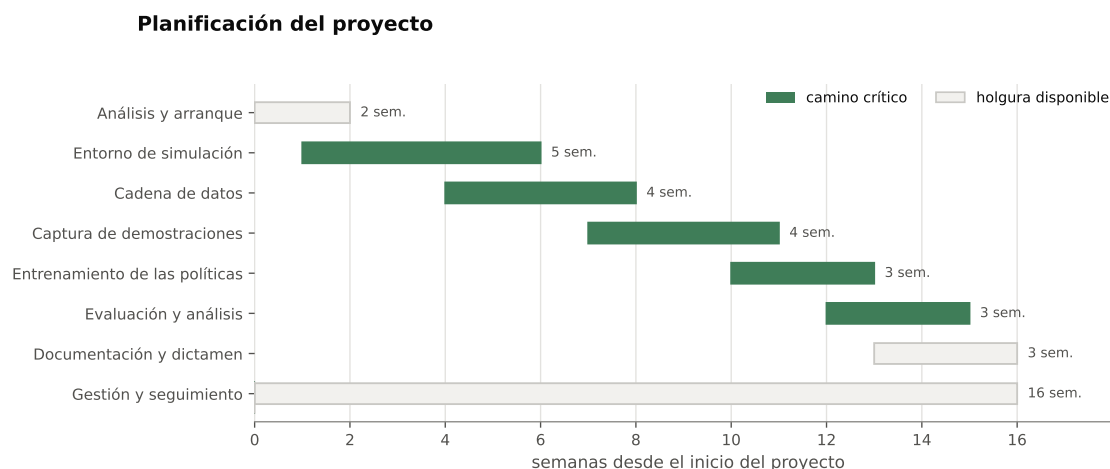


Figura 2.1: Planificación temporal del proyecto simulado, con indicación del camino crítico

El camino crítico —en verde en la figura 2.1— es la secuencia $T1 \rightarrow T2 \rightarrow T3 \rightarrow T4 \rightarrow T5$. No tiene holgura: un retraso de una semana en el entorno es una semana de retraso en el dictamen. Las únicas tareas con holgura son las que no están en él: el análisis inicial, que puede comprimirse, y la documentación, que puede absorber un retraso avanzando en paralelo con la evaluación. La consecuencia para la gestión es que el esfuerzo de control se concentra en las dos primeras tareas del camino, donde está la incertidumbre, y no en las últimas, donde el trabajo es predecible.

2.3. Gestión de recursos

2.3.1. Especificación de recursos

Recursos humanos. Cinco perfiles, ninguno a tiempo completo durante todo el proyecto: un ingeniero de robótica sénior ($T0$, $T1$, $T6$), un técnico de teleoperación ($T3$), un ingeniero de aprendizaje automático ($T2$, $T4$, y apoyo en $T5$), un ingeniero de pruebas ($T5$) y un jefe de proyecto ($T7$). El técnico de teleoperación es el recurso más específico: debe ser una sola persona durante toda la captura, porque el estilo del operador queda grabado en las demostraciones y dos estilos dan un conjunto de datos peor que uno.

Recursos materiales. La estación de trabajo con sus dos aceleradoras y el visor de realidad mixta, descritos en la tabla 2.2, más una red local de baja latencia entre

ambos: la transmisión de vídeo estereoscópico al visor no tolera una red inalámbrica saturada.

Infraestructura. Un contenedor con la pila de simulación, que aísla sus dependencias y permite reconstruirlo; almacenamiento para el conjunto de datos y los puntos de control —del orden de un terabyte—; y un repositorio de control de versiones con los cambios sobre el marco de terceros conservados como parches.

2.3.2. Asignación de recursos

El cuello de botella es la **estación de trabajo**, que es única y la necesitan todas las tareas del camino crítico. La asignación resuelve la contención por dos vías. La primera es la separación por aceleradora: la GPU 0 queda dedicada a la teleoperación y la grabación durante las sesiones de captura, porque la transmisión al visor no tolera compartir la GPU, y la GPU 1 corre en paralelo la aplicación del fondo fotorrealista, los entrenamientos y las evaluaciones. La segunda es la separación por horario: los trabajos largos y sin supervisión —reproducción de sesiones, entrenamientos, bloques de cientos de tiradas— se lanzan por la noche, de modo que la máquina produce veinticuatro horas y las personas la usan de día. Ambas prácticas salen del desarrollo real, donde una noche de máquina cubrió los seis entrenamientos y las seiscientas tiradas de la curva de eficiencia.

La carga humana está equilibrada por construcción: los perfiles se suceden a lo largo del camino crítico —sénior, técnico, aprendizaje automático, pruebas, sénior— y solo el jefe de proyecto acompaña todo el recorrido. El punto de solapamiento de personas está en las semanas 10 a 13, cuando captura, entrenamiento y evaluación coinciden, y es donde la máquina, no las personas, marca el ritmo.

2.4. Gestión de riesgos

2.4.1. Identificación de riesgos

El catálogo de riesgos de la tabla 2.5 se ha construido a partir de los que efectivamente se materializaron durante el desarrollo real, más los propios de cualquier proyecto de esta naturaleza. Es una lista corta a propósito: ocho riesgos que se pueden vigilar valen más que treinta que nadie revisa.

Tabla 2.5: Catálogo de riesgos

Id	Riesgo	Causa y efecto
R1	Inestabilidad de la pila de realidad mixta	Fallos de configuración sin síntoma visible (transmisión en negro, sobreexposición, cierres al arrancar). Bloquea la captura
R2	Incompatibilidad entre versiones de bibliotecas	Las bibliotecas de aprendizaje fijan versiones incompatibles con el simulador. Rompe la evaluación o el entrenamiento
R3	Invalidación de datos ya capturados	Un cambio de decisión o un fallo silencioso del <i>pipeline</i> obliga a descartar o regrabar sesiones
R4	Indisponibilidad de la estación de trabajo	Es única y compartida; una avería o un uso concurrente detiene el camino crítico
R5	Ampliación no controlada del alcance	Mejoras deseables —fondo fotorrealista, más tareas— desplazan el trabajo comprometido
R6	Pérdida de cambios sobre el marco de terceros	Modificaciones no versionadas sobre el simulador se destruyen al actualizarlo
R7	Baja del operador de teleoperación	Es una sola persona; su ausencia detiene la captura y cambiar de operador cambia los datos
R8	Resultados no concluyentes	La diferencia entre métodos es pequeña y el contraste no la acredita. Debilita el dictamen

2.4.2. Análisis de riesgos

Cada riesgo se valora en probabilidad e impacto en una escala de uno a cinco, y su exposición es el producto de ambos. La figura 2.2 los sitúa en la matriz; la tabla 2.6 recoge la valoración, el plan de mitigación —lo que se hace para que no ocurra— y el de contingencia —lo que se hace cuando ocurre.

Los dos riesgos que dominan la matriz son R1 y R3, y son también los dos que se materializaron con más coste en el desarrollo real: la pila de realidad mixta consumió sesiones enteras, y una sesión de datos completa quedó invalidada por un fallo silencioso del proceso de reproducción antes de que nadie lo advirtiera. La lección que incorpora el plan es la misma en ambos casos: verificar cada etapa de extremo a extremo con datos reales antes de invertir esfuerzo en la siguiente, aunque

Matriz de riesgos

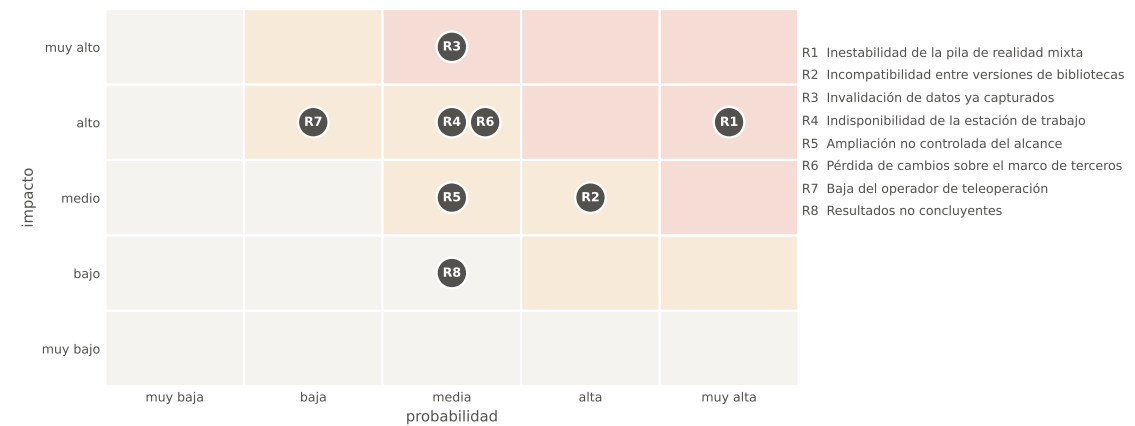


Figura 2.2: Matriz de riesgos del proyecto: probabilidad frente a impacto

ello retrase el arranque de la captura. Es más barato perder una semana en T1 que una sesión en T3.

Tabla 2.6: Análisis de riesgos: valoración, mitigación y contingencia

Id	P	I	P×I	Mitigación	Contingencia
R1	5	4	20	Recetas de lanzamiento verificadas y documentadas; tabla de fallos conocidos; monitor en tercera persona para aislar fallos	Semanas de margen en T1; contingencia presupuestaria
R2	4	3	12	Un entorno de ejecución por método, aislado del simulador; versiones fijadas por escrito	Fijar la última versión compatible y no actualizar
R3	3	5	15	Verificar una demostración de extremo a extremo antes de grabar en bloque; decisiones de captura cerradas antes del conjunto definitivo	Tratar la sesión perdida como prueba del <i>pipeline</i> ; regrabar en sesiones de veinticinco
R4	3	4	12	Reparto por GPU y por horario; trabajos largos desatendidos y reanudables	Reordenar el plan hacia tareas sin máquina (documentación)
R5	3	3	9	Alcance escrito con exclusiones explícitas; toda ampliación pasa por el jefe de proyecto	Diferir la ampliación a un anexo o a una fase posterior
R6	3	4	12	Copia versionada de cada cambio como parche; sincronización periódica	Regenerar desde los parches
R7	2	4	8	Sesiones cortas y frecuentes; documentar el gesto de captura	Reprogramar T3; si se cambia de operador, regrabar el conjunto entero
R8	3	2	6	Diseño emparejado y tamaño muestral suficiente desde el inicio	Presentar el resultado con su incertidumbre; el dictamen es válido aunque no sea concluyente

Capítulo 3

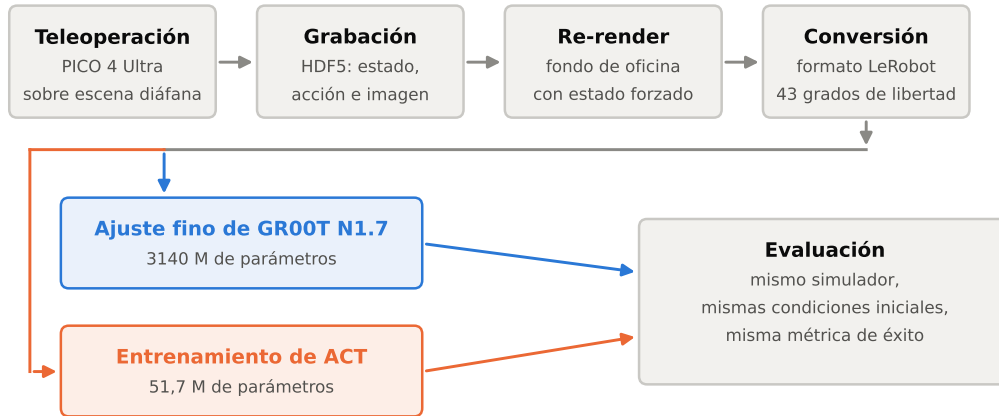
Solución

Este capítulo describe el sistema construido: qué se ha desarrollado, con qué arquitectura, cómo se ha implementado y qué pruebas garantizan que cada etapa hace lo que dice hacer. Es el capítulo más extenso de la memoria porque el trabajo no consiste en aplicar un modelo a un conjunto de datos ya existente, sino en construir la tarea, el conjunto de datos y el banco de evaluación desde cero, y en verificar cada pieza antes de apoyar la siguiente sobre ella.

3.1. Descripción de la solución

Lo desarrollado es un **banco de comparación de políticas de manipulación**: un entorno de simulación con una tarea instrumentada, un *pipeline* de captura y conversión de demostraciones, dos vías de entrenamiento que consumen el mismo conjunto de datos, y un protocolo de evaluación que sirve a ambas a través de la misma interfaz. La figura 3.1 lo resume.

El *pipeline* tiene cinco etapas y cada una consume la salida de la anterior. El operador teleopera al robot con unas gafas de realidad mixta sobre una escena diáfana y el grabador escribe cada demostración en un fichero HDF5 con el estado completo del robot, la acción y la imagen de la cámara de la cabeza. Un proceso de reproducción vuelve a ejecutar cada demostración con el fondo fotorrealista cargado y regraba la imagen, de modo que el conjunto de datos definitivo muestra una oficina real sin que el operador haya pagado su coste de render. Un único conversor produce a partir del HDF5 un conjunto de datos en formato LeRobot que sirve tanto a GR00T como a ACT. Cada método se entrena en su propio entorno de ejecución. Y ambos se evalúan con el mismo cliente, sobre el mismo simulador y las mismas condiciones

De la teleoperación a la tasa de éxito

Las dos vías consumen la misma conversión y se evalúan sobre el mismo banco: cualquier diferencia procede del método.

Figura 3.1: *pipeline* completo: teleoperación, grabación, reproducción con el fondo de oficina, conversión, entrenamiento por las dos vías y evaluación sobre el mismo simulador

iniciales, servidos por un proceso independiente que es lo único que cambia entre una evaluación y otra.

Esa última propiedad es la que da sentido a todo lo demás: el comando que lanza la evaluación es idéntico, byte a byte, para el replay de las acciones grabadas, para GR00T y para ACT. Lo único que cambia es quién escucha en el puerto. Cualquier diferencia en la tasa de éxito procede, por construcción, de la política.

Un módulo instrumental completa el sistema: la reconstrucción fotorrealista de una oficina real mediante *Gaussian Splatting*, que aporta el fondo de todas las imágenes de entrenamiento y evaluación. Su papel se describe en este capítulo donde interviene; su desarrollo técnico completo va en el anexo D.

3.2. El proceso de desarrollo

El desarrollo ha seguido un modelo iterativo e incremental, con ciclos cortos cerrados por una verificación ejecutable. La justificación es específica del dominio: en un *pipeline* de cinco etapas donde cada una consume la salida de la anterior, un error silencioso en una etapa temprana invalida todo el trabajo posterior sin producir ningún síntoma visible. Un fichero HDF5 sin imágenes, una conversión que salta demostraciones en silencio o una política que emite ceros por un problema de dispositivo son indistinguibles, desde fuera, de «la política no aprende». El desarrollo

se ordena, por tanto, de modo que cada etapa quede verificada con datos reales antes de invertir esfuerzo en la siguiente.

En la práctica, los pasos a seguir fueron estos, en orden: crear un entorno en Isaac Sim e integrar la válvula CAD así como el humanoide; fijar la pelvis del robot; grabar una única demostración con imagen, lo que exigió resolver el problema de sincronización del sensor descrito más abajo; decidir el gesto de agarre y regrabar; convertir y verificar una sesión; probar el lazo de evaluación con la reproducción de las acciones grabadas, antes de entrenar nada; entrenar ACT y GR00T sobre esa sesión y evaluarlos; incorporar el fondo fotorrealista por reproducción, y descubrir y corregir que la reproducción no reproducía la demostración; grabar el conjunto definitivo de cien demostraciones; y, por último, los tres experimentos del capítulo 4. El anexo A recoge el historial de confirmaciones que documenta esa secuencia.

3.2.1. Análisis

Definición de requisitos

Los requisitos se derivan directamente de los objetivos del trabajo. Los funcionales describen lo que el sistema hace; los no funcionales, las propiedades sin las cuales la comparación no sería válida.

Especificación de requisitos

Los casos de uso principales son cuatro, y cada uno corresponde a una etapa de la figura 3.1.

Capturar una sesión. Se arranca el servicio de transmisión a las gafas y el grabador; se inicia la teleoperación, se pulsa el botón de inicio y se ejecuta la tarea un número fijado de veces. El entorno reinicia entre demostraciones, sorteando la válvula y escribe cada demostración con éxito en el reinicio siguiente. La sesión termina en un fichero HDF5 que se audita antes de darse por buena.

Convertir un conjunto de datos. A partir de uno o varios ficheros de sesión, se aplica el fondo fotorrealista por reproducción, se fusionan las sesiones y se convierten al formato LeRobot. La conversión se verifica contra el HDF5 de origen.

Entrenar una política. En el entorno de ejecución del método elegido, se calculan las estadísticas del conjunto de datos y se lanza el entrenamiento. El punto

Tabla 3.1: Requisitos funcionales

Código	Descripción
RF-01	La tarea debe poder teleoperarse desde unas gafas de realidad mixta y grabarse en un fichero con el estado del robot, la acción y la imagen de la cámara de la cabeza en cada paso
RF-02	El éxito de un episodio debe determinarlo el propio entorno, sin intervención humana, con un criterio medible: media vuelta del volante; durante la grabación, sostenida diez pasos
RF-03	En cada reinicio, el entorno debe sortear la disposición de la válvula entre frontal y cenital y aplicar una perturbación aleatoria a su posición, y debe ser posible forzar una disposición
RF-04	El fondo fotorrealista debe poder aplicarse a demostraciones ya grabadas sin alterar su trayectoria
RF-05	Una única conversión debe producir un conjunto de datos que alimente a ambos métodos sin transformación adicional
RF-06	El sistema debe entrenar una política GR00T por ajuste fino y una política ACT desde cero a partir de ese conjunto de datos
RF-07	Cualquier política debe evaluarse con el mismo cliente, sobre el mismo entorno y la misma semilla, sirviéndose desde un proceso independiente
RF-08	Por cada episodio de evaluación deben registrarse el éxito, si la válvula llegó a moverse y el ángulo final alcanzado
RF-09	Debe poder medirse la latencia de inferencia de cada política en lazo cerrado, con observaciones reales

de control resultante se compara con el de partida para comprobar que ha cambiado.

Evaluar una política. Se arranca el servidor de la política en su entorno y el cliente de evaluación en el simulador. El cliente ejecuta el número de episodios pedido con la semilla fijada y escribe las métricas por episodio.

Desde el punto de vista del comportamiento, la propiedad que atraviesa los cuatro casos es que ninguna etapa da por buena la anterior sin comprobarla: la auditoría de la sesión, la verificación de la conversión, la comparación de pesos y la reproducción de las acciones grabadas como cota superior son parte de la especificación, no añadidos.

Tabla 3.2: Requisitos no funcionales

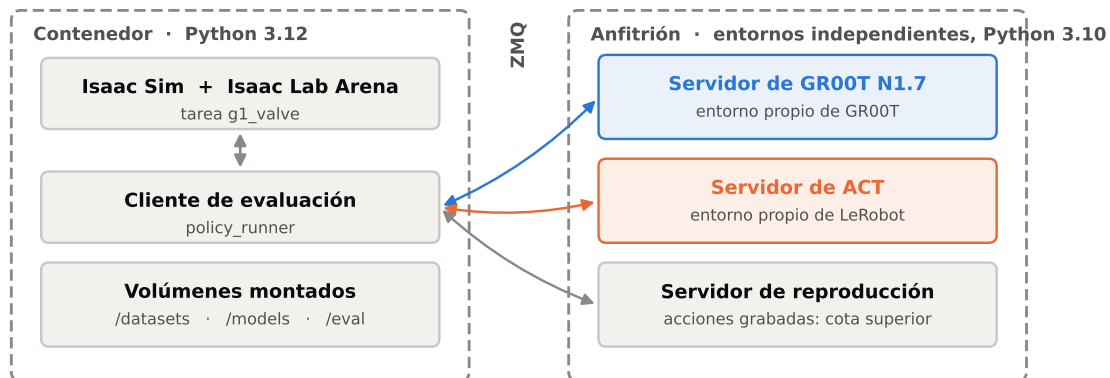
Código	Descripción
RNF-01	Reproducibilidad. Todas las poses de la escena deben estar medidas y documentadas; el reinicio debe ser determinista y las condiciones iniciales de evaluación deben depender solo de la semilla
RNF-02	Latencia. La inferencia de cualquier política debe caber en el presupuesto del lazo de control: un bloque de 40 acciones cada 0,8 segundos
RNF-03	Trazabilidad. Debe poder comprobarse que cada episodio del conjunto convertido procede de la demostración que dice, y que un punto de control entrenado difiere del de partida
RNF-04	Aislamiento. Las bibliotecas de aprendizaje no deben instalarse en el intérprete del simulador, cuyas versiones son incompatibles con ellas
RNF-05	Desatención. Los trabajos largos —reproducción, entrenamiento, evaluación— deben poder ejecutarse sin supervisión y reanudarse sin repetir lo hecho
RNF-06	Fluidez. La teleoperación debe mantener una tasa de imagen en las gafas suficiente para que el operador pueda agarrar un radio del volante

3.2.2. Diseño

Diseño de sistema

El sistema se reparte entre un contenedor que aloja el simulador y dos entornos de ejecución independientes en la máquina anfitriona, uno por método de aprendizaje, comunicados con el simulador mediante un protocolo de mensajería. La figura 3.2 lo muestra.

Esa separación no es estética. Hay **tres intérpretes de Python** y ninguno puede hacer el trabajo de los otros. El del simulador, Isaac Sim, fija sus propias versiones de PyTorch y NumPy y no arranca fuera del contenedor. El de GR00T necesita las dependencias de la versión N1.7 del modelo, que no están en el intérprete del simulador. Y LeRobot, que es la implementación de ACT, exige un NumPy moderno que el intérprete del simulador no admite y fija su propio PyTorch; instalarla en el simulador rompería la herramienta que se necesita para evaluar. Cada uno vive, por tanto, en su sitio: el simulador en el contenedor, y cada biblioteca de aprendizaje

Arquitectura: un cliente, tres servidores intercambiables

Las bibliotecas de aprendizaje fijan versiones incompatibles con las del simulador, así que viven en entornos separados.

Figura 3.2: Arquitectura del sistema: contenedor de simulación, entornos de ejecución de cada política y comunicación entre ambos

en un entorno virtual propio en el anfitrión. Los tres comparten los directorios de datos, modelos y resultados, montados en el contenedor, y comparten la interfaz de red local porque el contenedor usa la red del anfitrión.

La comunicación es un patrón cliente–servidor sobre ZMQ que el marco de simulación ya trae: el simulador es el cliente, envía la observación y recibe un bloque de acciones; la política es el servidor. GR00T, ACT y la reproducción de acciones grabadas implementan la misma interfaz de servidor, y el cliente no sabe cuál de los tres tiene delante. La tabla 3.3 recoge las tecnologías y las versiones exactas empleadas.

Diseño detallado

La tarea de simulación. La tarea se registra en el marco de simulación con el nombre `g1_valve` y se define en un único fichero propio. La escena de captura es deliberadamente diáfana: un plano de suelo, una luz de cúpula y nada más, porque cada elemento visual tiene que renderizarse en tiempo real para que las gafas puedan muestrear la escena. El robot es el Unitree G1 con el *embodiment* `g1_wbc_agile_pink`: las piernas las sostiene la política de equilibrio AGILE y el tren superior lo gobierna la cinemática inversa a partir de las poses de las muñecas que marca el operador, con la cintura bloqueada porque la tarea es estática. Si la pelvis no estuviera bloqueada sería imposible para el operador mantener el equilibrio de todo el cuerpo a la vez que intenta girar la válvula, el robot se desequilibraría y se giraría sobre sí mismo.

Tabla 3.3: Tecnologías empleadas y versiones

Componente	Versión	Función
Ubuntu	24.04.4 LTS	Sistema operativo del anfitrión
Docker	29.6.2	Contenedor del simulador
Controlador NVIDIA	580.173	Acceso a las dos GPU; CUDA 12.8
Isaac Sim	6.0.0	Simulador: física PhysX, render RTX, escenas USD
Isaac Lab	3.0.0	Marco de tareas y gestores sobre Isaac Sim
Isaac Lab Arena	0.2.1	Composición de escenas, teleoperación, grabación, evaluación y política remota
Isaac-GR00T	N1.7	Modelo, ajuste fino y servidor de GR00T
LeRobot	0.3.3	Formato de datos y entrenamiento de ACT
PyTorch	2.10 (contenedor), 2.7.1 (anfitrión)	Aprendizaje profundo, compilado para CUDA 12.8
Python	3.12 (contenedor), 3.10 (anfitrión)	Tres intérpretes independientes
CloudXR e IsaacTeleop	de Isaac Lab Arena	Transmisión estereoscópica a las gafas y retorno de poses por OpenXR
PICO 4 Ultra	—	Visor de realidad mixta y mandos
Pink IK	de Isaac Lab Arena	Cinemática inversa de los brazos
AGILE	red ONNX	Política de equilibrio de cuerpo completo
NuRec	de Omniverse	Render del fondo por <i>Gaussian Splatting</i>
ZMQ	pyzmq	Cliente-servidor entre simulador y políticas
HDF5 y Parquet	h5py 3.16, pyarrow	Formatos de grabación y de conjunto de datos

La válvula procede de un modelo CAD de una válvula de compuerta real, envuelto en una capa USD propia que hace tres cosas, todas necesarias y ninguna evidente. Primera, **reescala el recorrido del volante**: el CAD lo declara de $539,7^\circ$ a $2879,8^\circ$ —seis vueltas y media, fiel a una válvula multivuelta—, pero el marco normaliza la apertura sobre los límites de la articulación y el éxito es apertura mayor que 0,5, de modo que con los límites originales cada demostración habría exigido tres vueltas y cuarto completas dentro de un episodio. Eso es inalcanzable por teleoperación, todos los episodios habrían muerto por tiempo y ninguno se habría escrito jamás en el fichero. El recorrido se reescala a $0-360^\circ$, y así media vuelta es el umbral de éxito. Segunda, **conserva los materiales** físicos que el CAD referencia fuera del subárbol —fricción estática 1,2 y dinámica 1,0—, sin los cuales los dedos resbalan en el radio. Tercera, **quita el freno**: el modelo traía un amortiguamiento viscoso de 100 que exigía del orden de $1000 \text{ N}\cdot\text{m}$ para girar a $10^\circ/\text{s}$, cuando la mano del G1 entrega unidades; se baja a 0,01.

La válvula adopta en cada reinicio una de dos **disposiciones**, sorteadas al 50 %, que la figura 3.3 muestra. En la **frontal** el volante mira al robot con el eje horizontal —una válvula sobre una línea vertical, que se gira como un timón— y su centro queda en $(0,40, 0, 0,80) \text{ m}$ respecto al pelvis, justo por encima de la altura de reposo de las muñecas, 0,762 m. En la **cenital** el volante mira hacia arriba con el eje vertical —una válvula sobre una línea horizontal, que se alcanza por encima— y su centro queda en $(0,30, 0, 0,66) \text{ m}$: más cerca y más bajo, porque girar desde arriba exige acercar el codo al cuerpo y el volante debe quedar por debajo de las muñecas. Ninguna de esas cifras es un cálculo: las dos distancias se ajustaron con el operador dentro de las gafas, y la cenital pasó por 0,35 m —lejos— y 0,20 m —encima del propio torso, como predice el radio de 0,10 m del volante— antes de fijarse en 0,30 m. A la posición de cada disposición se suma en cada reinicio una perturbación uniforme de $\pm(4, 6, 3) \text{ cm}$, acotada por el alcance del brazo.

El robot aparece **ya de pie**, con el pelvis a 0,74 m. Es la línea más importante del fichero de la tarea, y se justifica con una medida. Si aparece con el pelvis a la altura del suelo, la política de equilibrio lo levanta a través del espacio que ocupa la válvula, y el resultado es un reinicio distinto cada vez y alguna caída. Medido sobre cinco reinicios: con el robot en el suelo y la válvula delante, la dispersión de la pose final es de $(0,300, 0,149, 0,310) \text{ m}$ y cae una de cinco veces; apareciendo de pie, la dispersión es de $(0,001, 0,001, 0,001) \text{ m}$ y no cae ninguna. No es una cuestión de orden: es una propiedad de calidad de los datos, porque cada demostración empieza



Figura 3.3: Las dos disposiciones de la válvula: frontal, con el eje de giro horizontal, y cenital, con el eje vertical. Vista de tercera persona con el fondo de oficina

desde la misma pose y una tirada fallida en evaluación es atribuible a la política y no a un mal reinicio.

El **pelvis está además fijado al mundo** durante la manipulación. La decisión inicial era mantenerlo libre bajo la política de equilibrio, por realismo, y la teleoperación la invalidó en la primera sesión: al girar el volante, el par de reacción llega por los brazos al tronco, y con un tronco libre la política de equilibrio lo absorbe de la única manera que puede, girando al robot entero. Cada demostración habría codificado esa deriva como parte de la tarea, y las poses de las muñecas, expresadas en el sistema de referencia del pelvis, habrían hecho que gestos idénticos parecieran acciones distintas. Con el pelvis fijado, la deriva medida sobre tres reinicios es de (0,000, 0,000, 0,000) m; la política de equilibrio sigue activa sobre las piernas, y el espacio de acción no cambia. Es la única concesión de realismo del entorno y se declara como tal en las limitaciones del trabajo.

El **criterio de éxito** es apertura mayor que 0,5, media vuelta. Durante la grabación se exige además que se sostenga diez pasos consecutivos, para que un roce no cuente como demostración; en evaluación el episodio termina en el paso en que se supera. El entorno avanza a 50 Hz (paso de física de 5 ms con decimación 4). El presupuesto de episodio es de 15 s por defecto; durante la captura se eleva a 30 s, porque el episodio termina solo al alcanzar el éxito y un presupuesto generoso solo alarga

los fallos, mientras que uno corto convierte un casi-éxito en un tiempo agotado que no se escribe; en evaluación se fija en 20 s.

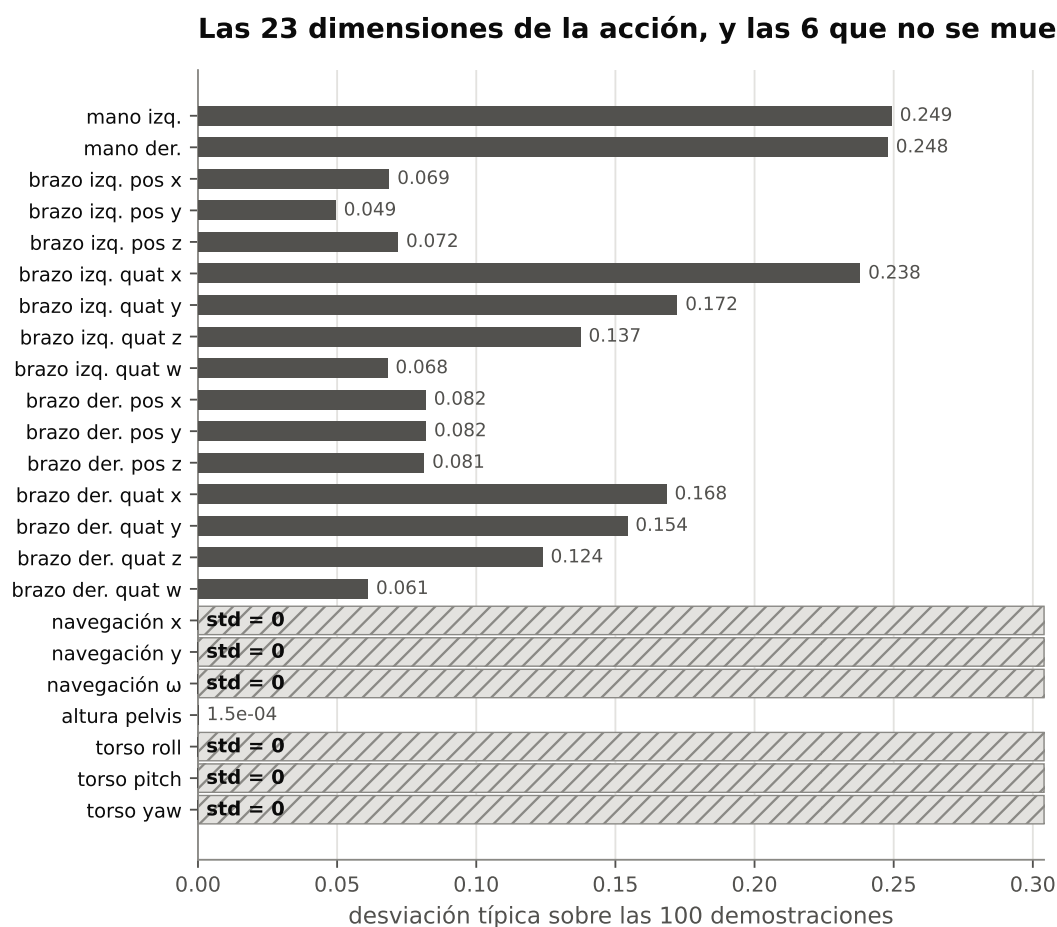
El espacio de acción. El operador gobierna al robot a través de un vector de **veintitrés dimensiones**, cuya composición recoge la tabla 3.4. Las poses de las muñecas se expresan en el sistema del pelvis; la cinemática inversa las convierte en los ángulos de las articulaciones de los brazos, y la política de equilibrio recibe los canales de mando de la base.

Tabla 3.4: Composición del espacio de acción de veintitrés dimensiones

Índices	Componente	Dim.	Contenido y estado en el conjunto de datos
0, 1	Estado de cada mano	2	Cierre de la mano. Vale $0,5 \cdot \text{gatillo} - 0,5 \cdot \text{empuñadura}$; el entorno lee 0 como mano abierta
2–4, 5–8	Muñeca izquierda	7	Posición xyz y cuaternión $xyzw$ en el sistema del pelvis
9–11, 12–15	Muñeca derecha	7	Ídem
16–18	Locomoción	3	Velocidad de avance, lateral y de giro. Congelada a cero: dimensiones muertas
19	Altura del pelvis	1	Constante 0,72 m: casi muerta
20–22	Orientación del torso	3	Alabeo, cabeceo y guiñada. Cintura bloqueada: dimensiones muertas

Dos propiedades de ese vector condicionaron el trabajo. La primera es el **estado de las manos**, que no es un bit de abierto o cerrado sino la combinación del gatillo y la empuñadura del mando, con signos opuestos: apretar los dos a la vez da cero, y cero significa mano abierta para el entorno. Pulsar más fuerte puede abrir la mano. Por eso el gesto de captura se fijó antes del conjunto definitivo —solo el gatillo— y por eso la primera demostración grabada, en la que el volante se giró empujando los radios con la mano abierta, tenía esas dos dimensiones constantes a cero. La segunda es que **seis de las veintitrés dimensiones tienen desviación típica exactamente cero** en el conjunto definitivo —las tres de locomoción, congeladas para que el operador no pudiera alejar al robot con el mando, y las tres de orientación del torso, bloqueadas—, y la altura del pelvis es casi constante. La figura 3.4 lo muestra. Una normalización por media y desviación divide por esos ceros; se dejaron como

están, documentadas, porque eliminarlas habría introducido una variable nueva en un experimento ya cerrado, y se señalan como línea de trabajo futuro.



6 de 23 dimensiones tienen desviación típica exactamente cero. La normalización divide por ese valor: o da división por cero, o un épsilon que amplifica el ruido hasta hacerlo señal.

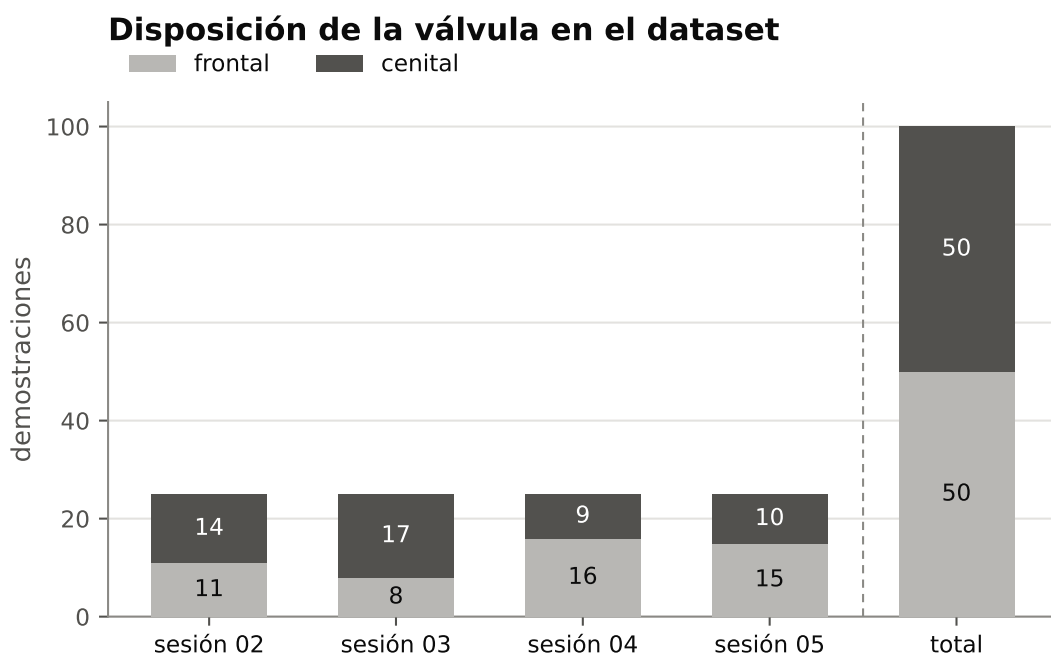
Figura 3.4: Desviación típica de cada una de las veintitrés dimensiones del espacio de acción sobre el conjunto de datos definitivo. Seis son exactamente cero

El conjunto de datos. Por cada demostración con éxito, se escribe un grupo en el fichero HDF5 con la acción de veintitrés dimensiones, la observación —posición de las cuarenta y tres articulaciones, poses de las muñecas, pose del robot— y la imagen RGB de la cámara de la cabeza a 640×480 píxeles, todo a 50 Hz, más el estado inicial completo de la escena y, en cada paso, el estado de todas las articulaciones del robot y de la válvula. Ese último detalle, que en el grabador de referencia es un subproducto, resultó ser lo que hace posible la reproducción exacta con el fondo fotorrealista (anexo D).

La conversión al formato LeRobot no toma como acción las veintitrés dimensiones del operador sino las **cuarenta y tres posiciones articulares** que la cinemática inversa produce a partir de ellas: es el espacio de articulaciones para el que existen la descripción del *embodiment* del G1 y el modelo de partida de GR00T, y es lo que ambas políticas aprenden a emitir. La consecuencia es que en evaluación el simulador se lanza con otro *embodiment*, `g1_wbc_agile_joint`, que acepta directamente esas posiciones articulares junto con los canales de mando de la base. Nada en el marco de simulación prueba ese viaje de ida y vuelta —la salida de la cinemática inversa como entrada del controlador—, y por eso se probó con la reproducción de las acciones grabadas antes de entrenar nada.

El conjunto definitivo son **cien demostraciones**, grabadas en cuatro sesiones de veinticinco —las sesiones 02 a 05; la 01, grabada empujando el volante con la mano abierta, es una prueba del *pipeline* y no entra—, todas con éxito, con la mano cerrada sobre un radio y la válvula sorteada en cada reinicio. El sorteo salió exactamente 50 frontales y 50 cenitales sin que nadie lo forzara: las sesiones 02 y 03 se escoraron hacia la cenital (19 frente a 31) y las 04 y 05 hacia la frontal (31 frente a 19), como muestra la figura 3.5. Las demostraciones duran entre 156 y 523 pasos —de 3,1 a 10,5 segundos, 6,5 de media— y suman 32 573 pasos, once minutos de demostración (figura 3.6). Alguna mano está cerrada en el 61 % de los pasos, de media. Tras la reproducción con el fondo de oficina y la conversión, el conjunto LeRobot tiene 32 473 fotogramas —el conversor descarta el último de cada demostración—, ocupa 100 MB, y su verificación contra el HDF5 de origen no arrojó ningún fallo.

La interfaz de las políticas. Toda política se sirve como un proceso que implementa la interfaz de servidor del marco: recibe la observación —la imagen de la cámara, el estado articular y la instrucción en lenguaje natural, «*turn the handwheel to open the valve*»— y devuelve un bloque de cuarenta acciones, que el cliente ejecuta a 50 Hz antes de pedir el siguiente. Cuarenta pasos son el horizonte del modelo de partida de GR00T, y ACT se entrena con el mismo tamaño de bloque en lugar de su valor por defecto de cien, para comparar ambas políticas al mismo horizonte y para que la demostración más corta no sea en su mayoría relleno. La acción que viaja tiene cincuenta dimensiones: las cuarenta y tres articulaciones más los siete canales de mando de la base, que se rellenan con sus valores de reposo, leídos de las estadísticas del conjunto de datos —la altura del pelvis, en particular, a 0,72 m, porque un cero en esa posición sienta al robot en el suelo.



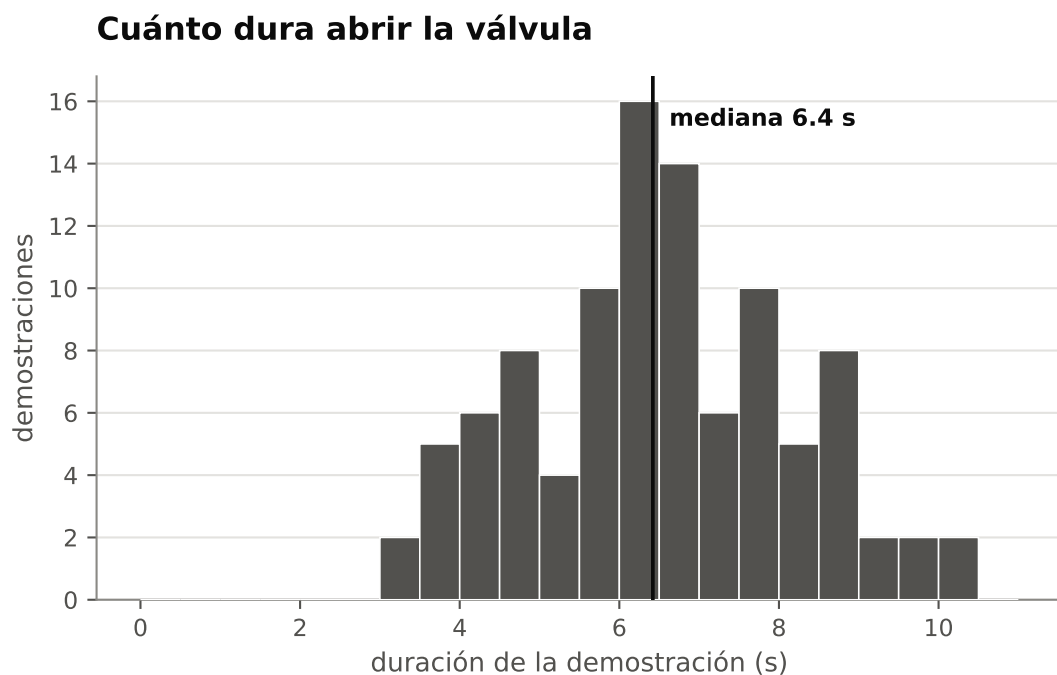
La disposición se sortea al 50 % en cada reset; nadie forzó el equilibrio. Las sesiones 02-03 salieron cargadas de cenitales y las 04-05 al revés, y el total quedó en 50/50 por casualidad.

Figura 3.5: Reparto de las dos disposiciones de la válvula en cada sesión de captura y en el conjunto acumulado

3.2.3. Implementación

El código propio vive en un repositorio propio, separado del marco de simulación de terceros, y se organiza en tres partes: `sim/`, con todo lo que define la tarea —el fichero del entorno, la válvula y su capa USD, la configuración de conversión, los parches sobre el marco y una veintena de guiones de medida y diagnóstico—; `train/`, con los guiones de reproducción, conversión, verificación, servicio de políticas, subconjuntos, medida de latencia y figuras; y `docs/`, con las guías de lanzamiento y de entrenamiento, los informes de resultados y esta memoria. El simulador se ejecuta desde un *checkout* del marco de terceros, y un guion de sincronización mantiene ambas copias en correspondencia; los cambios sobre ficheros del marco se conservan como parches contra la versión oficial, no como copias, para que quede explícito qué se ha tocado (anexo A).

Las decisiones de implementación que condicionaron el resultado, y los problemas que hubo que resolver para que el *pipeline* funcionase, son los siguientes. Cada uno costó al menos una sesión, y ninguno produjo un mensaje de error que apuntara a su causa.



n = 100 demostraciones, 11 min en total. El episodio se corta solo al detectar el éxito, así que esto mide la tarea, no el presupuesto de 30 s con el que se grabó.

Figura 3.6: Distribución de la duración de las cien demostraciones capturadas

La sincronización del sensor con la construcción del entorno. Grabar con cámara bajo realidad mixta fue imposible durante un día entero. El marco inicializa la simulación y actualiza la escena antes de construir los gestores de observación, pero esa actualización no renderiza, de modo que cuando el gestor pide a la cámara su primer fotograma la cámara no tiene ninguno. Sin realidad mixta el problema se resuelve solo; con ella, la cámara bloquea el proceso para siempre, y con la física en GPU la misma situación aflora como un acceso ilegal a memoria dentro del motor de física. Las escenas con un fondo USD grande funcionaban por accidente, porque su tiempo de carga dejaba que un fotograma se produjera antes, y eso hizo parecer durante horas que el problema era el contenido de la escena. Se descartaron por medida el suelo, sus texturas, la geometría visible, la luz y el antialiasing. La corrección es un parche de cuatro renders antes de construir los gestores, activo solo cuando hay sensores de render en la escena.

Física en CPU durante la captura. La combinación de física en GPU, realidad mixta y cámaras mata el entorno al crearse, siempre a los 23 segundos. Se bisecó: física en GPU con cámaras y sin realidad mixta funciona, con realidad mixta y sin cámaras funciona, y los tres juntos fallan. La captura se hace con la física en CPU, que no recorre ese camino del motor, y no cuesta tasa de imagen: los 20 fotogramas

por segundo del visor los fijan la calidad de render y el modo de iluminación, no el dispositivo de física. Es la razón de que los sesenta y cuatro núcleos de la estación no sean un exceso.

La pila de realidad mixta. Cuatro problemas independientes, cada uno con un síntoma idéntico —imagen negra o inutilizable— y ninguno con un error. El anclaje del visor quedaba a la altura del pelvis por un fallo al crear la primitiva de anclaje, resuelto con un parche que la crea con USD crudo. El antialiasing forzado por el marco activaba un modo de render que sobreexponía la escena, resuelto haciéndolo opcional. El visualizador del simulador no existe salvo que se pida explícitamente, de modo que el monitor de tercera persona transmitía un búfer vacío. Y el cortafuegos del anfitrión descarta en silencio los paquetes a puertos sin regla, con el agravante de que el servicio de transmisión a las gafas ocupa un puerto fijo que coincide con el del monitor, de modo que este último hubo que moverlo. El monitor de tercera persona, comparado con lo que veía el operador, fue la herramienta que aisló dos de los cuatro.

Tres intérpretes y sus fallos silenciosos. El entrenamiento de GR00T sobre dos GPU se queda parado antes de empezar, con ambos procesos consumiendo CPU y ninguna GPU trabajando; en una sola GPU entrena a 3,2 pasos por segundo, diez mil pasos en menos de una hora, así que no merecía la pena depurarlo. ACT emitía exactamente ceros, y no era el entrenamiento: mover la política a la segunda GPU por su índice corrompe los búferes de normalización sin lanzar nada, la desviación típica queda a cero, el estado normalizado se dispara y la red satura. Se sirve seleccionando la GPU por variable de entorno, y el servidor comprueba los búferes al arrancar y se niega a servir si los ve mal. Y el entrenador de GR00T reanuda desde cualquier punto de control que encuentre en el directorio de salida: uno viejo con el paso final ya alcanzado ejecuta cero pasos y guarda pesos intactos con un registro impecable; de ahí que cada entrenamiento use un directorio nuevo y que el punto de control se compare siempre con el de partida.

El fondo fotorrealista, aplicado por reproducción. La figura 3.7 muestra la misma demostración en la escena diáfana en la que se grabó y con el fondo de oficina que llega al conjunto de datos. El fondo cuesta unos 39 ms por paso y en el visor eso es la diferencia entre poder agarrar un radio y no poder, así que se aplica después, sin gafas y sin nadie delante: unos doce minutos por sesión de veinticinco. La primera versión reproducía las acciones grabadas en lazo abierto y **no reproducía la demostración**: las piernas las gobierna la política de equilibrio en lazo cerrado y no se reproducen desde el fichero, así que el estado diverge desde el primer paso

y el volante acaba, en el peor caso, 80° corto. La versión definitiva impone en cada paso el estado grabado de la escena, y las cincuenta demostraciones de prueba se reproducen con una diferencia máxima de 1,1°. El anexo D documenta el diagnóstico y lo relaizado en la implementación.

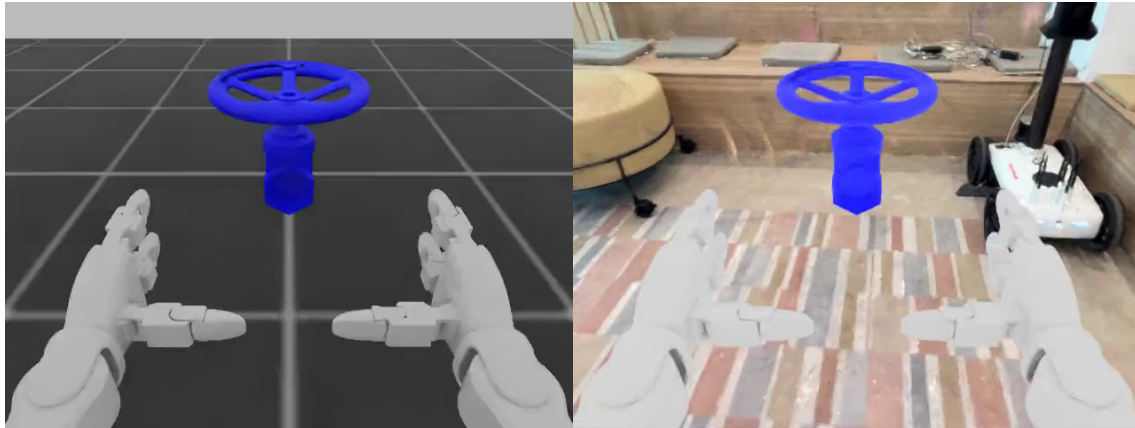


Figura 3.7: Escena diáfana empleada durante la teleoperación, frente a la misma demostración con el fondo de oficina reconstruido, que es la que llega al conjunto de datos. Vista de la cámara del robot

El agarre y el gesto. La primera demostración grabada giró el volante empujando los radios con la mano abierta, y funcionó. Se decidió agarrar, porque es más defendible en el contexto industrial y porque un agarre es lo que transmite par a una válvula dura. Fijado el gesto —solo el gatillo—, se grabó una demostración, se comprobó con el auditor que las dimensiones de las manos ya no eran constantes, y solo entonces empezaron las sesiones. Los dos mandos registran el cierre con el mismo signo, que no es lo que la documentación del retargeter predice; se dejó así porque es uniforme, y con la condición de que el gesto no cambiara a mitad del conjunto de datos.

3.2.4. Pruebas

Cada etapa del *pipeline* tiene una prueba ejecutable, y ninguna se apoya en mirar el resultado a ojo. La tabla 3.5 las recoge con el resultado obtenido sobre el sistema definitivo. La pieza central es la **reproducción de las acciones grabadas** como cota superior del sistema: si reproducir las órdenes de una demostración no abre la válvula, ninguna política entrenada sobre ellas lo hará, y eso debe saberse antes de invertir en la captura masiva. Se hizo antes de entrenar nada, y de haber fallado habría obligado a cambiar la representación de la acción con el conjunto de datos aún por grabar, no después.

Tabla 3.5: Pruebas de verificación del *pipeline*

Prueba	Qué verifica	Resultado
Medida de la escena	Pose real del volante y del robot tras el reinicio, con repeticiones para obtener la dispersión	Dispersión de 1mm; 0 caídas en 5 reinicios
Sorteo de la válvula	Que las dos disposiciones aparecen y la posición varía dentro de la perturbación pedida	Ambas presentes; perturbación dentro de $\pm(4, 6, 3)$ cm
Render vivo	Que el visualizador produce imagen antes de culpar a la red	PNG del viewport con contenido desde el segundo fotograma
Auditoría de sesión	Demostraciones, éxito, forma de la acción, dimensiones constantes y fotogramas negros de cada HDF5	100/100 con éxito; manos no constantes; 0 fotogramas negros
Reproducción con fondo	Que cada demostración reproducida conserva sus pasos, cruza el umbral y cambia de imagen	50/50 reproducen; diferencia máxima de $1,1^\circ$
Verificación del conjunto	Longitudes, contigüidad de índices, procedencia de cada episodio y estadísticas contra el HDF5	0 fallos sobre las 100 demostraciones
Cota superior	Reproducción de las acciones grabadas a través de la interfaz de evaluación	3/3 éxitos; la válvula se mueve en el 100 %
Comparación de pesos	Que el punto de control entrenado difiere del de partida en las capas entrenables y no en las congeladas	493 tensores congelados sin cambio; 537 entrenables con cambio
Búferes de ACT	Que la desviación típica de normalización no es cero antes de servir	Comprobación al arrancar; rechaza servir si falla
Emparejamiento	Que ambas políticas ven las mismas condiciones iniciales	Poses idénticas hasta 10^{-9} m
Latencia	Que ninguna inferencia supera el presupuesto del lazo	226 medidas, máximo 133 ms frente a 800 ms

Hay además una prueba que falló y se documenta como tal: el intento de girar el volante aplicando un par externo conocido devuelve siempre cero grados, incluso con 50 N·m, porque el camino de fuerzas externas es una operación nula en este *backend* de física. No invalida nada —la válvula gira cuando la mano la empuja— pero descarta esa vía para caracterizar su dureza.

3.3. El producto del desarrollo

El entregable es el repositorio, público, con tres piezas que un tercero necesita para reproducir el trabajo:

`https://github.com/izanvines/TFM\_ValveOperator`

La **guía de lanzamiento** recoge, verificados, los cuatro modos de ejecución del simulador —ver la escena, teleoperar, grabar y aplicar el fondo—, las tres líneas que van en todo comando y qué pasa si faltan, los puertos, las variables de entorno y una tabla de fallos típicos con su causa. La **guía de entrenamiento** empieza donde acaba la anterior: qué corre en cuál de los tres intérpretes, la conversión, las estadísticas, la prueba del lazo, los dos entrenamientos, cómo servir cada política y cómo saber si un entrenamiento ha entrenado de verdad. Y los **guiones**, que son los que ejecutaron el sistema definitivo: el *pipeline* completo de las cien demostraciones, los entrenamientos, las evaluaciones y la noche de la curva de eficiencia, todos reanudables.

Lo que recibe la política es una sola imagen, la de la cámara de la cabeza del robot, y conviene verla para calibrar la dificultad de la tarea. La figura 3.8 muestra el primer y el último fotograma de una demostración frontal y de una cenital, tomados del conjunto de datos definitivo: lo que quede fuera de ese encuadre no existe para la política.

El repositorio incluye también vídeos de tercera persona de cada política abriendo la válvula, con el fondo de oficina, y las figuras de la memoria en formato vectorial junto con los guiones que las generan a partir de los ficheros de resultados, de modo que cualquier cifra del capítulo 4 puede regenerarse desde los datos.

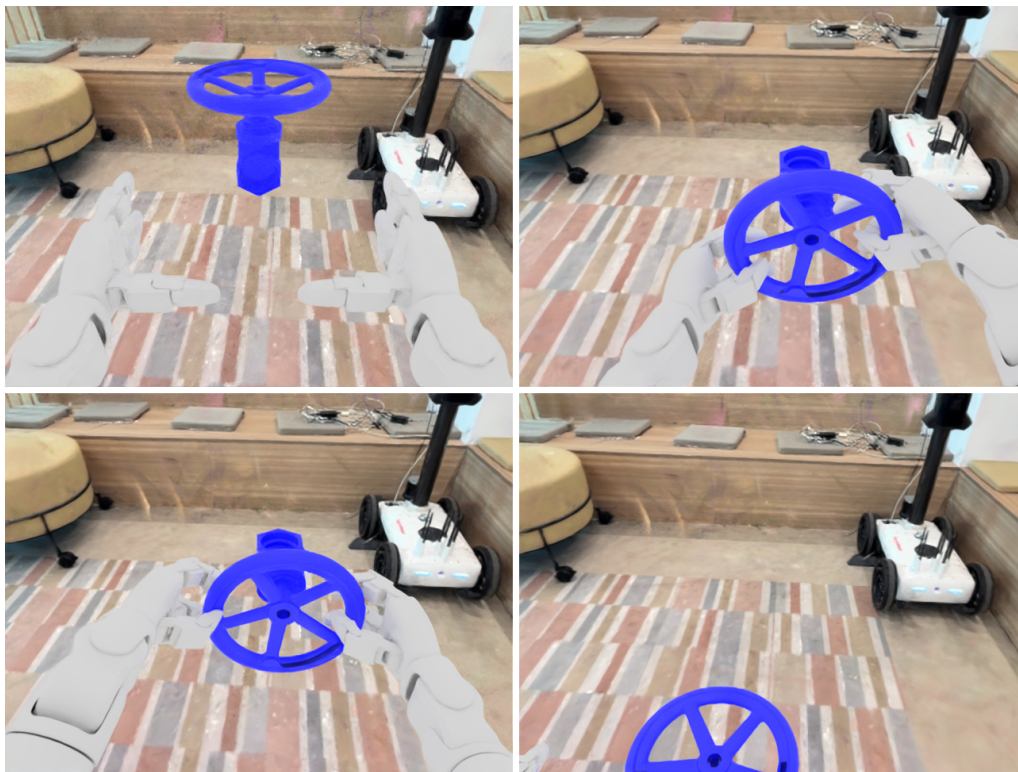


Figura 3.8: La única entrada visual de la política: la cámara de la cabeza del robot, 640×480 píxeles. Arriba, primer y último fotograma de una demostración frontal; abajo, de una cenital. Fotogramas del conjunto de datos definitivo, con el fondo de oficina

Capítulo 4

Evaluación

Este capítulo demuestra la validez de la solución construida y responde a la pregunta del trabajo. Describe primero el protocolo experimental —qué se mide, cómo se mide y por qué el diseño permite atribuir las diferencias observadas al método y no al montaje— y analiza después los resultados de los tres experimentos realizados: la comparación principal entre las dos políticas sobre doscientas tiradas emparejadas, la curva de eficiencia en datos y la medida de latencia de inferencia en lazo cerrado.

Se ha puesto especial cuidado en separar lo que los datos sostienen de lo que sugieren. Varias de las conclusiones de este capítulo son deliberadamente más débiles de lo que las cifras brutas invitarían a escribir, y en cada caso se explica por qué. Todas las cifras proceden de los ficheros de resultados que el simulador escribe por episodio, y las figuras se regeneran a partir de ellos con los guiones del repositorio.

4.1. Proceso de evaluación

4.1.1. Forma de evaluación

Métricas

Se emplean dos métricas, ambas registradas automáticamente por el simulador en cada episodio:

Tasa de éxito. Proporción de episodios en los que el volante supera media vuelta de apertura —180°— dentro del presupuesto de tiempo. El episodio termina en el mismo paso en que se supera. Es la métrica principal, y es binaria: un episodio que se queda en 178° cuenta lo mismo que uno que no pasa de 60°.

Tasa de movimiento de la articulación. Proporción de episodios en los que el volante llega a moverse más de un 5 % de su recorrido, con independencia de cuánto. Distingue el fallo por no completar el giro del fallo por no llegar siquiera a la válvula.

Se acompañan de dos magnitudes continuas que el fichero de resultados permite recuperar y que la tasa de éxito, por ser binaria, esconde: el **ángulo máximo alcanzado** por el volante en cada episodio, que convierte el contador de fallos en una distribución, y el **paso en el que se cruza el umbral**, que mide cuánto tarda cada política en resolver la tarea cuando la resuelve.

El banco es común a las dos políticas

Las dos políticas se sirven como procesos independientes que implementan la misma interfaz de comunicación con el simulador. El comando que lanza la evaluación es idéntico para ambas hasta el último carácter: mismo entorno, mismo *embodiment*, mismo fondo, misma semilla, mismo presupuesto de episodio y misma instrucción en lenguaje natural. Lo único que cambia es qué proceso atiende la petición en el puerto. Cualquier diferencia en la tasa de éxito procede, por construcción, de la política. La tabla 4.1 fija los parámetros.

Tabla 4.1: Parámetros fijos de toda evaluación

Parámetro	Valor
Entorno y <i>embodiment</i>	g1_valve , g1_wbc_agile_joint (43 articulaciones + 7 canales de base)
Fondo	Oficina reconstruida, el mismo con el que se generaron las imágenes de entrenamiento
Condiciones iniciales	Disposición sorteada al 50 % y posición perturbada $\pm(4, 6, 3)$ cm, a partir de la semilla
Semillas	42 (bloque 1) y 7 (bloque 2); 100 episodios por bloque y política
Presupuesto de episodio	20 s, es decir 1000 pasos a 50 Hz
Éxito	Apertura mayor que 0,5; el episodio termina en el paso en que se supera
Bloque de acciones	40 pasos por inferencia, para ambas políticas
Instrucción	<i>turn the handwheel to open the valve</i>
Puntos de control	GR00T: paso 10 000 del ajuste fino; ACT: paso 20 000

El diseño es emparejado

Las condiciones iniciales de cada episodio —disposición y posición de la válvula— se generan a partir de la semilla, de modo que la tirada i de una política se enfrenta exactamente a la misma escena que la tirada i de la otra. No es una suposición: las poses de la válvula que ambos ficheros de resultados registran en el estado inicial de cada episodio coinciden con diferencia máxima exactamente cero en las doscientas tiradas. Cada política vio la misma válvula, en la misma posición, en el mismo orden.

Esto convierte el experimento en un diseño de medidas emparejadas, y el contraste apropiado es la **prueba exacta de McNemar** sobre los casos discordantes: solo los episodios en los que una política acierta y la otra falla llevan información sobre la diferencia entre ambas. Una comparación de proporciones independientes desaprovecharía el emparejamiento y sobreestimaría la incertidumbre. Los intervalos de confianza de las tasas individuales se calculan por el método de Wilson, que a diferencia del intervalo normal se mantiene dentro de $[0, 1]$ con proporciones próximas a los extremos, que es justo donde caen estos resultados. Para el efecto de la disposición, que compara dos grupos distintos de episodios dentro de una misma política, se emplea la prueba exacta de Fisher. Ningún contraste presupone normalidad; con tamaños de muestra de decenas de casos discordantes, las versiones exactas son las que corresponden.

4.1.2. Casos de prueba

Ensayo previo del *pipeline*

Pasada completa con las veinticinco demostraciones de la primera sesión —una sola disposición, sin agarre, sin perturbación de la posición—, con el único objetivo de recorrer el *pipeline* entero con datos reales y localizar dónde se rompe antes de grabar el conjunto definitivo. Se evaluaron tres servidores con el mismo cliente: la reproducción de las acciones grabadas, como cota superior, resolvió 3 de 3; GR00T, 10 de 10 tanto a mil como a diez mil pasos de ajuste fino; ACT, 10 de 10. Ambas políticas abrían la válvula más deprisa que la demostración de la que aprendieron —183 y 208 pasos de media frente a los 256 de la reproducción— y con un giro monótono el 95–98 % del tiempo, lo que descarta que fueran un reloj memorizado. Pero **un 10 de 10 contra un 10 de 10 no distingue nada**, que es exactamente lo que una comparación tiene que hacer: con la válvula siempre en la misma pose y el mismo reinicio, la tarea era trivial. El ensayo acredita que el *pipeline* funciona; no compara métodos. Sí produjo tres decisiones que cambiaron el conjunto definitivo: el

agarre, el margen sobre el umbral de éxito y la iluminación de evaluación, descritas en el capítulo 3.

Comparación principal

Doscientas tiradas por política sobre las políticas entrenadas con las cien demostraciones definitivas, en dos bloques de cien con semillas distintas. El segundo bloque se añadió cuando el primero dejó la diferencia entre políticas en $p = 0,134$: doblar la muestra es lo que corresponde ante una diferencia pequeña, y los dos bloques son sorteos distintos de las mismas distribuciones, lo que hace legítimo agruparlos.

Curva de eficiencia en datos

Reentrenamiento de ambos métodos sobre subconjuntos **estratificados y anidados** de veinticinco y cincuenta demostraciones —12 frontales y 13 cenitales, 25 y 25— con la misma receta y la misma semilla de entrenamiento que el conjunto completo, y evaluación de las seis configuraciones resultantes sobre las mismas cien condiciones iniciales del primer bloque. La estratificación es esencial: las sesiones 02 y 03 se escoraron hacia la disposición difícil (19 frontales frente a 31 cenitales), de modo que tomar «las primeras veinticinco» habría confundido *menos datos* con *más disposición difícil* y la curva habría caído por el motivo equivocado.

Latencia de inferencia

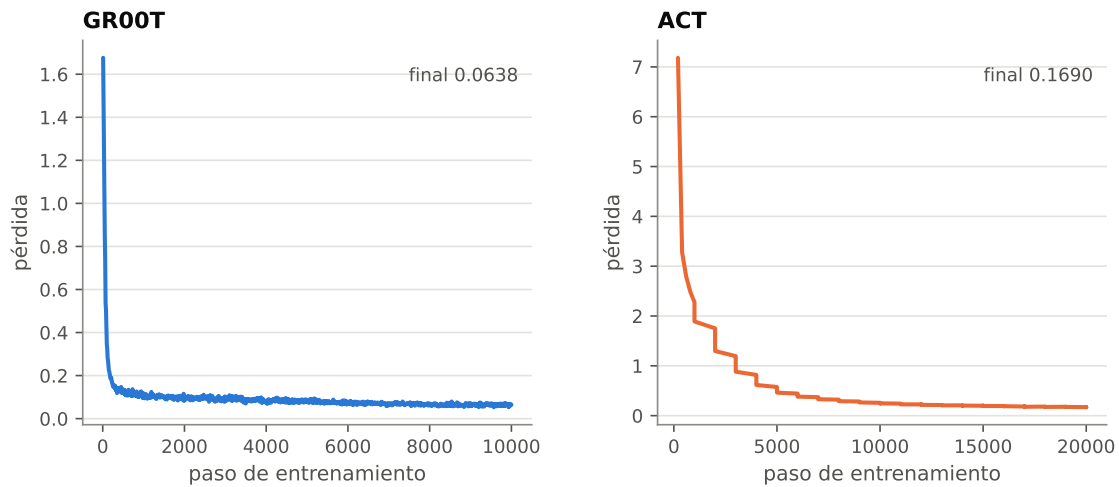
Medida del tiempo de cálculo de cada bloque de acciones **en lazo cerrado dentro del simulador**, con las observaciones reales que la política recibe y no con un tensor sintético, sincronizando la GPU antes y después de cada llamada. Diez tiradas por política, y se descarta la primera llamada de cada una, que es compilación de núcleos. El presupuesto contra el que se contrasta lo impone el lazo de control: un bloque de cuarenta acciones cada 0,8 segundos.

4.2. Análisis de resultados

4.2.1. Convergencia del entrenamiento

Ambos entrenamientos convergen, y las dos curvas de la figura 4.1 cuentan cosas distintas. La de GR00T cae de 1,68 a 0,35 en los primeros cien pasos y a 0,10 en los primeros mil; los nueve mil restantes la llevan a 0,064. Es el perfil esperable de un ajuste fino: el modelo de partida ya sabía mover este robot —se parte del ajuste

Convergencia del entrenamiento



La pérdida es un error de regresión contra las acciones del operador: que baje significa que la red copia bien las demostraciones, no que el robot abra la válvula. La evidencia de aprendizaje es la tasa de éxito (figura 2).

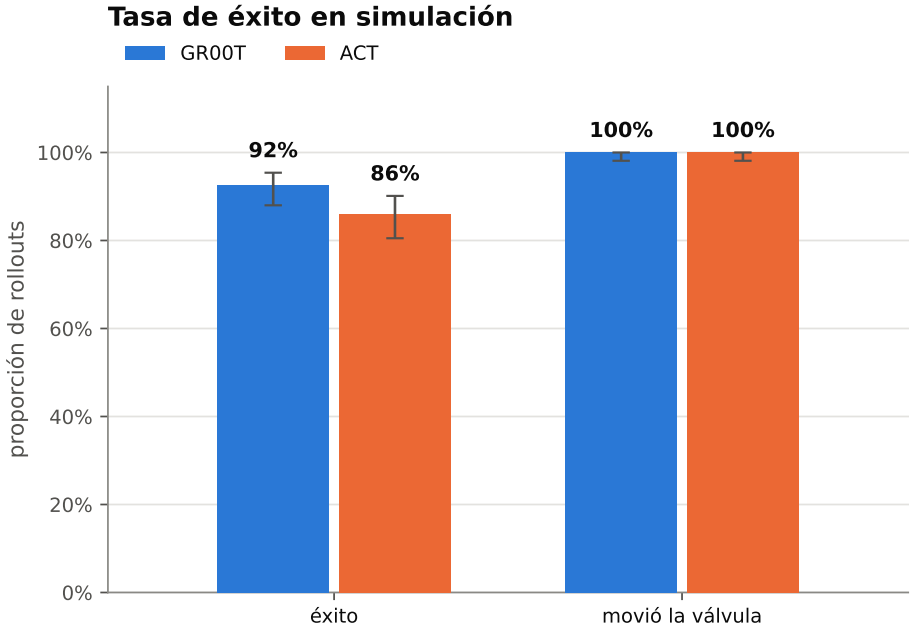
Figura 4.1: Convergencia del entrenamiento de ambos métodos, en paneles separados porque las dos pérdidas no son comparables entre sí

de NVIDIA sobre el mismo *embodiment* y la misma cámara— y lo que aprende en los primeros cientos de pasos es la tarea, no el cuerpo. La de ACT parte de 7,2, baja a escalones durante los primeros cinco mil pasos y después se aplana: la segunda mitad del entrenamiento la mueve de unos 0,20 a 0,169. Veinte mil pasos son aproximadamente 31 épocas sobre las cien demostraciones, y la mitad final apenas cambia la pérdida; se conservó el número de pasos del ensayo previo para no introducir una variable.

Conviene subrayar, no obstante, lo que ninguna de las dos curvas demuestra. La pérdida en aprendizaje por imitación es un error de regresión contra las acciones del operador: que baje significa que la red copia bien las demostraciones, no que el robot abra la válvula. Una política puede exhibir una pérdida excelente y una tasa de éxito nula si aprende la media de las demostraciones, y las dos pérdidas —*flow matching* en un caso, L1 sobre acciones en el otro— no son comparables entre sí, por lo que van en paneles separados. La evidencia de aprendizaje es la tasa de éxito en simulación; las curvas solo acreditan que la optimización convergió y que, como se verificó comparando los pesos con los del modelo de partida, algo cambió de verdad.

Tabla 4.2: Resultado sobre doscientas tiradas por política

Política	Tiradas	Éxito	IC 95 %	Bloque 1 / 2	Movió la válvula	Ángulo medio
GR00T N1.7	200	92 % (185/200)	88–95 %	91 / 94	100 %	177°
ACT	200	86 % (172/200)	81–90 %	83 / 89	100 %	173°



GR00T: $n=200$ · ACT: $n=200$. Barras de error: intervalo de Wilson al 95 %. «Movió la válvula» es la métrica de progreso parcial (revolute_joint_moved_rate): distingue no llegar a tocarla de girarla sin completar media vuelta.

Figura 4.2: Tasa de éxito de ambas políticas con intervalos de confianza de Wilson al 95 %, y tasa de movimiento de la válvula

4.2.2. Comparación principal

La tabla 4.2 y la figura 4.2 recogen el resultado agregado. GR00T resuelve el 92 % de las tiradas y ACT el 86 %. Dos lecturas inmediatas. La primera es que **el orden se mantiene en los dos bloques** —91 frente a 83 con la semilla 42, 94 frente a 89 con la semilla 7— y que los dos bloques son coherentes entre sí: el segundo salió algo más fácil para ambas políticas, como cabe esperar de otro sorteo, y la diferencia entre políticas es de ocho y cinco puntos respectivamente. La segunda es que **los intervalos de confianza se solapan**: 88–95 % frente a 81–90 %. Mirar solo las barras de la figura no permite afirmar que la diferencia sea real, y esa es precisamente la razón de que el diseño sea emparejado.



Figura 4.3: Primera tirada de evaluación de GR00T con la semilla 42 —válvula en disposición cenital, resuelta en 8,5 segundos—, en vista de tercera persona y con el fondo de oficina, a los 0,5, 1,75, 4 y 6,75 segundos: reposo, primer agarre de un radio, tirón, y nuevo agarre tras soltar. El giro desde arriba se hace a golpes, no de una pasada. La política solo ve la cámara de la cabeza del robot

Tabla 4.3: Tabla de contingencia emparejada sobre las doscientas tiradas

	ACT acierta	ACT falla
GR00T acierta	160	25
GR00T falla	12	3

El contraste emparejado

De las doscientas tiradas, ciento sesenta las resuelven ambas políticas, veinticinco solo GR00T, doce solo ACT y tres ninguna (tabla 4.3). Solo los treinta y siete casos discordantes llevan información, y la prueba exacta de McNemar sobre ellos arroja $p = 0,047$.

Ese valor cruza el umbral del 5 % por un solo episodio, y es frágil. Basta cambiar un rollout de sitio para verlo: un reparto de veinticuatro contra trece daría $p = 0,099$, y de veinticinco contra trece, $p = 0,073$. Con los cien primeros rollouts la misma comparación daba $p = 0,134$, y doblar la muestra la movió de «no se puede afirmar nada» a «está en el límite», que es exactamente el comportamiento que se espera de una diferencia real pero pequeña. La afirmación que los datos sostienen es que *GR00T va consistentemente por delante en los dos bloques y la diferencia se aproxima a la significación estadística*; afirmar que GR00T es significativamente mejor que ACT sería sobreinterpretar el resultado, y la frase se caería en cuanto alguien preguntase qué pasa con otra semilla.

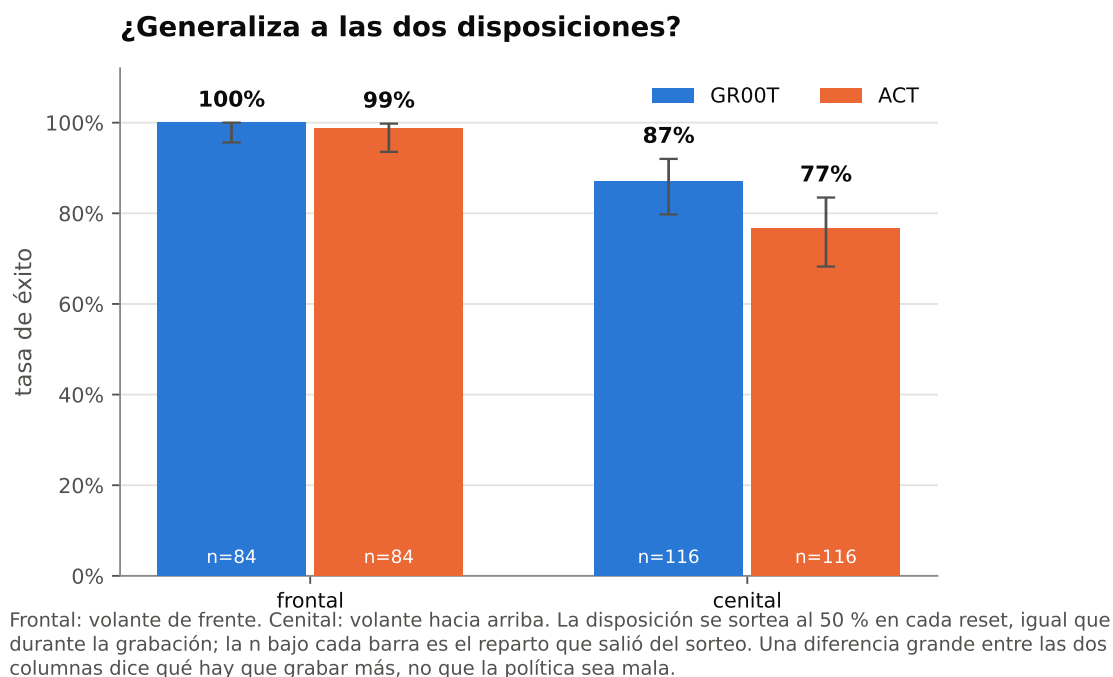


Figura 4.4: Tasa de éxito de cada política separada por disposición de la válvula. El reparto entre disposiciones es el que salió del sorteo

Dónde vive la diferencia

Lo que sí resulta significativo, y con mucho margen, es el efecto de la disposición (figura 4.4). Con el volante de frente las dos políticas están saturadas: GR00T resuelve 84 de 84 y ACT 83 de 84. Con el volante hacia arriba, GR00T resuelve 101 de 116 (87 %) y ACT 89 de 116 (77 %). La prueba exacta de Fisher da $p = 0,00021$ para GR00T y $p = 0,0000018$ para ACT: **ambas políticas son claramente peores en la disposición cenital**, y eso aguanta cualquier contraste. No es un artefacto de una política: es que la tarea es más difícil por ese lado, y es coherente con todo lo demás que se midió de esa disposición —fue la que hubo que acercar dos veces en las gafas antes de resultar cómoda de operar, y la que rompía la reproducción en lazo abierto, porque agarrar un radio desde arriba depende del punto exacto de contacto mucho más que empujar de frente.

Y de ahí sale toda la diferencia entre políticas. **Treinta y seis de los treinta y siete casos discordantes son cenitales.** En la disposición frontal hay un único caso discordante —una tirada que GR00T resuelve y ACT no— y ninguno en sentido contrario, de modo que no hay margen donde separarse. Restringiendo el contraste emparejado a la disposición cenital, el reparto es veinticuatro contra doce y $p = 0,065$; en la frontal, uno contra cero y $p = 1$. La lectura correcta no es «GR00T es

mejor en general», sino «*GR00T aguanta algo mejor la disposición difícil*» —y ni siquiera eso alcanza la significación por sí solo al restringir la muestra.

Los tres episodios que fallan las dos políticas son también cenitales, y no son uniformemente difíciles: en uno las dos se quedan cerca de los 90°, pero en otro GR00T llega a 133° y ACT a 176°, y en el tercero GR00T a 149° y ACT a 74°. Solo una condición inicial de doscientas derrota a ambas políticas en el mismo punto; las demás derrotas son propias de cada una.

Cuánto tardan cuando aciertan

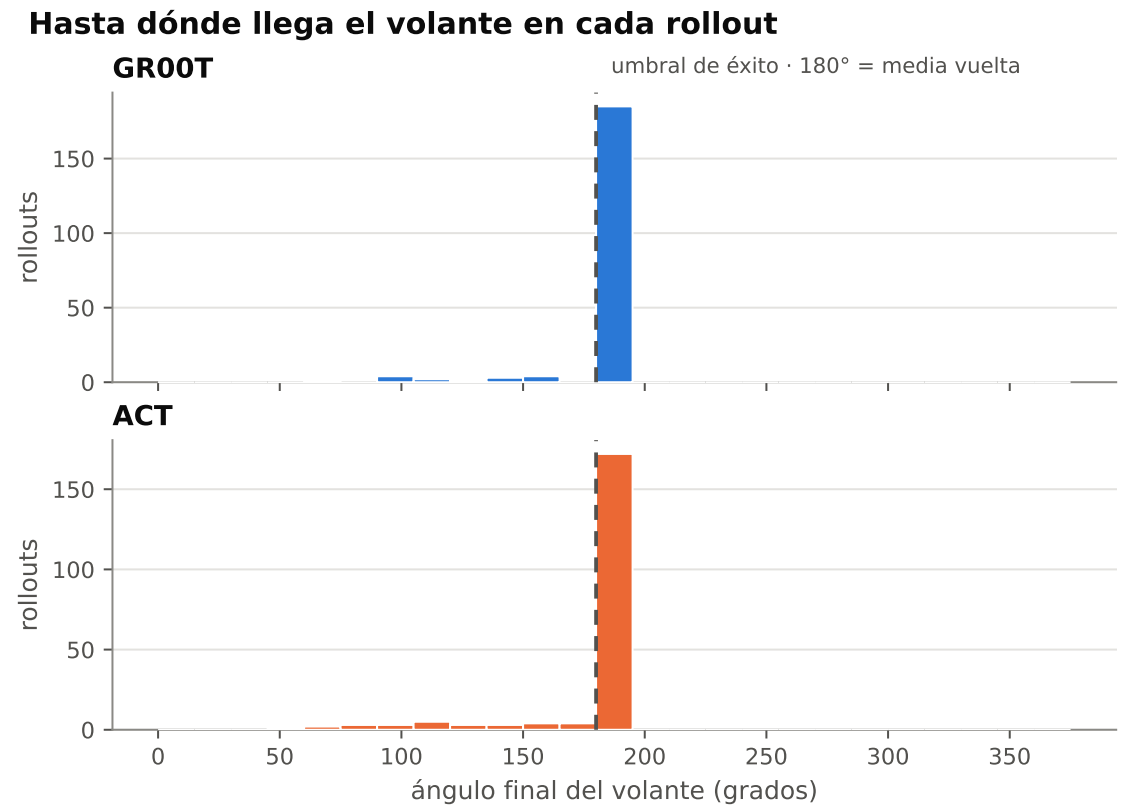
Tabla 4.4: Pasos hasta cruzar el umbral de éxito, en los episodios con éxito

Política	Frontal (mediana)	Cenital (mediana)	Global (mediana)	Percentil 90
GR00T N1.7	312 pasos (6,2 s)	418 pasos (8,4 s)	356 pasos (7,1 s)	597 pasos
ACT	311 pasos (6,2 s)	421 pasos (8,4 s)	366 pasos (7,3 s)	688 pasos

El paso en el que el volante cruza los 180° añade una dimensión que la tasa de éxito no tiene (tabla 4.4). Dos observaciones. La primera es que **las dos políticas son igual de rápidas**: 312 frente a 311 pasos de mediana en la frontal, 418 frente a 421 en la cenital. La diferencia entre métodos no es de velocidad de ejecución; ambas copian el ritmo del operador. La segunda es que **la disposición cenital cuesta 2,1 segundos más** a las dos, con exactitud casi idéntica. La dificultad de esa disposición se manifiesta también en el tiempo, no solo en la tasa: alcanzar un radio desde arriba y girar lleva más pasos de aproximación aunque salga bien. Donde las políticas se separan es en la cola: el percentil 90 de ACT está 91 pasos por encima del de GR00T, es decir, ACT tiene más éxitos lentos. Con un presupuesto de 1000 pasos y una mediana de 356–366, el tiempo no es lo que limita a ninguna de las dos.

4.2.3. Distribución de los fallos

La tasa de movimiento de la articulación es del 100 % para ambas políticas en las cuatrocientas tiradas, de modo que no discrimina en esta comparación. Pero es en sí misma un resultado: no hay ni un solo episodio en el que el robot no llegue a la válvula y la gire, y el giro mínimo observado en cuatrocientas tiradas es de 59°. Todos los fallos son de *no completar media vuelta*, nunca de *no encontrarla*. Las dos políticas han aprendido a aproximarse, a agarrar y a girar; lo que a veces les falta es terminar.



Un fallo a 170° y un fallo a 10° cuentan igual en la tasa de éxito y no son lo mismo: el primero es una política que casi resuelve la tarea, el segundo una que no la ha aprendido.

Figura 4.5: Distribución del ángulo máximo alcanzado por el volante en cada tirada, con el umbral de éxito marcado. La barra de los éxitos está pegada al umbral porque el episodio termina en el paso en que lo cruza

La figura 4.5 hay que leerla con una cautela que no es evidente. Los éxitos se acumulan todos entre 180,0° y 184,0° —en las dos políticas, sin excepción— y eso **no dice nada de hasta dónde giraría la política**: el episodio termina en el mismo paso en que se cruza el umbral —se ha comprobado en los 357 éxitos—, y lo que el volante avanza por encima de 180° son las décimas de segundo que tarda en registrarse el cruce. Durante la grabación, en cambio, el éxito debía sostenerse diez pasos, y por eso las demostraciones acaban algo más allá. La información de la figura está en la cola de fallos, y ahí las dos distribuciones sí difieren (tabla 4.5).

Tabla 4.5: Caracterización de los episodios fallidos

Política	Fallos	Ángulo máximo	Mediana	$\geq 170^\circ$	$< 100^\circ$	Retroceden
GR00T N1.7	15 (todos cenitales)	73–165°	133°	0	3	13 de 15
ACT	28 (27 cenitales)	59–178°	121°	4	8	24 de 28

Los quince fallos de GR00T se concentran entre 73° y 165° , con mediana en 133° ; ninguno llega a 170° y solo tres se quedan por debajo de 100° . Los veintiocho de ACT se reparten más: van de 59° a 178° , con mediana en 121° , cuatro casi-éxitos por encima de 170° y ocho por debajo de 100° . ACT falla más veces y falla de forma más dispersa, por los dos extremos: tiene más tiradas que se quedan a un paso y más tiradas que se quedan lejos. Los cuatro casi-éxitos de ACT ilustran el coste de una métrica binaria —una política que gira 178° puntúa igual que una que no ha aprendido la tarea— y son la razón de dar el ángulo junto con la tasa.

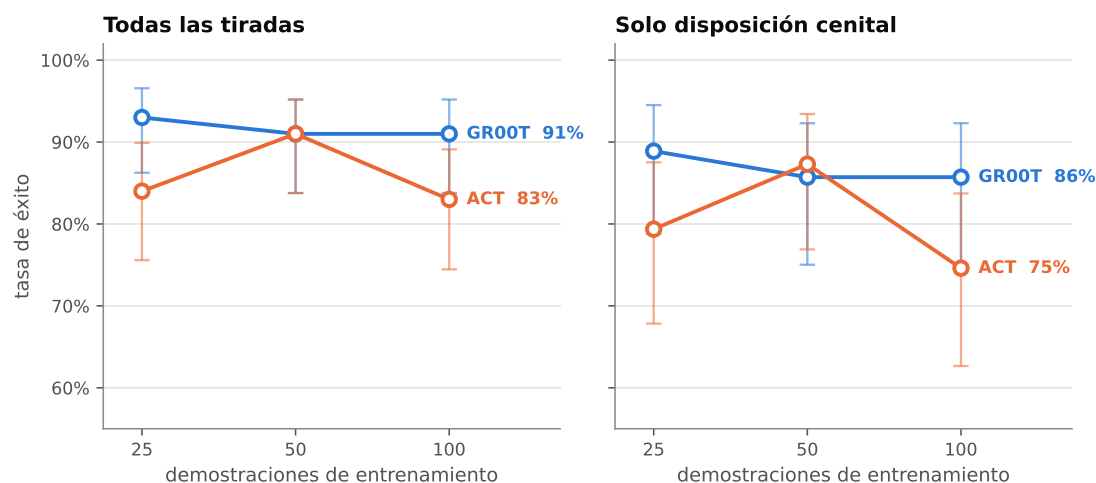
La trayectoria del ángulo dentro de cada fallo dice además **cómo** se falla, y no es como cabría suponer. Ningún fallo agota el tiempo con el volante parado en un punto: el ángulo máximo se alcanza tarde —en el paso 774 de mediana para GR00T y 851 para ACT, de 1000— y en dos de cada tres fallos el ángulo aún subía en los últimos cien pasos. Sobre todo, en trece de los quince fallos de GR00T y en veinticuatro de los veintiocho de ACT **el volante retrocede** más de cinco grados desde su máximo, hasta 99° en el peor caso. La imagen que dibujan esas trayectorias es la de una política que gira, pierde el radio, ve cómo el volante se vuelve atrás —la articulación no tiene fricción seca que lo retenga— y vuelve a intentarlo, sin llegar a acumular media vuelta antes de que se acabe el tiempo. Es un fallo de agarre sostenido, no de localización ni de intención, y es coherente con que ocurra casi exclusivamente en la disposición cenital, donde el agarre desde arriba depende del punto exacto de contacto con el radio. Esa lectura es una interpretación de las trayectorias, no una medida directa del contacto; pero es la única compatible con retrocesos de decenas de grados en episodios que siguen intentándolo.

4.2.4. Eficiencia en datos

Tabla 4.6: Tasa de éxito frente al número de demostraciones, cien tiradas por punto

Política	25 demostraciones	50 demostraciones	100 demostraciones
GR00T N1.7	93 % (86–97)	91 % (84–95)	91 % (84–95)
ACT	84 % (76–90)	91 % (84–95)	83 % (75–89)
Solo cenital, GR00T	89 %	86 %	86 %
Solo cenital, ACT	79 %	88 %	75 %

La hipótesis que justifica el empleo de un modelo fundacional es que GR00T se despegara con pocas demostraciones y ACT necesitara más para alcanzarlo. **No se observa** (figura 4.6, tabla 4.6). GR00T está en el 93 % con veinticinco demostraciones y no mejora con cien; ACT va de 84 a 91 y vuelve a 83. Ninguna diferencia dentro

Eficiencia en datos: 25, 50 y 100 demostraciones

100 tiradas por punto, semilla 42: los seis entrenamientos se evalúan sobre las mismas condiciones iniciales. Barras: intervalo de Wilson al 95 %. Los subconjuntos de 25 y 50 son estratificados (12F/13C y 25F/25C), así que entre puntos solo cambia la cantidad de datos. Cada punto es un único entrenamiento: la variación mezcla el efecto de los datos con la varianza de semilla, y ninguna diferencia dentro de una misma política es significativa (McNemar emparejado, $p \geq 0,11$ en los dos paneles): con esta tarea, 25 demostraciones ya saturan lo que el tamaño del dataset puede dar.

Figura 4.6: Tasa de éxito frente al número de demostraciones de entrenamiento, en el total de las tiradas y restringida a la disposición cenital. Cien tiradas por punto sobre las mismas condiciones iniciales; cada punto es un único entrenamiento

de una misma política es significativa: para GR00T, 25 frente a 100 demostraciones da siete casos discordantes contra cinco y $p = 0,77$; para ACT, 25 frente a 100 da trece contra doce y $p = 1$, y 50 frente a 100, quince contra siete y $p = 0,13$. Entre políticas, en cada punto: $p = 0,078$ con veinticinco demostraciones, empate exacto con cincuenta —91 % las dos, seis discordantes en cada sentido— y $p = 0,134$ con cien.

Tres lecturas, en orden de solidez. Primera, y es la más firme: **con esta tarea, veinticinco demostraciones ya saturan lo que el tamaño del conjunto de datos puede dar**. Cuadruplicarlas no mueve la tasa de éxito de ninguna de las dos políticas. Es un resultado negativo, y es el más útil del experimento: dice que el 8–15 % de fallo que queda **no es un problema de cantidad de datos**, de modo que grabar hacia cuatrocientas demostraciones con el mismo reparto —que era el plan inicial— no lo habría arreglado, y explica por qué el conjunto definitivo se cerró en cien. Segunda: lo que importa es la **composición**. El panel derecho de la figura lo enseña: en la disposición frontal GR00T resuelve 37 de 37 en los tres puntos y ACT entre 34 y 36 de 37, saturado desde veinticinco demostraciones; toda la variación vive en la cenital. Es el segundo argumento independiente, tras el de la

sección anterior, para reequilibrar cualquier captura adicional hacia la disposición difícil. Tercera, y es una tendencia, no un hallazgo: GR00T aguanta mejor con pocos datos. 93 % contra 84 % con veinticinco demostraciones es la mayor separación de los tres puntos y va en la dirección que la hipótesis predice, pero $p = 0,078$ y con cincuenta el orden se invierte hasta el empate.

Debe leerse con una cautela que se hace explícita en el pie de la figura: cada punto corresponde a **un único entrenamiento**, de modo que la variación entre puntos mezcla el efecto de los datos con la varianza de semilla del propio entrenamiento. El sube-y-baja de ACT —84, 91, 83— es casi con seguridad ruido de entrenamiento y no una relación real con el tamaño del conjunto; separar ambas cosas exigiría varias semillas por punto, que no cabían en el calendario. La frase defendible es «la curva es plana», nunca «ACT empeora con más datos».

4.2.5. Latencia de inferencia

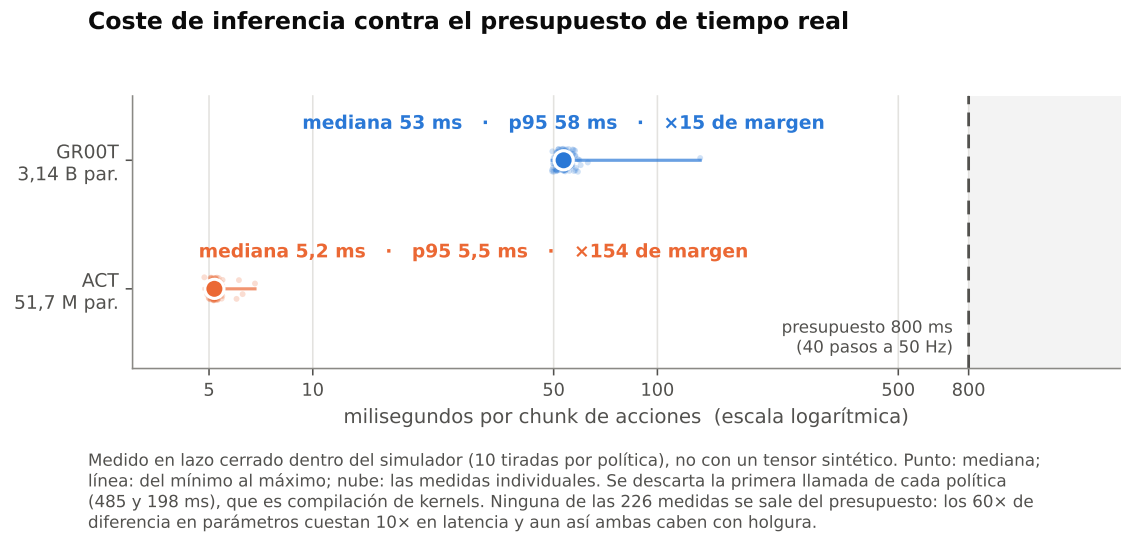


Figura 4.7: Coste de inferencia de cada política por bloque de acciones, en escala logarítmica, frente al presupuesto que impone el lazo de control

Tabla 4.7: Latencia de inferencia por bloque de cuarenta acciones, medida en lazo cerrado

Política	Parámetros	Mediana	Percentil 95	Máximo	Primera llamada	Margen
GR00T N1.7	3140 M	53 ms	58 ms	133 ms	485 ms	×15
ACT	51,7 M	5,2 ms	5,5 ms	6,8 ms	198 ms	×154

Es el otro eje donde 3140 millones de parámetros contra 51,7 millones tiene que notarse, y se nota (figura 4.7, tabla 4.7): la mediana de GR00T es diez veces la de ACT. Pero el presupuesto son 800 ms —un bloque de cuarenta acciones a 50 Hz—, y **ninguna de las 226 medidas se sale**, ni el máximo de GR00T. Los sesenta y un veces de diferencia en parámetros cuestan diez veces en latencia, y aun así las dos políticas caben con holgura: GR00T con quince veces de margen sobre su mediana y seis sobre su peor caso, ACT con más de ciento cincuenta.

Dos matices sobre la figura. ACT es, además de rápida, **estable**: entre la mediana y el máximo hay 1,6 ms. GR00T tiene una **cola**: su máximo es dos veces y media la mediana, y la nube de medidas muestra unas pocas llamadas dispersas hacia los cien milisegundos. Con este presupuesto la cola es irrelevante, pero es el tipo de comportamiento que habría que caracterizar antes de acortar el bloque de acciones. Y la primera llamada de cada política —485 y 198 ms— se descarta porque es compilación de núcleos, pero en un despliegue real existe: es el tiempo que la política tarda en estar lista tras arrancar.

La conclusión práctica es que **la latencia no es un argumento contra el modelo fundacional en esta tarea**. Dicho lo cual, conviene situar la medida: el servidor corre solo en una GPU de 96 GB y el enlace con el simulador es local. En un robot real, con más carga sobre la misma GPU y una red por medio, los 53 ms son el suelo, no el techo.

4.2.6. Amenazas a la validez

Los resultados anteriores sostienen una comparación limpia dentro de sus límites, y conviene enumerar esos límites con la misma precisión que las cifras.

Una sola tarea, una sola configuración. Todo lo medido describe la apertura de una válvula de volante en dos disposiciones, con un G1 cuyo pelvis está fijado durante la manipulación. Las tasas no se transfieren a otras tareas; lo que se transfiere es el banco. Y la base fija es una concesión que en un despliegue no existiría: el par de reacción del volante lo absorbería la política de equilibrio, con consecuencias que este trabajo no mide.

Una sola semilla de entrenamiento por configuración. Cada punto de la curva de eficiencia y cada una de las dos políticas de la comparación principal es un único entrenamiento. La varianza entre semillas de entrenamiento no está medida, y es la explicación más probable de las oscilaciones de la curva. Afecta menos a

la comparación principal —que se apoya en doscientas tiradas emparejadas de un mismo par de políticas— que a cualquier afirmación sobre la forma de la curva.

La comparación no es simétrica en su punto de partida, por diseño. GR00T arranca de un modelo preentrenado y, además, del ajuste de NVIDIA sobre el mismo *embodiment* y la misma cámara; ACT arranca de pesos aleatorios. Eso es exactamente lo que se quiere comparar —un modelo fundacional frente a una arquitectura entrenada desde cero—, pero significa que la ventaja de GR00T en la disposición difícil puede proceder tanto de su arquitectura como de haber visto ya a este robot manipular en simulación. El experimento no separa las dos cosas.

Las demostraciones terminan al cruzar el umbral. El grabador cierra cada demostración diez pasos después del éxito, de modo que el operador nunca sigue girando y las demostraciones acaban entre 180° y 232° . La política aprende de trayectorias que se detienen justo después de media vuelta, y parte de los fallos —los cuatro de ACT por encima de 170° en particular— son casi-éxitos por construcción del protocolo de captura, no por incapacidad. Un margen mayor al grabar habría dado probablemente tasas más altas a las dos políticas.

Evaluación en distribución. Las políticas se evalúan en el mismo simulador, con el mismo fondo y el mismo sorteo de condiciones iniciales con que se generaron sus datos de entrenamiento. No se ha medido ninguna forma de generalización fuera de esa distribución —otra válvula, otro fondo, otra iluminación—, que es donde la literatura sitúa la ventaja de los modelos fundacionales. El resultado de la curva de eficiencia debe leerse con esa acotación: dice que los datos no son el límite *en distribución*.

Sin validación sobre el robot físico. La distancia entre el simulador y el robot no se mide. La dinámica de contacto del agarre —que es, según el análisis de los fallos, lo que decide los episodios cenitales— es la parte del simulador de la que menos garantías hay, y la articulación de la válvula no tiene fricción seca, lo que hace que el volante retroceda al soltarlo de una forma que una válvula real no haría.

Un único operador y un espacio de acción con dimensiones muertas. Las cien demostraciones las grabó la misma persona, y seis de las veintitrés dimensiones de la acción son constantes. Lo primero da un conjunto consistente a costa de un sesgo de estilo; lo segundo se documenta en el capítulo 3 y no se corrigió para no abrir un experimento cerrado.

Ninguna de estas amenazas invalida la afirmación central del capítulo, que es también la más modesta: sobre esta tarea y con estos datos, el modelo fundacional

va por delante de la política clásica por un margen pequeño y difícil de acreditar, y la variable que domina el resultado es la disposición de la válvula, no el método.

Conclusión

Este trabajo se planteó dos objetivos encadenados: construir y validar un *pipeline* completo de aprendizaje por imitación sobre un robot humanoide para una tarea industrial concreta, y emplearlo para responder, con evidencia y en igualdad de condiciones, si un modelo fundacional visión-lenguaje-acción supera a una política de imitación clásica entrenada desde cero. Los dos se han cumplido, y la respuesta al segundo cabe en una frase: sobre esta tarea, el modelo fundacional va por delante de la política clásica por un margen pequeño y difícil de acreditar estadísticamente —92 % frente a 86 % sobre doscientas tiradas emparejadas, con $p = 0,047$ y un solo episodio de margen—, mientras que la variable que de verdad domina el resultado, con dos contrastes por debajo de $p = 0,001$, es la disposición de la válvula, no el método. Es un resultado menos vistoso que «el modelo grande gana», y bastante más útil para entender que la complejidad de la tarea influirá en los resultados y dependerá estrechamente en la calidad de las demostraciones grabadas.

Los dos experimentos secundarios cierran la lectura. La curva de eficiencia en datos es plana: veinticinco demostraciones saturan lo que la cantidad de datos puede dar, para las dos políticas, de modo que el 8–15 % de fallo restante no se arregla grabando más de lo mismo. Y la latencia, medida en lazo cerrado, no descarta a ninguna: sesenta y un veces más parámetros cuestan diez veces más tiempo de inferencia, y aun así las dos caben con holgura en el presupuesto del lazo de control (ejecutado eso sí en una GPU externa, no en la que lleva integrada el robot). Si hubiera que resumir el dictamen para el cliente del supuesto del capítulo 2: el modelo fundacional es una opción válida y algo más robusta en la disposición difícil, pero la inversión que mueve el resultado no está en el método, sino en qué demostraciones se graban.

Aportaciones realizadas

- **Una tarea de manipulación industrial instrumentada y reproducible** sobre el Unitree G1: válvula procedente de un modelo CAD real, dos disposiciones sorteadas con perturbación de posición, criterio de éxito automático, y todas las poses medidas —el reinicio es determinista a un milímetro—. Está versionada como un fichero de entorno, una capa USD y un conjunto de parches sobre el marco de terceros, de modo que se puede reconstruir sin depender de una máquina concreta.
- **Un *pipeline* completo verificado etapa a etapa**, de la teleoperación en realidad mixta a la política evaluada en lazo cerrado, con una prueba ejecutable por etapa y documentación de lanzamiento y de entrenamiento con las recetas exactas y sus fallos conocidos. Incluye la reproducción de demostraciones con el estado grabado impuesto en cada paso, que es lo que permite cambiar el fondo de un conjunto de datos ya capturado sin alterar lo que enseña.
- **Un conjunto de cien demostraciones teleoperadas**, con las dos disposiciones al 50 %, agarre del volante y fondo fotorrealista de una oficina real, en un formato que sirve a ambas familias de métodos sin transformación adicional.
- **Un banco de comparación con interfaz común**: un cliente de evaluación y tres servidores intercambiables —la reproducción de acciones, GR00T y ACT— de modo que el comando de evaluación es idéntico para todos y cualquier diferencia es atribuible a la política.
- **Una comparación emparejada con el contraste apropiado**, y un análisis que va más allá de la tasa de éxito: tiempo hasta el umbral, distribución y dinámica de los fallos —que resultan ser retrocesos del volante tras perder el agarre, no estancamientos—, curva de eficiencia estratificada y latencia en lazo cerrado.
- **Un hallazgo negativo y accionable**: en esta tarea la limitación dominante no es la cantidad de demostraciones sino su composición, y toda la diferencia entre métodos vive en la disposición más difícil.
- **La documentación de los parches y errores** que costaron sesiones enteras y no produjeron ningún error —la sincronización del sensor, el estado de las manos, los búferes de normalización, la reproducción en lazo abierto—, documentada para evitar repetir los mismos errores en futuras investigaciones.

Trabajos futuros

Las líneas de continuación salen directamente de los límites que el capítulo 4 declara, y están ordenadas por lo que cada una aportaría.

1. **Reequilibrar la captura hacia la disposición cenital**, en torno a un 40/60, y dar margen sobre el umbral de éxito al grabar —seguir girando unos grados más, o exigir más pasos sostenidos—. Son las dos intervenciones que atacan lo único que, según los datos, limita el resultado, y ninguna requiere cambiar el sistema.
2. **Varias semillas de entrenamiento por configuración**. Es lo que separa el efecto de los datos de la varianza del entrenamiento en la curva de eficiencia, y lo que permitiría afirmar o descartar la tendencia de GR00T con pocas demostraciones.
3. **Fricción seca en la articulación de la válvula**. El análisis de los fallos muestra que el volante retrocede al perder el agarre; una válvula real no lo haría. Añadir un par de rozamiento constante haría la tarea más fiel y, previsiblemente, cambiaría la distribución de los fallos. Este cambio haría que los modelos tuvieran que aprender a prevenir el deslizamiento y no solo a reaccionar ante él.
4. **Evaluar fuera de distribución**: otra válvula, otro fondo, otra iluminación, otra instrucción. Es donde la literatura sitúa la ventaja de los modelos fundacionales y donde este trabajo, por diseño, no ha mirado. Es decir, randomizar, no solo la física para obtener resultados más robustos, si no también el entorno e input visual.
5. **Liberar la base**. Retirar la fijación del pelvis y dejar que la política de equilibrio absorba el par de reacción, que es la condición de un despliegue real. Hacer esto requerirá re-entrenar la política, pero no es algo que exija un cambio de método. La nueva política deberá ser capaz de mantener el robot fijo frente a perturbaciones de posición y de orientación provocadas por empujar objetos con los brazos, algo que la política actual no puede hacer.
6. **Un tercer brazo con Diffusion Policy**. No porque ACT se haya quedado corta —el capítulo 4 muestra que resuelve la tarea a seis puntos del modelo fundacional—, sino por completar el cuadro con la otra arquitectura de referencia entrenada desde cero. Su formulación generativa está pensada para demostraciones con varias soluciones válidas, y el agarre de un radio desde

arriba, donde se concentran todos los fallos, es exactamente ese caso: hay varios radios y varias formas de cogerlos. Sería una comprobación de si la manera de modelar la acción cambia algo en la disposición difícil, más que la búsqueda de una política mejor. Junto a ello, la eliminación de las seis dimensiones muertas del espacio de acción, que se dejó fuera del experimento para no abrir una variable nueva. El banco empleado admite ambas cosas sin cambios: LeRobot trae la implementación de referencia, y el servidor de ACT es la plantilla del que haría falta.

7. **El robot físico.** Todo lo anterior se hace en simulación, y la validación sobre el G1 real es la línea que da sentido a las demás y la única que este trabajo no ha podido abordar. Hay dos caminos, y conviene recorrerlos en este orden. El primero, y el más directo, es **grabar las demostraciones en el propio robot**: el mismo sistema de teleoperación, con las gafas y los mandos, pero con las poses de las muñecas yendo al G1 físico y la cámara real devolviendo la imagen. Los datos serían de la distribución de despliegue y no habría brecha que cerrar; el *pipeline* de conversión, entrenamiento y servicio de políticas es el mismo. Su límite es evidente: cada demostración consume tiempo de robot y de operador, cada fallo es un riesgo físico, y la disponibilidad de la máquina es la que es. Cien demostraciones que en simulación son una tarde son, sobre el robot, semanas.

Por eso el segundo camino es el que de verdad escala, y es la razón de fondo de varias decisiones de este trabajo: **seguir perfeccionando el entorno de simulación que ya existe hasta cerrar la brecha entre simulación y realidad**, y desplegar en el robot la política entrenada en simulación. El fondo fotorrealista por *Gaussian Splatting* es el primer paso en esa dirección —la política ya ha entrenado sobre imágenes de un espacio real, no sobre un plano gris—, y lo siguiente es medir qué queda de distancia y reducirla donde importe: la dinámica de contacto del agarre y el par de la válvula por el lado físico, y la iluminación, el ruido del sensor y la pose exacta de la cámara por el visual. La herramienta estándar para que la política tolere lo que el simulador no acierta a reproducir es la **aleatorización de dominio** (*domain randomization*): variar durante la captura y el entrenamiento la iluminación, las texturas y la posición de la cámara, y también la fricción, el amortiguamiento y la masa del volante, de modo que la política aprenda a resolver la tarea bajo un rango de condiciones que contenga a la real en lugar de ajustarse a una sola. El entorno ya aleatoriza la disposición y la posición de la válvula;

extender esa misma maquinaria a los parámetros visuales y físicos es el trabajo natural de continuación, y el banco de evaluación de este trabajo serviría para medir, antes de tocar el robot, cuánto cuesta cada aleatorización en tasa de éxito. El objetivo final es que la primera vez que la política toque un volante de verdad no sea la primera vez que ve uno que no se comporta exactamente como el simulado.

Problemas encontrados

El reparto real del esfuerzo quedó muy escorado hacia la construcción del *pipeline* frente al entrenamiento de los modelos: los dos entrenamientos definitivos cabían en una tarde, y hacer posible que existieran costó semanas. Los obstáculos que lo explican comparten una propiedad: casi ninguno produjo un mensaje de error. El síntoma era siempre el mismo —una imagen en negro, una política que no hacía nada, un fichero que no se escribía— y la causa, otra distinta cada vez.

La pila de realidad mixta fue el primero y el más largo. Cuatro problemas independientes —el anclaje del visor a la altura del pelvis, un antialiasing forzado que sobreexponía la escena, un visualizador que no existe salvo que se pida y un cortafuegos que descarta paquetes en silencio— dieron todos la misma imagen inutilizable, y se resolvieron uno a uno con el monitor de tercera persona como única herramienta capaz de decir qué veía el simulador frente a lo que veía el operador.

La cámara sin fotograma costó un día entero. Grabar con imagen bajo realidad mixta se bloqueaba para siempre, y la evidencia apuntaba al contenido de la escena porque las escenas grandes funcionaban. Funcionaban por accidente: su tiempo de carga dejaba que se produjera un fotograma antes de que el gestor de observaciones pidiera el primero. Descartadas por medida seis hipótesis, la corrección fueron cuatro renders antes de construir los gestores. Con la misma combinación de realidad mixta y cámaras, la física en GPU fallaba además a los veintitrés segundos con un acceso ilegal a memoria, y la grabación pasó a la CPU.

Dos decisiones de diseño se tomaron al revés y hubo que darles la vuelta. La base del robot se dejó libre por realismo, y la primera sesión mostró que el par de reacción del volante giraba al robot entero; se fijó. Y la primera demostración se grabó empujando el volante con la mano abierta, porque el estado de las manos no es un bit sino una resta entre el gatillo y la empuñadura del mando, y apretar los dos a la vez equivalía a abrir; se fijó el gesto y se regrabó. Ninguna

de las dos habría aparecido sin auditar las dimensiones de la acción de la primera demostración.

Los límites de la articulación habrían impedido escribir ninguna demostración: el CAD autoriza seis vueltas y media, el marco normaliza sobre los límites, y media apertura habría exigido tres vueltas y cuarto en un episodio. Se descubrió leyendo el fichero antes de grabar, no después.

Los tres intérpretes y sus fallos sin mensaje. GR00T sobre dos GPU se queda parado antes de empezar; ACT emitía ceros exactos porque mover el modelo a la segunda GPU por su índice pone a cero los búferes de normalización; y el entrenador de GR00T reanuda en silencio desde cualquier punto de control que encuentre, de modo que es posible «entrenar» cero pasos y obtener un registro impecable. Los tres se resolvieron, y los dos últimos dejaron una comprobación automática para que no puedan repetirse.

La reproducción que no reproducía es el problema más serio, y el que más se parece a lo que un despliegue real haría: invalidó una sesión completa de datos sin ningún síntoma, porque las piernas las gobierna la política de equilibrio en lazo cerrado y no se reproducen desde el fichero. Se localizó paso a paso hasta las velocidades de los tobillos y se corrigió imponiendo el estado grabado; el anexo D lo documenta. Su lección es la que organiza todo el capítulo 3: verificar cada etapa con datos reales antes de apoyar la siguiente sobre ella, aunque retrase el arranque.

Y un problema pequeño con consecuencias grandes: un trabajo largo lanzado desde una terminal que después se cerró murió a la mitad, porque el proceso del contenedor muere con su cliente. Desde entonces todo lo largo corre desacoplado y deja un centinela por etapa, y fue eso lo que permitió ejecutar el *pipeline* completo, los seis entrenamientos y las seiscientas tiradas sin nadie delante.

Opiniones personales

Empecé este trabajo pensando que la parte difícil sería entrenar un modelo de tres mil millones de parámetros, y terminé convencido de que la parte difícil era todo lo demás. Los dos entrenamientos definitivos duraron cincuenta y tres y veinte minutos; que existieran datos sobre los que entrenarlos, y un banco que dijera la verdad sobre lo entrenado, es lo que ocupó el proyecto. Si tuviera que quedarme con una sola lección, sería esa: en aprendizaje por imitación sobre un robot real —o

sobre uno simulado con la fidelidad suficiente— el modelo es la pieza más madura del *pipeline*, y el esfuerzo se va a las que no lo son.

La segunda lección es sobre los fallos silenciosos. Casi todo lo que me costó tiempo no dio un error: dio un resultado plausible y falso. Una política que no hace nada es indistinguible de una mal entrenada; una reproducción que abre la válvula un 64 % de las veces parece un conjunto de datos regular y no un proceso roto. He aprendido a desconfiar de cualquier cifra que no venga acompañada de la prueba que la sostiene, y he intentado que la memoria refleje esa disciplina: no hay un número en ella que no pueda regenerarse desde los ficheros.

Sobre la pregunta del trabajo, mi percepción ha cambiado en las dos direcciones. GR00T me ha impresionado más de lo que esperaba por lo poco que necesita: veinticinco demostraciones y un ajuste fino de una hora para un 93 % de éxito en una tarea con dos disposiciones. Pero ACT me ha sorprendido más, porque con cincuenta millones de parámetros y desde cero se queda a seis puntos, a diez milisegundos de latencia y a doscientos megabytes de disco frente a doce gigabytes. En distribución, y con datos suficientes, la diferencia entre ambos es pequeña y frágil; la ventaja del modelo fundacional, si está donde la literatura dice, está fuera de la distribución, y ese es el experimento que haría a continuación. Mientras tanto, lo honesto es decir que el coste de ingeniería de desplegar un modelo fundacional —tres intérpretes, puntos de control de doce gigabytes, un ajuste fino que se cuelga en dos GPU— es una variable real de la decisión, y que para una tarea acotada y bien demostrada la alternativa clásica no es una opción de segunda.

Haría dos cosas de otra manera. Grabaría con más margen sobre el umbral de éxito desde la primera sesión, porque parte de los fallos que cuento son casi-éxitos por construcción del protocolo. Y pondría fricción seca en la válvula antes de grabar nada, porque un volante que retrocede al soltarlo no es un volante. Las dos cosas se ven claras ahora que los datos las señalan, y las dos eran relativamente sencillas de implementar.

Lo que me llevo, además de un sistema que funciona, es una forma de trabajar: medir antes de suponer, verificar cada etapa antes de apoyar la siguiente, y escribir la afirmación más débil que los datos sostienen en lugar de la más vistosa que permiten.

Lista de referencias

- [1] NVIDIA et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. Cincuenta y nueve autores en orden alfabético, con NVIDIA como primer autor, tal como indica el propio artículo. Este trabajo emplea la variante posterior N1.7.
- [2] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems XIX (RSS 2023)*, Daegu, Republic of Korea, 2023. arXiv:2304.13705.
- [3] Eric Krotkov, Douglas Hackett, Larry Jackel, Michael Perschbacher, James Phippine, Jesse Strauss, Gill Pratt, and Christopher Orlowski. The DARPA robotics challenge finals: Results and perspectives. *Journal of Field Robotics*, 34(2):229–240, 2017.
- [4] Unitree Robotics. Unitree G1 humanoid agent. <https://www.unitree.com/g1>, 2024.
- [5] Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In David S. Touretzky, editor, *Advances in Neural Information Processing Systems 1 (NIPS 1988)*, pages 305–313. Morgan Kaufmann, 1988.
- [6] Stéphane Ross, Geoffrey J. Gordon, and J. Andrew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635, Fort Lauderdale, FL, USA, 2011. PMLR.
- [7] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems XIX (RSS 2023)*, Daegu, Republic of Korea, 2023. arXiv:2303.04137. Versión extendida en

- The International Journal of Robotics Research*, 44(10–11):1684–1704, 2025, doi:10.1177/02783649241273668.
- [8] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Joseph Dabis, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, Jasmine Hsu, Julian Ibarz, Brian Ichter, Alex Irpan, Tomas Jackson, Sally Jesmonth, Nikhil J. Joshi, Ryan Julian, Dmitry Kalashnikov, Yuheng Kuang, Isabel Leal, Kuang-Huei Lee, Sergey Levine, Yao Lu, Utsav Malla, Deeksha Manjunath, Igor Mordatch, Ofir Nachum, Carolina Parada, Jodilyn Peralta, Emily Perez, Karl Pertsch, Jornell Quiambao, Kanishka Rao, Michael Ryoo, Grecia Salazar, Pannag Sanketi, Kevin Sayed, Jaspiar Singh, Sumedh Sontakke, Austin Stone, Clayton Tan, Huong Tran, Vincent Vanhoucke, Steve Vega, Quan Vuong, Fei Xia, Ted Xiao, Peng Xu, Sichun Xu, Tianhe Yu, and Brianna Zitkovich. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems XIX (RSS 2023)*, Daegu, Republic of Korea, 2023. arXiv:2212.06817, diciembre de 2022.
- [9] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of The 7th Conference on Robot Learning (CoRL 2023)*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR, 2023. arXiv:2307.15818.
- [10] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P. Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. OpenVLA: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning*

- (*CoRL 2024*), volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 2025. arXiv:2406.09246.
- [11] Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. In *IEEE International Conference on Robotics and Automation (ICRA 2024)*, pages 6892–6903, 2024. arXiv:2310.08864, octubre de 2023. Se cita como colaboración, con 389 autores, tal como el propio artículo indica.
- [12] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, James Tanner, Quan Vuong, Anna Walling, Haohuan Wang, and Ury Zhilinsky. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems XXI (RSS 2025)*, 2025. arXiv:2410.24164, octubre de 2024.
- [13] NVIDIA. NVIDIA Isaac Sim. <https://developer.nvidia.com/isaac/sim>, 2025. Este trabajo emplea la versión 6.0.0.
- [14] Mayank Mittal, Calvin Yu, Qinxu Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlkar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023. Base del actual Isaac Lab.
- [15] NVIDIA Isaac Lab-Arena Contributors. Isaac Lab-Arena: Composable environment creation and policy evaluation for robotics. <https://github.com/isaac-sim/IsaacLab-Arena>, 2025. Versión 0.2.1, la empleada en este trabajo.
- [16] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Khalil Meftah, Maxime Ellerbach, Jess Moss, and Thomas Wolf. LeRobot: State-of-the-art machine learning for real-world robotics in PyTorch. <https://github.com/huggingface/lerobot>, 2024. Este trabajo emplea la versión 0.3.3.
- [17] Xuxin Cheng, Jialong Li, Shiqi Yang, Ge Yang, and Xiaolong Wang. Open-TeleVision: Teleoperation with immersive active visual feedback. In *Procee-*

- dings of The 8th Conference on Robot Learning (CoRL 2024)*, volume 270 of *Proceedings of Machine Learning Research*, pages 2729–2749. PMLR, 2025. arXiv:2407.01512.
- [18] PICO Technology. PICO 4 Ultra. <https://www.picoxr.com/global/products/pico4-ultra>, 2024.
- [19] NVIDIA. NVIDIA CloudXR 6.0. <https://developer.nvidia.com/cloudxr-sdk>, 2025.
- [20] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):139:1–139:14, July 2023.
- [21] Francis Williams, Jiahui Huang, Jonathan Swartz, Gergely Klár, Vijay Thakkar, Matthew Cong, Xuanchi Ren, Ruilong Li, Clement Fuji-Tsang, Sanja Fidler, Eftychios Sifakis, and Ken Museth. fVDB: A deep-learning framework for sparse, large scale, and high performance spatial intelligence. *ACM Transactions on Graphics*, 43(4):1–15, July 2024. arXiv:2407.01781.
- [22] NVIDIA and OpenVDB. fVDB Reality Capture. <https://github.com/openvdb/fvdb-reality-capture>, 2025. Versión 0.5.0, la empleada en este trabajo.
- [23] Johannes L. Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4104–4113, 2016. Artículo de referencia de COLMAP; este trabajo emplea la versión 4.2 de desarrollo compilada con CUDA.

Anexo A

Control de versiones

El trabajo se versiona en dos repositorios con papeles distintos, y la relación entre ambos es la parte del control de versiones que más conviene explicar, porque es donde se perdió trabajo una vez y donde se decidió cómo no volver a perderlo.

A.1. Organización del repositorio

El **repositorio propio**, publicado como `TFM_ValveOperator`, es el entregable:

`https://github.com/izanvines/TFM\_ValveOperator`

Contiene todo lo que define el trabajo y nada que sea de terceros: la tarea de simulación, los guiones de la cadena de datos y de evaluación, la documentación de lanzamiento y de entrenamiento, los informes de resultados, las figuras y esta memoria. Su estructura es la siguiente.

Qué queda fuera, y por qué. No se versionan los conjuntos de datos —el HDF5 definitivo ocupa 17,5 GB y el conjunto LeRobot 100 MB—, los puntos de control de los modelos —12 GB el de GR00T, 198 MB el de ACT—, los ficheros de resultados de las tiradas ni el activo fotorrealista de la oficina (253 MB). El criterio no es solo el tamaño: todos ellos se regeneran desde los guiones del repositorio a partir de las demostraciones grabadas, y lo que se entrega es la capacidad de regenerarlos, no el resultado. Tampoco se versionan los marcos de terceros —Isaac Lab Arena, Isaac-GR00T y LeRobot—, que se obtienen de sus repositorios en las versiones que la tabla 3.3 fija. Las fuentes $\text{\LaTeX}$ de esta memoria tampoco están en el repositorio público, que lleva solo el PDF: se mantienen en un repositorio local aparte, con un guion que hace las tres pasadas de $\text{\LaTeX}$ más BibTeX, informa de referencias sin resolver

Tabla A.1: Organización del repositorio entregado

Ruta	Contenido
<code>sim/environments/</code>	El fichero de la tarea <code>g1_valve</code> : escena, poses medidas, robot, aleatorización y criterio de éxito
<code>sim/assets/</code>	La válvula CAD, la capa USD propia que la envuelve y su nota de procedencia
<code>sim/config/</code>	La configuración de conversión de HDF5 a LeRobot
<code>sim/patches/</code>	Los cambios sobre el marco de simulación, como parches contra la versión oficial (tabla A.2)
<code>sim/scripts/</code>	Veinte guiones de medida, diagnóstico, grabación y ejecución desatendida
<code>sim/sync.sh</code>	Sincronización entre este repositorio y el <i>checkout</i> vivo del marco
<code>train/scripts/</code>	Reproducción con fondo, verificación del conjunto de datos, servidores de política, subconjuntos, latencia y figuras
<code>reconstruction/</code>	Los guiones propios y la receta de la reconstrucción de la oficina (anexo D)
<code>docs/</code>	Informes de resultados, figuras en PDF y PNG, las notas de trabajo fechadas de julio y agosto, y esta memoria en PDF
<code>videos/</code>	Vídeos de cada política abriendo la válvula
<code>LAUNCH.md</code> , <code>TRAIN.md</code>	Las dos guías: simular y grabar; del HDF5 a la política evaluada
<code>CLAUDE.md</code>	El cuaderno de bitácora del proyecto: estado, decisiones con fecha y trampas conocidas

y cajas desbordadas, y copia el PDF al repositorio público, y con un comprobador que detecta lo que el compilador no dice: figuras que faltan, referencias a etiquetas inexistentes, citas a claves que no están en la bibliografía y marcadores pendientes de rellenar.

A.2. Los cambios sobre el marco de simulación

La simulación no se ejecuta desde el repositorio propio sino desde un *checkout* de Isaac Lab Arena en su versión 0.2.1, con el trabajo en una rama propia, `tfm/g1-valve`, que acumula diez confirmaciones sobre la base oficial entre el 18 y

el 21 de agosto. El submódulo de Isaac Lab que Arena incorpora apunta a su vez a una rama local del mismo nombre, porque dos de los cambios necesarios están en él y no en Arena.

Los cambios sobre ficheros que ya existen en el marco se conservan en el repositorio propio **como parches**, no como copias completas, para que quede explícito qué se ha tocado y para poder reaplicarlos sobre una versión posterior. La tabla A.2 los recoge.

Tabla A.2: Parches aplicados sobre el marco de simulación

Parche	Líneas	Qué cambia
<code>object_library</code>	61	La clase <code>Valve</code> : ruta del activo CAD, nombre de la articulación y umbral de apertura
<code>cli</code>	20	Registra <code>g1_valve</code> como entorno invocable
<code>background_library</code>	80	Registra el fondo fotorrealista <code>office_gs</code> y su pose (anexo D)
<code>g1_pink_locomanipulation_pipeline</code>	49	Lee <code>ARENA_STATIC_BASE</code> para congelar los canales de locomoción
<code>teleop</code>	32	Hace opcional el antialiasing forzado que sobreexponía la escena
<code>record_demos</code>	24	Lo mismo, para la ruta de grabación
<code>isaacclab_prerender</code>	84	El que desbloquea la grabación: renderiza antes de construir los gestores para que la cámara tenga fotograma
<code>isaacclab_submodule</code>	139	Los dos cambios del submódulo: el anterior y el anclaje XR creado con USD crudo cuando la primitiva falla
<code>lightwheel_reintento_5xx.py</code>	27	Reintentos de red en el cliente de activos, que vive solo dentro del contenedor y se pierde al reconstruirlo

Dos cosas se aprendieron aquí a un precio que conviene dejar escrito. La primera es que un `git checkout` o un `git submodule update` en el *checkout* vivo destruye estos cambios sin avisar; por eso existen como parches en el repositorio propio y por eso `sync.sh` imprime los comandos de aplicación en lugar de aplicarlos, ya que reaplicar un parche sobre un fichero que ya lo lleva lo corrompe en silencio. La segunda es que **los parches se generan contra la base oficial**, no contra el árbol de trabajo. Hasta el 20 de agosto se generaban con la diferencia respecto al

árbol, que solo ve cambios sin confirmar: en cuanto el trabajo empezó a confirmarse en la rama, siete de los ocho parches quedaron vacíos y la documentación seguía afirmando que ahí estaba el registro. Desde entonces se generan contra el ancestro común con la rama oficial, y `sync.sh` avisa si alguno sale vacío.

A.3. Historial de desarrollo

El repositorio propio tiene treinta y cuatro confirmaciones entre el 17 y el 25 de agosto de 2026, más las de la redacción de la memoria. El criterio para los mensajes es que cada confirmación sea un incremento verificable y que el mensaje diga qué cambia y qué se descubrió, en imperativo y en una línea, con el detalle en el cuerpo; el historial se lee como un cuaderno del proyecto. En orden, y agrupando: la documentación del punto de partida y de las decisiones de diseño (17 de agosto); la integración de la válvula, la fijación del pelvis, la grabación en CPU y la primera demostración con imagen (18); la conversión, la verificación y la prueba del lazo de evaluación con la reproducción de acciones, antes de entrenar nada (19); los dos entrenamientos del ensayo y su documentación (19–20); el fondo de oficina, la aleatorización de la válvula y los parches regenerados contra la base oficial (20); la reproducción con el estado grabado impuesto y el acercamiento de la válvula cenital (21); la grabación del conjunto definitivo y la comparación sobre cien demostraciones (22); el segundo bloque de tiradas y el contraste emparejado (24); la curva de eficiencia y la latencia (25); y, desde el 25, la memoria.

El `CLAUDE.md` de la raíz cumple una función distinta de la del historial: es el estado vivo del proyecto, con las decisiones fechadas y sus reversiones, las trampas conocidas y lo que queda abierto. Donde el historial dice qué se hizo, ese fichero dice qué se sabe.

Anexo B

Seguimiento de proyecto fin de máster

Este anexo recoge el seguimiento del trabajo real, por oposición a la simulación de gestión empresarial del capítulo 2: cómo se hizo el seguimiento, qué se planificó al principio y qué se ejecutó de verdad, con las desviaciones y su causa.

B.1. Forma de seguimiento

El seguimiento se apoyó en cuatro instrumentos, todos con fecha.

Notas de trabajo. Cada fase de investigación o de puesta en marcha se cerró con una nota en el directorio de trabajo, fechada y con su estado: el plan del trabajo en sus dos versiones (6 y 7 de julio), la investigación sobre control de cuerpo completo y teleoperación con las gafas (14 de julio), el acceso remoto y el visor de la simulación (13 y 27 de julio), la guía de teleoperación de la tarea (verificada el 23 de julio), la reconstrucción de la oficina (30 de julio y 3–4 de agosto) y el estado de la teleoperación con las gafas físicas (4 de agosto). Cada nota dice qué funciona, qué está parcheado y qué hipótesis se han descartado, para no volver a probarlas.

El cuaderno de bitácora del repositorio. Desde el 17 de agosto, un único fichero en la raíz del repositorio recoge el estado del proyecto, las decisiones con su fecha y su justificación, y las reversiones cuando las hubo. Es el documento que se actualiza al final de cada sesión de trabajo y el que se lee al principio de la siguiente.

El control de versiones. El historial de confirmaciones descrito en el anexo A, con un incremento verificable por confirmación.

Las sesiones de trabajo con el asistente de programación. El desarrollo se hizo con el asistente de inteligencia artificial que la declaración inicial de esta memoria describe, y cada sesión queda registrada con su fecha: dieciocho sesiones entre el 7 de agosto y el 4 de septiembre, repartidas entre el directorio de trabajo, el repositorio y la reconstrucción de la oficina. Son, en la práctica, el registro más fino de qué se intentó, qué falló y qué se decidió, y la fuente de la que se han reconstruido las fechas de este anexo.

B.2. Planificación inicial

El plan de partida es el del 6–7 de julio de 2026, que a su vez revisaba un borrador anterior. El borrador original planteaba un trabajo de nueve meses con una comparación a tres bandas entre aprendizaje por imitación, refuerzo y refuerzo residual sobre dos simuladores. La revisión de julio lo redujo a lo que cabía en **dos meses con acceso confirmado al G1 físico**:

- Grabar demostraciones teleoperadas **sobre el robot real** abriendo y cerrando una válvula, con el tren inferior sostenido por el controlador de equilibrio de serie y solo los brazos gobernados por la política.
- Entrenar **una política ACT** con las herramientas de Unitree para LeRobot y su sistema de teleoperación, siguiendo la receta que el fabricante ya había validado en sus propios conjuntos de datos.
- Evaluar la tasa de éxito **sobre el robot**, con variación del ángulo inicial de la válvula, en unos cincuenta a cien episodios.
- Como ampliaciones si sobraba tiempo: la mano diestra Inspire y una comparación con otros métodos, GR00T entre ellos, explícitamente descartados como núcleo por «más pesados y más exigentes en datos».
- Fuera de alcance: el aprendizaje por refuerzo, la comparación entre simuladores y la locomanipulación completa. **La simulación quedaba limitada a comprobaciones cualitativas**, con el argumento de que la transferencia de simulación a realidad en manipulación con contacto sostenido era un problema abierto.

El plan tenía ocho semanas: alcance y pruebas del controlador de serie (1), captura del conjunto de datos (2), primer entrenamiento y despliegue en el robot (3), iteración y primera demostración (4), evaluación formal (5), ampliación o análisis de fallos (6), figuras y vídeos (7) y redacción (8).

B.3. Planificación final

Lo ejecutado se aleja del plan en su premisa central y coincide con él en casi todo lo demás. La figura del capítulo 2 da la versión simulada; lo real fue esto.

Julio: investigación y puesta en marcha. Se investigó la pila de control de cuerpo completo y teleoperación con las gafas —el sistema de NVIDIA para control de cuerpo completo con el servicio de PICO y MuJoCo como simulador de pruebas—, y se descartó en favor de Isaac Lab Arena, que traía integrados el *embodiment* del G1 con su política de equilibrio, la teleoperación en realidad mixta, la grabación y la evaluación. Se resolvió el acceso remoto a la estación y el visor de la simulación, se verificó la teleoperación de la tarea con el emulador web de las gafas (23 de julio) y se cargó por primera vez un fondo fotorrealista en la escena (27 de julio).

Del 30 de julio al 4 de agosto: la reconstrucción de la oficina. Primer intento con la herramienta de captura de NVIDIA, reconstrucción a partir de 101 fotografías (3 de agosto) y, al día siguiente, la definitiva a partir de vídeo (anexo D). El 4 de agosto quedaron conectadas las gafas físicas, con un problema abierto entre el fondo fotorrealista y la realidad mixta que acabaría resolviéndose no en la escena de captura sino aplicando el fondo después.

Del 17 al 25 de agosto: el desarrollo. Nueve días de trabajo concentrado en los que se tomaron y, en varios casos, se revirtieron las decisiones de diseño (tabla B.1): la integración de la válvula y la primera demostración con imagen (18), el ensayo previo del *pipeline* completo (19–20), el fondo aplicado por reproducción y su corrección (20–21), las cuatro sesiones de captura del conjunto definitivo (21–22), la comparación principal (22 y 24) y los dos experimentos secundarios (25).

Del 25 de agosto al 5 de septiembre: la memoria, sobre la plantilla de la titulación, con el borrador enviado al tutor para su revisión.

Dos desviaciones alteraron el plan más que las demás. La primera es **el paso a la simulación**, que cambió la pila entera pero hizo posible lo que el plan original no habría podido ofrecer: doscientas tiradas emparejadas por política, seis entrenamientos sobre subconjuntos estratificados y una medida de latencia en lazo cerrado, todo ello en noches de máquina. La segunda es **la reducción del conjunto de datos** de cuatrocientas a cien demostraciones, que no fue una renuncia sino una consecuencia de medir: la curva de eficiencia dijo que el tamaño no era el factor limitante, y el tiempo previsto para grabar se dedicó a los experimentos que sí respondían a la pregunta.

Tabla B.1: Desviaciones entre la planificación inicial y la final

Previsto	Realizado	Causa de la desviación
Grabar y evaluar sobre el G1 físico	Todo en simulación	Se disponía del robot, pero no de un espacio acondicionado para experimentar con seguridad ni del <i>firmware</i> , los controladores y el entorno de despliegue para inferencia en tiempo real
Una sola política, ACT	Comparación ACT frente a GR00T	Arena integra GR00T para este mismo robot; la comparación pasó a ser la pregunta del trabajo
Herramientas de Unitree para LeRobot y su teleoperación	Isaac Lab Arena, CloudXR y PICO 4 Ultra	Con la simulación cambió la pila: Arena trae <i>embodiment</i> , equilibrio, teleoperación, grabación y evaluación
50–100 demostraciones; después, 400 (17 de agosto)	100	La curva de eficiencia mostró que 25 ya saturan; grabar más del mismo reparto no mejoraba nada
Fondo fotorrealista fuera de alcance (17 de agosto)	Dentro, aplicado por reproducción (19 de agosto)	Reduce la distancia entre las imágenes de entrenamiento y las de un despliegue; su coste de render se evita aplicándolo después
Robot de pie bajo su controlador de equilibrio	Pelvis fijado (18 de agosto)	El par de reacción del volante giraba al robot entero durante la teleoperación
Empujar el volante con la mano abierta	Agarrar un radio con el gatillo (21 de agosto)	Más defendible en el contexto industrial; obligó a fijar el gesto y a descartar la primera sesión como datos
Presupuesto de episodio de 15 s	30 s en captura, 20 s en evaluación	Un episodio que agota el tiempo no se escribe; el presupuesto solo alarga los fallos
100 tiradas por política	200, en dos bloques	Con 100 la diferencia quedaba en $p = 0,134$; doblar la muestra es lo que corresponde ante una diferencia pequeña
Mano diestra Inspire como ampliación	Descartada	El modelo de G1 con esa mano rompería el equilibrio, el <i>retargeting</i> y el espacio de 43 articulaciones
Plazo de dos meses desde julio	Memoria en septiembre	La puesta en marcha de la pila de realidad mixta y la reconstrucción consumieron julio y principios de agosto

Anexo C

Protocolo de evaluación

La plantilla reserva este anexo a los cuestionarios empleados durante la fase de evaluación. En este trabajo **no procede**: la evaluación no se basa en la valoración subjetiva de usuarios, sino en la ejecución automatizada de casos de prueba dentro del simulador, con un criterio de éxito medido por el propio entorno. En su lugar se documenta aquí el protocolo experimental completo, que cumple la misma función de permitir que un tercero reproduzca la evaluación. Los parámetros del experimento —semillas, presupuesto, criterio de éxito, instrucción— están en la tabla 4.1 del capítulo 4; aquí van las condiciones operativas, el procedimiento y el análisis.

C.1. Condiciones de la evaluación

Cada una de esas condiciones viene de un fallo que costó tiempo, y las tres que más importan son las que no dan error si se incumplen: evaluar con una iluminación distinta de la de entrenamiento —la política recibe la mitad de luz en su única entrada visual y simplemente lo hace peor—, servir ACT en la segunda GPU por su índice —los búferes de normalización quedan a cero y la política emite ceros— y dejar el tiempo de espera del cliente en su valor por defecto.

C.2. Procedimiento

Una tirada de evaluación de N episodios sigue estos pasos, que son los que ejecuta el guion `eval_valve_100.sh` sin supervisión:

1. **Arrancar el servidor de la política** en su entorno de ejecución. Para GR00T, un fichero de configuración con la ruta del punto de control; para

Tabla C.1: Condiciones operativas de toda tirada de evaluación

Condición	Valor y motivo
Simulador	Contenedor <code>isaacclab_arena-latest</code> , física y render en la GPU 0, un único entorno paralelo
Servidor de política	Proceso en su propio entorno de ejecución en el anfitrión, en la GPU 1; para ACT, seleccionada por variable de entorno y nunca por índice de dispositivo
Comunicación	ZMQ sobre la interfaz local, puerto 5561; tiempo de espera del cliente de 300 s, porque el valor por defecto de 15 s aborta el episodio
Fondo e iluminación	<code>office_gs</code> con la intensidad de la luz de cúpula sin fijar, porque así se generaron las imágenes de entrenamiento
Base del robot	Pelvis fijado; canales de locomoción sin uso
Presupuesto	20 s por episodio, 1000 pasos a 50 Hz
Registro	Activado con <code>ARENA_EVAL_RECORD=1</code> ; sin cámaras en el fichero de resultados, para que cien episodios ocupen megabytes y no gigabytes
Vídeo	Tirada aparte de cinco episodios con el visualizador y la cámara de tercera persona; no se mezcla con la de métricas

ACT, el punto de control, las estadísticas del conjunto de datos y la GPU por variable de entorno. El servidor de ACT comprueba al arrancar que los búferes de normalización no son nulos y se niega a servir si lo son.

2. **Esperar** a que el registro del servidor diga `listening on tcp://`. Lanzar el cliente antes agota el tiempo de espera.
3. **Lanzar el cliente** en el contenedor con el comando de evaluación —idéntico para toda política—, el número de episodios y la semilla. El cliente reinicia el entorno, sortea la válvula, pide bloques de cuarenta acciones al servidor y avanza hasta el éxito o el presupuesto.
4. **Recoger el fichero de resultados**, que el simulador escribe en su directorio de registros, y moverlo al directorio de resultados con el nombre de la política y del bloque.

5. **Detener el servidor** desde el anfitrión y el cliente desde dentro del contenedor, donde corre como *root*; un *pkill* desde fuera no tiene permiso.
6. Repetir del paso 1 para la otra política **con la misma semilla**, que es lo que garantiza el emparejamiento; y comprobar, leyendo las poses iniciales de ambos ficheros, que coinciden.

Para la curva de eficiencia el mismo procedimiento se encadena seis veces —tres subconjuntos por política— en el guion de la noche final, que escribe un centinela por etapa y por eso puede relanzarse sin repetir lo hecho. Para la latencia, el servidor se envuelve en una subclase que cronometra cada llamada con sincronización de GPU a ambos lados y escribe una línea por bloque de acciones; la subclase debe redefinir el constructor de línea de comandos de la clase base, que de otro modo instancia la política sin cronometrar.

C.3. Registro de resultados

El fichero de resultados es un HDF5 con un grupo por episodio, escrito por los mismos registradores que el grabador de demostraciones. La tabla C.2 recoge los campos y, a continuación, las magnitudes que el análisis deriva de ellos.

Tabla C.2: Campos registrados por episodio

Campo	Forma	Contenido
<code>success</code>	(1)	Éxito del episodio, booleano
<code>revolute_joint_state</code>	(T)	Apertura normalizada de la válvula en cada paso
<code>actions</code>	(T , 50)	Acción enviada al entorno: 43 articulaciones y 7 canales de base
<code>processed_actions</code>	(T , 43)	Objetivos articulares tras el controlador
<code>obs/robot_joint_pos</code> , <code>obs/robot_joint_vel</code>	(T , 43)	Estado articular observado
<code>obs/*_wrist_pose_pelvis_frame</code>	(T , 4, 4)	Pose de cada muñeca en el sistema del pelvis
<code>initial_state/articulation/*</code>	(1, ·)	Estado inicial completo del robot y de la válvula: posiciones y velocidades articulares, pose y velocidad de la raíz
<code>states/articulation/*</code>	(T , ·)	El mismo estado, en cada paso
atributos	—	<code>num_samples</code> (T) y <code>success</code>

De ahí se derivan, por episodio: la **disposición**, a partir del cuaternión de la pose inicial de la válvula —una componente de módulo 0,707 identifica la frontal; cuatro componentes de 0,5, la cenital—; el **ángulo máximo** alcanzado, como el máximo de la apertura por 360°; si la válvula **se movió**, como apertura máxima superior a 0,05; el **paso en que se cruza el umbral**, como el primer índice con apertura mayor que 0,5; y la **duración**, T . La verificación del emparejamiento compara `initial_state` de la válvula entre las dos políticas, episodio a episodio.

C.4. Análisis estadístico

Las tasas se acompañan de intervalos de confianza al 95 % por el **método de Wilson**, que para k éxitos en n tiradas centra el intervalo en $(\hat{p} + z^2/2n)/(1 + z^2/n)$ con $z = 1,96$ y se mantiene dentro de $[0, 1]$ cuando $\hat{p}$ se acerca a los extremos, que es el caso aquí.

La comparación entre políticas usa la **prueba exacta de McNemar**: con b episodios que resuelve solo GR00T y c que resuelve solo ACT, bajo la hipótesis nula de igualdad cada caso discordante cae a un lado u otro con probabilidad 1/2, y el valor p bilateral es la probabilidad binomial de un reparto al menos tan desigual como el observado. Los episodios que resuelven ambas o ninguna no entran, porque no llevan información sobre la diferencia. La sensibilidad del resultado se comprueba recalculando p para repartos que difieren en un episodio.

El efecto de la disposición dentro de una misma política se contrasta con la **prueba exacta de Fisher** sobre la tabla de dos por dos de disposición frente a éxito, porque compara dos grupos distintos de episodios y no hay emparejamiento que aprovechar. Las diferencias dentro de una misma política entre puntos de la curva de eficiencia vuelven a ser emparejadas —los seis entrenamientos se evaluaron sobre las mismas cien condiciones iniciales— y usan McNemar.

Todo ello lo calculan los guiones de figuras del repositorio, que leen directamente los ficheros de resultados y escriben, junto a las figuras, un informe en texto con cada cifra y cada valor p ; la primera llamada de cada política se descarta en la medida de latencia por ser compilación de núcleos. Ninguna cifra del capítulo 4 se ha calculado a mano.

Anexo D

Reconstrucción fotorrealista mediante Gaussian Splatting

Este anexo desarrolla el módulo de reconstrucción fotorrealista de la escena introducido en el capítulo 3. Es una **contribución instrumental** del proyecto, no un objetivo del trabajo: da soporte al *pipeline* de captura y generación de datos aportando el fondo sobre el que se entrenan y evalúan ambas políticas, pero ninguna conclusión del capítulo 4 depende de sus detalles. Se documenta aquí porque supuso un esfuerzo de ingeniería considerable y porque uno de los problemas que planteó invalidó silenciosamente una sesión completa de datos antes de detectarse. Se describe con el nivel de detalle necesario para reproducirlo, no más; el propio proyecto de reconstrucción conserva la receta paso a paso.

D.1. El activo y su integración en la escena

La herramienta principal es **fVDB** [21], el marco de NVIDIA para aprendizaje profundo sobre datos espaciales dispersos, a través de su caja de herramientas de captura de la realidad, *fVDB Reality Capture* [22], que implementa la reconstrucción por *Gaussian Splatting* [20] y la exportación a los formatos que Isaac Sim lee. Las poses de cámara y la nube de puntos inicial de la que parte la reconstrucción las produce **COLMAP** [23], la referencia en *Structure-from-Motion*, compilado con soporte CUDA para la arquitectura de las aceleradoras de la estación. El resto es un puñado de guiones propios entre una etapa y la siguiente.

Hubo dos reconstrucciones de la oficina. La primera, el 3 de agosto de 2026, a partir de 101 fotografías de un teléfono móvil, sirvió para montar y validar el *pipeline*

entero, incluida una malla de colisión que finalmente no hizo falta —la física de la escena es un plano de suelo y la válvula, y el activo se usa solo como fondo visual—. La definitiva, al día siguiente, partió de un **vídeo** de la misma oficina, y es la que llega al conjunto de datos. Sus etapas y cifras:

1. **Fotogramas.** Del vídeo se extraen 1089 fotogramas y se seleccionan 545 con una ventana deslizante que se queda con el más nítido de cada grupo —nitidez medida como la varianza del laplaciano—, en lugar de un umbral global, que descartaría bloques enteros justo donde la cámara giraba deprisa y rompería el emparejamiento secuencial.
2. **Structure-from-Motion.** COLMAP con un único modelo de cámara compartido, extracción y emparejamiento de características en GPU, y reconstrucción incremental: **538 imágenes registradas** en un solo modelo, 142 619 puntos tridimensionales, longitud media de traza 8,3 y error de reproyección medio de 1,18 píxeles; después, las imágenes se rectifican a un modelo de cámara sin distorsión.
3. **Escala métrica y vertical.** COLMAP no conoce la escala. Un guion propio estima la vertical a partir de la orientación de las cámaras, ajusta un plano de suelo por RANSAC sobre los puntos más bajos —tras un recorte robusto por percentiles, sin el cual los puntos triangulados a través de ventanas y reflejos inflaban la escena hasta dimensiones absurdas—, rota la escena a Z arriba con el suelo en $Z = 0$ y la escala fijando la altura media de las cámaras en 1,5 m. Es una escala aproximada, con un error del orden del 5 %, suficiente para un fondo. Hacerlo **antes** de entrenar, y no corregirlo después sobre el activo exportado, es la decisión que evita toda una familia de problemas de matrices.
4. **Reconstrucción.** `frgs reconstruct` sin normalización de escena —para conservar las unidades—, doscientas épocas sobre las 538 imágenes, 107 600 pasos en unos treinta minutos en una GPU: **3 962 281 gaussianas**. Un guion propio las reduce después a 2 144 018 eliminando las de opacidad inferior a 0,005 —el refinado de la herramienta deja de podar a mitad del entrenamiento— y las de escala desproporcionada, que son pocas, suelen ser artefactos de zonas mal observadas y cuestan en render lo que un tile entero.
5. **Exportación.** `frgs convert` al formato **NuRec** de Omniverse, en lugar del formato *ParticleField* más reciente, porque sobre la misma reconstrucción Nu-

Rec se ve claramente mejor en Isaac Sim 6.0 y aquí solo importa cómo se ve desde la cámara del robot; la contrapartida es que NVIDIA lo va a retirar.

6. **Envoltura e integración.** Una capa USD propia referencia el activo exportado y lo endereza y coloca —una traslación y una rotación de -89° en X , verificadas contra el dato: las 538 cámaras quedan a 1,50 m de altura media y el suelo a $0,58^\circ$ de la horizontal—, añade un plano de colisión de 10×10 m en $Z = 0$ y una escena de física. El parche sobre la biblioteca de fondos del marco lo registra como fondo `office_gs`, con una rotación de $+90^\circ$ en Z para orientarlo respecto al robot y con variables de entorno para probar otro fichero, escala o pose sin tocar código.

Cuatro trampas costaron más que el resto del proceso junto, y ninguna produjo un error. **El exportador deja el activo tumbado:** escribe en la primitiva del volumen una «matriz de conversión por defecto» pensada para datos con Y arriba, que sobre un dato ya en Z arriba aplica una rotación que no hace falta; como esa matriz es su propia inversa, la corrección exacta es ponerla a identidad, no buscar ángulos a ojo. **USD ignora los ajustes de render que el activo trae consigo** cuando se referencia desde otra capa, de modo que el fichero de la tarea los reaplica como ajustes del simulador cuando el fondo lleva la etiqueta `nurec`. **El orden del cuaternión** de la pose en el marco es $xyzw$, como dice el nombre del campo y al contrario de lo que afirmaba una nota anterior; la identidad es $(0, 0, 0, 1)$, y escribirla como $wxyz$ tumba el fondo 89° . Se midió sobre el escenario ya compuesto con un guion propio, tras tres intentos fallidos de predecirlo, porque hay tres capas de composición encima del fichero de origen y con tres capas conviene medir. Y **el fondo heredado que existía al empezar** —una oficina distinta, exportada con otra herramienta y de la que no se conservaban ni las imágenes ni el espacio de trabajo— arrastraba una matriz transpuesta que USD lee como proyectiva y que deformaba el activo en cuanto se le aplicaba una pose; es la razón de que se capturara de nuevo en lugar de reutilizarlo.

D.2. El procedimiento de reproducción sobre demostraciones grabadas

El fondo no se renderiza durante la teleoperación: cuesta unos 39 ms por paso de simulación, y en las gafas eso es la diferencia entre poder agarrar un radio del volante y no poder. Las demostraciones se graban en la escena diáfana y el fondo se aplica después, sin gafas y sin nadie delante, con `render_demos.py`: reproduce

cada demostración con el fondo cargado, vuelve a grabar la imagen de la cámara de la cabeza y escribe un nuevo HDF5 con la misma estructura. Cuesta unos doce minutos por sesión de veinticinco demostraciones, y el fichero resultante ocupa aproximadamente el doble que el original —110 MB por demostración frente a 60— porque la oficina comprime peor que un plano gris. Se comprobó que el fondo **no perturba la física**: la misma demostración dura los mismos pasos con y sin él. El éxito se recalcula sobre la reproducción, no se copia del original, y se verifica con `compare_rerender.py` que cada demostración conserva su número de pasos, cruza el umbral y ha cambiado de imagen.

D.3. La divergencia de estado y su corrección

La primera versión reproducía las acciones grabadas en lazo abierto, y **no reproducía la demostración**. Se descubrió el 21 de agosto y había invalidado ya la salida de una sesión entera: de veinticinco demostraciones que abrieron la válvula al teleoperar, solo dieciséis la abrían al reproducirse, y la peor pasaba de $191,1^\circ$ a $113,7^\circ$. Sobre la primera sesión, frontal y sin agarre, la misma reproducción había perdido una de veinticinco y había parecido correcta.

La opción `-validate_states` de la herramienta, que compara en cada paso el estado grabado con el reproducido, localizó la causa con exactitud: la divergencia aparece en el **paso 1**, no gradualmente, y no en las posiciones articulares sino en las **velocidades**, en los índices 14 y 18 —el cabeceo y el alabeo del tobillo derecho—. Con eso la explicación es completa: el HDF5 almacena la acción de veintitrés dimensiones, que gobierna los brazos y los canales de mando, pero **las piernas las gobierna la política de equilibrio en lazo cerrado y no se reproducen: se vuelven a ejecutar**. Aterrizan en otro sitio desde el primer paso, el tronco acaba ligeramente distinto, las muñecas agarran el radio por otro punto y al final del episodio el volante se queda hasta 80° corto. Empujar de frente con la mano abierta tolera esa deriva; agarrar un radio desde arriba, no.

La corrección no es hacer determinista la física, sino **no simular**: el HDF5 graba el estado completo de la escena en cada paso, de modo que la herramienta lo impone tras cada `step()` en lugar de dejar correr los controladores. Medido sobre las cincuenta demostraciones de las sesiones 02 y 03: **50 de 50 reproducen, con una diferencia máxima de $1,1^\circ$** en el ángulo final, frente a los 81° anteriores, y sin fotogramas negros. El residuo de $1,1^\circ$ es un paso de física: el fotograma se renderiza dentro de `step()`, justo antes de imponer el estado. Dos trampas de implementa-

ción: hay que usar `env.scene.reset_to()` y nunca `env.reset_to()`, que además exporta un episodio y **vuelve a sortear la disposición de la válvula en cada paso**; y en algunos caminos los datos de la articulación llegan como *arrays* de Warp, que no admiten indexado escalar y fallan después de dos minutos de carga, por lo que se normalizan antes. La opción `-libre` conserva el comportamiento antiguo, solo para comparar.

D.4. Parámetros y configuración

Tabla D.1: Parámetros del fondo fotorrealista y versiones de las herramientas

Parámetro o herramienta	Valor
OFFICE_GS_USD	Ruta del activo; por defecto el de la oficina de Madrid reconstruida a partir del vídeo
OFFICE_GS_ROT, _POS, _SCALE	Pose y escala del fondo, cuaternión en orden <i>xyzw</i> ; por defecto $+90^\circ$ en <i>Z</i> , sin traslación, escala 1
OFFICE_GS_LIGHT	Intensidad de la luz de cúpula, 3000 por defecto. Solo se lee con el fondo fotorrealista; todas las reproducciones la dejan sin fijar, porque fijarla en una sesión y no en otra parte el conjunto de datos en dos exposiciones
Ajustes de render	Los que trae el activo NuRec, reaplicados por la tarea como ajustes del simulador
fvDB	fvdb-core 0.5.1 y fvdb-reality-capture 0.5.0, sobre PyTorch 2.11 con CUDA 12.8, en un entorno propio con Python 3.12
COLMAP	4.2 de desarrollo, compilado con CUDA para la arquitectura sm_120; el de la distribución no trae CUDA y sus opciones tienen otros nombres
Isaac Sim	6.0, con la transmisión de geometría activada, sin la cual las gaussianas se muestran como esferas grises

Los guiones implicados, todos en el repositorio:

`rerender_demos.py` la reproducción con el estado grabado impuesto.

`compare_rerender.py` la verificación de la reproducción contra el original.

`hdf5_to_video.py` la revisión a ojo, sin arrancar el simulador.

`measure_gs_pose.py` la medida de la pose real del fondo en el escenario compuesto.

`capture_viewport.py` la comprobación del render, antes de culpar a cualquier otra cosa.